

UNIVERSITÀ DEGLI STUDI DI PISA
FACOLTÀ DI INGEGNERIA
CORSO DI LAUREA IN INGEGNERIA INFORMATICA

TESI DI LAUREA

DIFFSERV SIMULATIONS USING THE
NETWORK SIMULATOR:
REQUIREMENTS, ISSUES AND SOLUTIONS

CANDIDATO
SERGIO ANDREOZZI

RELATORI
PROFESSOR LUCIANO LENZINI
PROFESSOR LUIGI RIZZO
PROFESSOR JARI PORRAS
M.SC. KARI HEIKKINEN

ANNO ACCADEMICO 2000/2001

ABSTRACT

Author	Sergio Andreatti
Title	DiffServ simulations using the Network Simulator: requirements, issues and solutions
Year	2001
Master's Thesis	116 pages, 46 figures, 10 tables and 3 appendices
Supervisors	Professor Luciano Lenzini Professor Luigi Rizzo Professor Jari Porras M. Sc. Kari Heikkinen
Keywords	DiffServ, QoS, simulation, Network Simulator

IP networks are destined to become the ubiquitous global communication infrastructure. An increasing number of different applications are continually conveyed by them causing a fragmentation of performance and service requirements. Continuous efforts have been made to develop a number of new technologies for enhancing Quality of Service capabilities. The DiffServ Architecture is the traffic management scheme defined by IETF to provide scalable services differentiation on the Internet.

This thesis work aims to explore and to improve the simulations capabilities of the *Network Simulator (NS)*, in addition the traffic management skills of that new architecture are evaluated. The main focus is on the versatility of building complete service models and on providing a wide range of performance parameters.

Key contributions reached during this work are: firstly, an improved simulation module with a wider set of mechanisms, a better versatility of services definition and an extended set of performance parameters. Secondly, valuable test beds for mechanisms testing are set. In particular, a complete service model with a Premium Service targeted for conversational applications like Voice over IP, an Olympic Services sub-model targeted for relative services differentiation and a Best Effort Service for backward compatibility is configured, relative traffic generators are set and performance parameters are monitored.

SOMMARIO

Autore	Sergio Andreozzi
Titolo	Simulazioni dell'architettura DiffServ con l'ausilio del Network Simulator: requisiti, problematiche e soluzioni
Anno	2001
Luogo	Lappeenranta
Tesi di Laurea	116 pagine, 46 figure, 10 tabelle e 3 appendici
Relatori	Professor Luciano Lenzini Professor Luigi Rizzo Professor Jari Porras M. Sc. Kari Heikkinen
Parole chiave	DiffServ, QoS, simulazione, Network Simulator

Le reti basate sul protocollo IP stanno evolvendo verso una infrastruttura di comunicazione globale dove un sempre crescente numero di applicazioni viene veicolato. Questa evoluzione sta causando una frammentazione dei servizi e delle prestazioni richieste alle reti stesse. Sforzi vengono continuamente fatti per migliorarne la capacità di fornire la Qualità del Servizio desiderata. In questo contesto, l'architettura DiffServ si inserisce come un nuovo schema di gestione del traffico di rete definito dall'IETF allo scopo di realizzare una differenziazione dei servizi scalabile all'interno di Internet.

Questa tesi ha come scopo sia l'esplorazione ed il miglioramento delle capacità di simulazione del *Network Simulator* (NS), sia la valutazione delle capacità di gestione del traffico della nuova architettura. Particolare attenzione è rivolta alla versatilità nel definire nuovi modelli di servizio e alla possibilità di valutarne le prestazioni.

Diversi sono i risultati ottenuti: sensibili miglioramenti sono stati apportati al simulatore introducendo un insieme di nuovi meccanismi e funzionalità che hanno reso possibile la simulazione di modelli di servizio completi e la relativa valutazione delle prestazioni; inoltre alcuni scenari sono stati esplorati, in particolare è stato implementato un modello comprendente un servizio di tipo Premium per applicazioni quali Voice over IP, comprendente i servizi del modello Olympic e comprendente un servizio di tipo Best Effort per la compatibilità all'indietro; i relativi generatori di traffico sono stati configurati ed i relativi parametri di qualità sono stati monitorati.

TIIVISTELMÄ

Tekijän nimi	Sergio Andreatozzi
Työn nimi	Diffserv-simulaatiot NS-simulaattorilla: vaatimukset, kysymykset ja ratkaisut
Työn valmistumisvuosi	2001
Diplomityö	116 sivua, 46 kuvaa, 10 taulukkoa ja 3 liitettä
Tarkastaja	Professori Luciano Lenzini Professori Luigi Rizzo Professori Jari Porras M. Sc. Kari Heikkinen
Hakusanat	DiffServ, QoS, simulaatio, Network Simulator

IP-verkoista on tulossa maailmanlaajuinen verkkotekniikka, joka on läsnä kaikkialla. Kasvava määrä erilaisia sovellutuksia tuodaan Internet-verkkoon, joten käytössä oleva kapasiteetti pirstaloituu koko ajan. Erilaisia organisaatioissa, kuten esim. IETF, on tehty jatkuvaa kehitystyötä Internet-verkon palvelunlaadun parantamiseksi. Näistä ehkä merkittävin liikenteenhallintatekniikka on Diffserv-arkkitehtuuri, jolla mahdollistetaan skaalattavat differentioidut palvelut Internet-verkoissa.

Tämä diplomityö käyttää välineenään simulointia, työkaluna NS-simulaattori, simuloidakseen Diffserv-arkkitehtuurin menetelmiä erilaisille sovellutuksille. Työn pääpaino on monimuotoisten palvelumallien kehityksessä ja kyseisten mallien parametrien simuloinnissa.

Työn empiirisen osan tuloksena on paranneltu NS-simulaattorin Diffserv-moduulia; Moduuliin lisättiin mekanismeja, joilla voidaan paremmin toteuttaa monimuotoisten sovellutuksien määrittelyjä. Lisäksi määriteltiin lisäparametreja, joilla sovellutusten suorituskykyä voidaan parantaa. Tätä varten rakennettiin simulaattoriin testiverkko. Tässä testiverkossa simuloitiin palvelumallia, joka koostui korkeasta luokasta, ns. olympiamallista ja perinteisestä Internet-liikenteestä. Korkeammassa luokassa ovat sovellutukset kuten reaaliaikaiset äänipalvelut ja olympiamalli on suunniteltu palvelujen differentioitiin. Tässä palvelumallissa simuloitiin ja mitattiin suorituskyky-parametreja.

FOREWORD

This thesis, prepared at Lappeenranta University of Technology (Finland), is the conclusion to a series of enriching experiences acquired throughout my University course.

First, I would like to express my gratitude to my parents, who have given me the opportunity to follow my natural inclinations and to have provided me with both moral and material support.

Then I want to thank Professor Luciano Lenzini, Professor Luigi Rizzo and Professor Jari Porras for their faith in me and their suggestions.

A particular acknowledgement is for Kari Heikkinen who led this work and for Marie Labbe who smoothed out the English language expressions.

I really appreciated the critical comments and beneficial suggestions from Tiziana Ferrari about the simulation results and the help from Chen-Nee Chuah for the VoIP traffic model.

Finally, my thoughts go to those who have been really important to me in the last years: Roberta, Rosella, Monica V., Monica M., Silvia, Elisa and, last but not least, Gabriella.

Sergio Andreatti

LIST OF ABBREVIATIONS AND ACRONYMS

AF	Assured Forwarding
ATM	Asynchronous Transfer Mode
CBQ	Class Based Queueing
DIFFSERV	Differentiated Services
DS	Differentiated Services
EF	Expedited Forwarding
IESG	Internet Engineering Steering Group
IETF	Internet Engineering Task Force
IP	Internet Protocol
IPDV	IP Packet Delay Variation
LLQ	Low Latency Queueing
MPLS	MultiProtocol Label Switching
OWD	One Way Delay
PDB	Per Domain Behaviour
PHB	Per Hop Behaviour
QoS	Quality of Service
RED	Random Early Detection
RSVP	resource ReSerVation Protocol
SCFQ	Self-Clocked Fair Queueing
SFQ	Start-time Fair Queueing
SLA	Service-Level Agreement
SLS	Service-Level Specification
TCA	Traffic Conditioning Agreement
TCP	Transmission Control Protocol
TCS	Traffic Conditioning Specifications
ToS	Type of Service
UDP	User Datagram Protocol
VoIP	Voice over IP
WFQ	Weight Fair Queueing
WRED	Weighted Random Early Detection

TABLE OF CONTENTS

INTRODUCTION

1. TRAFFIC MANAGEMENT BEFORE DIFFSERV

- 1.1 A broad view network model
- 1.2 The Internet Service Provision before DiffServ
 - 1.2.1 The Best Effort Model
 - 1.2.2 Integrated Service Model
 - 1.2.3 IP over ATM
- 1.3 Comparison among these models

2. THE DIFFSERV MODEL

- 2.1 Design principles
- 2.2 Differentiated Services framework
 - 2.2.1 Differentiated Services field definition
 - 2.2.1.1 Backwards compatibility issues for the IPv4 ToS field
 - 2.2.1.2 The DS field structure
 - 2.2.2 Traffic classification and conditioning
 - 2.2.3 Per-Hop Behaviors
 - 2.2.3.1 A default PHB
 - 2.2.3.2 Class Selector PHB's
 - 2.2.3.3 Expedited Forwarding PHB
 - 2.2.3.4 Assured Forwarding PHB Groups

3. DIFFSERV SIMULATIONS USING *NS*

- 3.1 The need for simulation
- 3.2 Conceptual model for a DiffServ router
 - 3.2.1 Hierarchical view of the model
 - 3.2.2 Basic building blocks of a DiffServ router
- 3.3 The Network Simulator (*NS*)
 - 3.3.1 *NS* architecture
 - 3.3.1.1 Network topology
 - 3.3.1.2 The delay model

3.3.2 DiffServ support

3.3.3 DiffServ module improvements

3.3.3.1 Router design

3.3.3.2 Measurements and performance analysis

4. SIMULATION SCENARIOS AND RESULTS

4.1 Scenario 1: schedulers comparison for delay and jitter sensitive traffic

4.2 Scenario 2: the level of importance differentiation in the AF PHB group

4.3 Scenario 3: a complete service model

5. CONCLUSIONS

5.1 Service models simulation capabilities of the Network Simulator

5.2 Performance parameters monitoring

5.3 Mechanisms test beds

6. FUTURE WORK

REFERENCES

APPENDIX A – Results diagrams for scenario 1

APPENDIX B – Results diagrams for scenario 2

APPENDIX C – Results diagrams for scenario 3

LIST OF FIGURES

Figure 1.1	A broad view network model
Figure 2.1	DiffServ Domain
Figure 2.2	IPv4 and IPv6 headers
Figure 2.3	The original IPv4 ToS octet
Figure 2.4	DS field definition
Figure 2.5	DiffServ Traffic Conditioning Block (TCB)
Figure 3.1	DiffServ router major functional blocks
Figure 3.2	Traffic conditioning and queueing elements
Figure 3.3	An example classifier
Figure 3.4	A general meter
Figure 3.5	Variants of RED
Figure 3.6	The Node structure
Figure 3.7	The Link structure
Figure 3.8	DiffServ module UML Class Diagram
Figure 3.9	DiffServ Traffic Conditioner Block in the <i>NS</i> core router object
Figure 3.10	DiffServ Traffic Conditioner Block in the <i>NS</i> edge router object
Figure 3.11	DiffServ+ module UML Class Diagram
Figure 4.1	Scenario 1: test bed
Figure 4.2	Scenario 2: test bed
Figure 4.3	WRED parameter sets
Figure 4.4	Scenario 3: test bed
Figure A.1	Relationship between EF Service rate and average OWD vs. packet size for WFQ scheduler – real network (source [48])
Figure A.2	Relationship between EF Service rate and average OWD vs. packet size for WFQ scheduler – simulated network
Figure A.3	Impact of BE packet size on EF OWD and IPDV for PQ scheduler – real network (source [48])
Figure A.4	Impact of BE packet size on EF OWD and IPDV for PQ scheduler – simulated network
Figure A.5	Comparison of average and frequency distributed OWD with WFQ and PQ for different EF frame packet sizes – real network (source [48])
Figure A.6	Comparison of average and frequency distributed OWD with WFQ and PQ for different EF frame packet sizes – simulated network
Figure A.7	Comparison of average and frequency distributed IPDV with WFQ and PQ - real network (source [48])

Figure A.8	Comparison of average and frequency distributed IPDV with WFQ and PQ – simulated network
Figure B.1	Telnet traffic aggregate performance parameters in absence of other AF traffic
Figure B.2	FTP traffic aggregate performance parameters in absence of other AF traffic
Figure B.3	HTTP traffic aggregate performance parameters in absence of other AF traffic
Figure B.4	Global parameters performance comparison – no DiffServ enabled
Figure B.5	WRED parameter set 1 (<i>staggered</i>)
Figure B.6	WRED parameter set 2 (<i>staggered</i>)
Figure B.7	WRED parameter set 3 (<i>partially overlapped</i>)
Figure B.8	WRED parameter set 4 (<i>partially overlapped</i>)
Figure B.9	WRED parameter set 5 (<i>overlapped</i>)
Figure B.10	WRED parameter set 6 (<i>overlapped</i>)
Figure B.11	Global parameters performance comparison
Figure C.1	Departure rate for n0->n466 egress interface per service category
Figure C.2	Departure rate for n0->n466 egress interface per traffic category
Figure C.3	Queue length per traffic category and MBS (only for VoIP)
Figure C.4	Olympic Services sub-model performance parameters
Figure C.5	OWD and IPDV comparison among VoIP and RealAudio (one monitored connection per traffic category)
Figure C.6	caPL and boPL per traffic category

LIST OF TABLES

Table 1.1	QoS schemes – comparison at a glance
Table 2.1	The DSCP Pools
Table 2.2	DiffServ AF codepoints table
Table 3.1	A BA classifier configuration example
Table 3.2	An MF classifier configuration example
Table 4.1	TCS for e1->core egress interface
Table 4.2	TCS for e1->core egress interface
Table 4.3	TCS for n0->n466 egress interface
Table 4.4	Numerical results
Table 4.5	TCS for n0->n466 egress interface

INTRODUCTION

IP networks are destined to become the ubiquitous global communication infrastructure. An increasing number of different applications are continually conveyed by them causing a fragmentation of performance and service requirements. Continuous efforts have been made to develop a number of new technologies for enhancing Quality of Service capabilities. The DiffServ Architecture is the traffic management scheme defined by IETF to provide scalable services differentiation on the Internet.

This thesis work aims to explore and to improve the simulations capabilities of the Network Simulator (*NS*), in addition the traffic management skills of that new architecture are evaluated. The main focus is on the versatility of building complete service models and on providing a wide range of performance parameters.

Key contributions reached during this work are: firstly, an improved simulation module with a wider set of mechanisms, a better versatility of services definition and an extended set of performance parameters. Secondly, valuable test beds for mechanisms testing are set. In particular, a complete service model with a Premium Service targeted for conversational applications like Voice over IP, an Olympic Services sub-model targeted for relative services differentiation and a Best Effort Service for backwards compatibility is configured, relative traffic generators are set and performance parameters are monitored.

In Chapter 1, a comparison among the former traffic management schemes is provided and the reasons that raised the need for a new model are pointed out.

Chapter 2 introduces the DiffServ model as defined by IETF. The whole forwarding architecture is depicted, on the basis of standardized and draft documents.

Chapter 3 reviews the implementation issues of Differentiated Services. Various suitable building blocks are recalled and simulation feasibility is checked. Several contributes are added to the simulator to improve network modelling capabilities, service model design and performance parameter collection.

In Chapter 4, three different scenarios are depicted. The first scenario is an attempt to understand how the simulator properly models a real IP network DiffServ enabled. The second scenario provides a test bed for algorithmic droppers targeted for AF group and for TCP-based traffic, moreover the best WRED parameter setting structure is evaluated. The third scenario aims to verify the flexibility of building complete service models and to provide a wide set of performance parameters.

1. TRAFFIC MANAGEMENT BEFORE DIFFSERV

To understand the current need for services differentiation and the related issues, it is useful to retrace the evolution of the QoS model since the Internet was born. The basic principles which inspired the design of the TCP/IP protocol suite were that [1]:

1. Each distinct network needs to stand on its own and no internal changes can be required to any such network to connect it to the Internet.
2. Communications should be on a best effort basis. If a packet did not make it to the final destination, it should shortly be retransmitted from the source.
3. Black boxes should be used to connect the networks; these have been later called gateways and routers. There should be no information retained by the gateways about the individual flows of packets passing through them, thereby keeping them simple and avoiding complicated adaptation and recovery from various failure modes.
4. There should be no global control at the operations level

These fundamental design principles and the idea of an open-architecture networking determined the success of the current Internet.

Since 1972, when the first email was sent, an increasing number of services has moved to IP networks. This has determined not only a traffic increment, but mostly a fragmentation of the network services needed to support these new applications.

The Best Effort Service, subject to unpredictable delays and data loss, does not fulfil these needs anymore. A new model capable of delivering differentiated QoS requirements is needed.

A possible definition of ideal QoS provision is *the activity to give guarantee that every flow is protected from all external effects in a way that the quality always satisfies the predefined requirements* [2]. The DiffServ model represents an important step toward this goal.

1.1 A broad view network model

The End-to-End QoS provision challenge involves the whole Internet in different ways. For instance, a required service from the end-user to the service provider generates a set of requirements inside the network. The worthy analysis presented in [2] helps us to better understand what the customer requirements set off.

In this analysis, the author considers the technological QoS provisioning issues in a wider Business Model. Now follows a brief presentation:

The basic entities of this model are:

- Service Provider: refers to both activities of customer relations and operation & management of the network.
- End-User: refers to any person who is using the Internet for any purpose; since a person has a variety of emotions that influence the use of the service, it is important to consider choices not only based on technical facts.
- Mechanism: refers to any piece of equipment or software that allows different politics of packet treatment.
- Network: refers to the basic entity managed by the service provider and used to transmit information from one end-user to another.
- Application: refers to both user applications and protocols that are not directly controllable by the service provider.
- Vendors: refers to companies that supply network components to service providers and end-users.

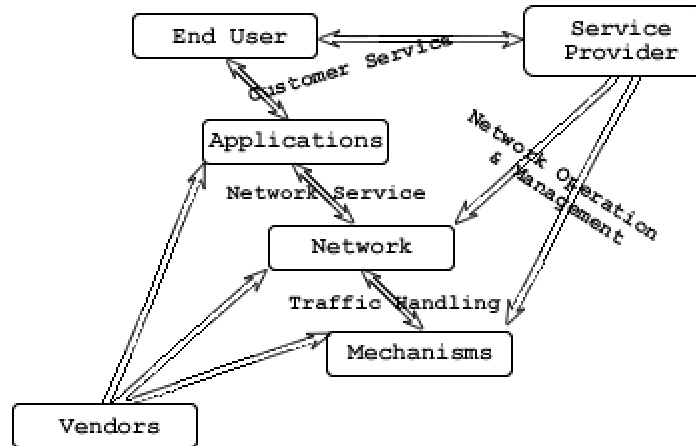


Figure 1.1 A broad view network model

It is worthy to analyse also the basic relationships among these entities. As displayed in Figure 1.1, *customer service* is used to describe the relationship between end user and service provider. Starting from the Service-Level Agreement (SLA) as the formal part that describes the type of service to be provided, this relationship passes through non technical aspects to finally reach customer satisfaction.

Network service is used to describe the relationship between applications and network characterized by parameters that enable the network to provide the applications of the needed resources. Then *traffic handling* is used to describe the relationship between network and mechanisms and represents all the elaborations a packet receives. With appropriate mechanisms (e.g. classification, marking, metering, shaping, dropping) it is possible to utilize the same network resources to better fulfil the low level services. Finally, *network operation & management* is the relationship between network, mechanism and service provision. It covers issues, such as building reasonable services based on available mechanisms, making traffic measurements for network-planning purposes and solving fault situations.

1.2 The Internet Service Provision before DiffServ

A brief description of the previous service models and a comparison among them will lead us to the foundations of the emerging DiffServ Model. The basic design principles and the implementation issues will be focused. The purpose is to understand the historical reasons for this new service model.

1.2.1 The Best Effort Model

The basic QoS model of the Internet is called Best Effort because the network tries to transmit as many packets as possible and as soon as possible, but does not give any guarantees. That means that the packets are subject to data loss, data duplication or out-of-order delivery. The TCP protocol solves these problems by assigning a sequence number to all data transmitted in the network and requiring a positive acknowledgment from the receiver.

Another key function of the TCP protocol is that no traffic-control action is taken until congestion is observed (the opposite approach is congestion avoidance). Receivers bind packet loss to congested network and control senders by returning a window size indicating a range of acceptable sequence numbers.

A further limitation of the Best Effort Model is that packets are delivered with unpredictable delays. This means that it cannot properly support real-time applications, except in case of low load levels and almost empty buffers. Even under these conditions, it is not possible to provide loss-free service or different levels of loss ratios because of the intrinsic traffic-control mechanism. This lack of versatility raised the need for a new QoS model.

1.2.2 Integrated Service Model

The need for a new Internet service model able to support Integrated Services (transport of audio, video, real-time, and classical data traffic within a single network infrastructure) led to the birth of the IntServ WG at the end of 1993. The WG focused on the definition of a minimal set of global requirements to change the Internet into a robust integrated-service communications infrastructure.

The basic issues were:

- defining the services to be provided
- defining the interfaces between applications and network services, routers and subnetworks
- developing router validation requirements which can ensure that the proper service is provided

The desired QoS is provided by resource reservation along the path, therefore signalling and state maintaining at each hop is needed. The resource ReSerVation Protocol used for signalling is RSVP [4]. Its major features include the use of “soft state” in the routers, receiver-controlled reservation requests and the use of IP multicast for data distribution.

The signalling and reservation of the desired QoS are needed for each flow in the network. A flow is defined as an individual, unidirectional data stream between two applications, and is uniquely identified by the 5-tuple (Source IP address, Source Port, Destination IP Address, Destination Port and the Transport Protocol).

Two types of services have been implemented:

1. Guaranteed Service [5]: traffic parameters (TSpec) using a token bucket and service-level parameters (RSpec) describe this strict Guaranteed Service that provides for firm bounds on end-to-end delay and assured bandwidth for traffic that conforms to the reserved specifications

2. Controlled Load Service [6]: traffic parameters (TSpec) using a token bucket describe this service for those applications that work well on unloaded nets, but which degrade quickly under overloaded conditions. The separation between pure Best Effort flows and Controlled Load flows is achieved by prioritization

The drawbacks of this model are:

1. the reservations in each device along the path are “soft”, which means that they need to be refreshed periodically; if refresh packets are lost there is a risk of reservation time out
2. the need for signalling and maintaining the state of each flow in each router is a strong limitation to scalability

An effort to solve these problems can be found in [7], an RFC document from the RSVP Working Group. Further efforts [8] are currently made inside the MPLS WG, where the ReSerVation Protocol has been involved to support signalling for the raising MPLS Model.

1.2.3 IP over ATM

ATM is the telecommunications transport technique selected for the efficient transfer of information across the network in the Broadband Integrated-Services Digital Network Model (B-ISDN). Efficient means that ATM should be able to fulfill the quality of service requirements of all supported connections over a network with a limited amount of resources. The B-ISDN model is an attempt to set up a single unified, worldwide high-speed network. It is conceived to support all types of existing and new emerging applications (data, voice, video and multimedia traffic in general).

The basic key concepts of ATM technology are:

- Connection-oriented
- Bandwidth reservation as a congestion avoidance mechanism
- Fixed-size packets called cells to enable high performances
- Wide service categories definition

Despite the wide standardization effort within the International Telecommunications Union and ATM Forum (since 1991), the B-ISDN Model has not widely succeeded. In fact, there are only a few real demonstrations of customer service based on end-to-end ATM connections.

The main historical reasons are:

- the initial lack of cost-effective interfaces for copper
- the initial lack of compatibility among network devices even if coming from the same vendors
- the fact that customer equipment is usually connected to a LAN and the impossibility of mapping application QoS requirements on the Ethernet level; (the IEEE 802.1D solved this problem, but now all efforts are concentrated on IP networks; therefore efforts are concentrated on mapping IP QoS requirements over IEEE 802.1D [9])

Conversely, ATM is widely used in the Internet backbone mainly as a primary intermediate layer between optical transmission and IP packet forwarding. Thus, nowadays it is regarded as one of the several technologies of the network layer for the Internet Architecture to deliver IP packets.

1.3 Comparison among these models

An interesting comparison comes from [2]. The author defines four attributes used to compare the service models:

1. Fairness. This is the key attribute of the relationship between the end-user and the service provider. Essentially, it is related to whether customers feel the market is fair, i.e. customers want to be sure that everyone pays the same price for the same product
2. Versatility. How the model is able to satisfy the different needs of customers
3. Robustness. This is related to the way the network is capable of managing the undesirable behavior of some end-users
4. Cost Efficiency. It is related to the ability of the model to meet the previous attributes at the lowest price

In the following table, the core part of the analysis is summarized. Refer to [2] for a complete description.

Table 1.1 QoS schemes - comparison at a glance

Model	Fairness	Versatility	Robustness	Cost Efficiency
Best Effort	Poor due to lack of robustness	Impossible to manage delays and packet-loss ratios	Weak because TCP-based traffic management runs in customers equipment	Good for adaptive applications; Poor for real-time applications
IntServ	It is difficult to build an understanding and consistent customer service	Good	Good	Limited on scalability
IP over ATM	Seems to be difficult to build fair services from the end-user point of view	Very good	Very good; needs attention in managing statistical multiplexing	Difficult in managing the network due to the complex QoS model

2. THE DIFFSERV MODEL

The B-ISDN brought about the idea of services differentiation. For the reasons explained above, IP networks were more successful than ATM networks. Inside the INTSERV WG, an attempt to provide services differentiation was made. The limitation of this model (scalability, flexibility in building customer services) raised the need for a new, and in a way, complementary model.

This is evident from the first paragraph of the first official document related to DiffServ Model [10]:

*“Differentiated services enhancements to the Internet protocol are intended **to enable scalable service** discrimination in the Internet without the need for per-flow state and signalling at every hop. A **variety of services may be built** from a small, well-defined set of building blocks which are deployed in network nodes.”*

Hence, scalability and flexibility on building customer service are the main targets of this QoS model. The next paragraphs focus on the design principles, the DiffServ architecture, the building blocks of DiffServ network and a conceptual model for a DiffServ router.

2.1 Design principles

The IESG approved the DiffServ Working Group [3] on 26 February 1998 and since that date, lots of efforts have been made to build a framework for Differentiated Services. The most recent overview of this framework can be found in [11]:

“The differentiated services framework enables quality-of-service provisioning within a network domain by applying rules at the edges to create traffic aggregates and coupling each of these with a specific forwarding path treatment in the domain through use of a codepoint in the IP header”

The framework tries to accomplish a list of requirements presented in [12] and summarized as follows [2]:

- *Versatility*: a wide variety of end-to-end services should be possible to realize; network services should be independent of applications, and they should be directly applicable with current applications and with current network services
- *Simplicity*: the overall system or parts of it should not depend on signalling for individual flows; only a small set of forwarding behaviours should be necessary
- *Cost efficiency*: information about individual flows or customers should not be used in core nodes. Only states of aggregate streams should be used in core nodes

A basic design choice was to decouple the fairly well-understood behaviour in the forwarding path from the more complex and still emerging background policy and allocation component that configures parameters used in the forwarding path. This aims to speed the deployment of the model in the network. Moreover, it enhances the flexibility in building new services because once the network interior is stable, it is possible to provide new services by the available set of forwarding behaviours and applying traffic conditioning policies at the edge.

Another design choice was to base the architecture on forwarding behaviours and therefore to decouple it from the control plane. This principle deals with the original design of the Internet where the decision was made to separate the forwarding and routing components. Even if it can lead to a non-complete network utilization (i.e. if multiple parallel or alternate paths are available, it is not possible to balance the

traffic load on the various links, routers and switches in the network), it provides the simplicity required to speed the deployment of the architecture. A solution to this drawback is currently discussed inside the MPLS WG and is named *Diff-Serv-aware MPLS Traffic Engineering* [13].

It is important to remark that the DiffServ architecture provides asymmetric service differentiation since it is related to only one direction of traffic flow [12].

2.2 Differentiated Services framework

The atomic entity defined by the framework to provide service differentiation is the DiffServ domain, *a contiguous portion of the Internet over which a consistent set of differentiated services policies are administered in a coordinated fashion* [10] (issues related to inter-domain QoS provisioning are currently discussed in the DiffServ WG). The next figure shows a possible scenario:

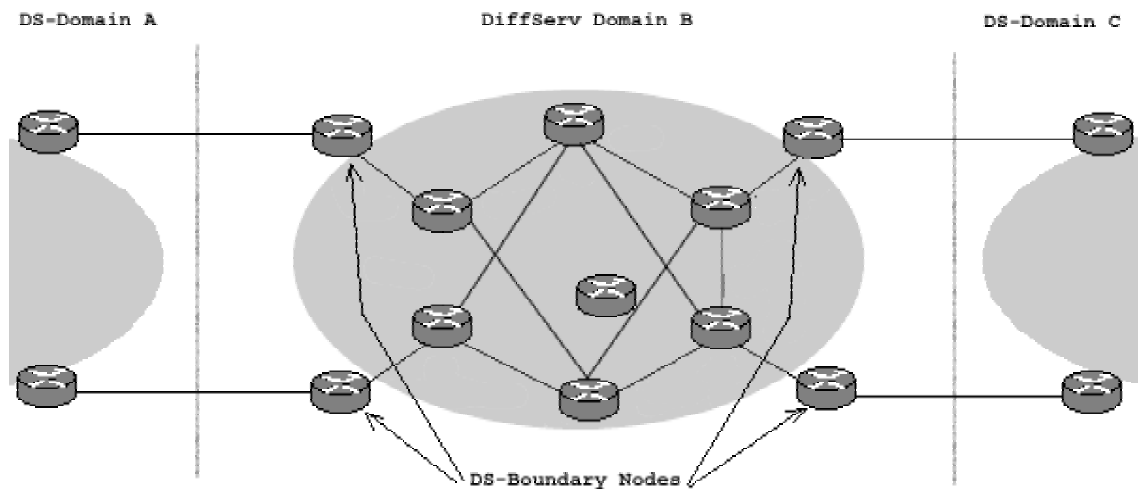


Figure 2.1 DiffServ Domain

Packets arriving at the boundary of a DS domain experience classification and traffic conditioning. The purpose is to impose restrictions on the composition of the resultant *aggregates*. Each packet is marked with a codepoint, and each codepoint identifies a particular aggregate. Aggregates receive differentiated treatment inside

the DS domain. In particular, the externally observable forwarding treatment an aggregate stream receives in an interior node is called *per-hop behaviour* or *PHB* (multiple codepoints can map to the same PHB). In more concrete terms, PHB refers to the packet scheduling, queueing, policing or shaping behaviour of an interior node and it is a basic block to build services. The other ones are *traffic conditioners* and they are typically deployed in DS boundary nodes.

Generalization of the concepts of aggregate (or behaviour aggregate) and per-hop behaviour have been introduced in [11] to meet inter-domain issues. The first one is *traffic aggregate* and refers to packets which are mapped to the same PHB everywhere within a DS domain. The second one is *per-domain behaviour* or *PDB* and describes the edge-to-edge behaviour across a DS domain experienced from a traffic aggregate. This level of abstraction makes it easier to compose cross-domain services as well as making it possible to hide the details of a network's internals while exposing information sufficient to enable QoS.

A particular PHB (or list of PHB's) and traffic conditioning requirements are associated with each PDB. This requires understanding what happens to a PHB under aggregation. Different streams of traffic that belong to the same traffic aggregate merge and split as they traverse the network. The PDB should describe the invariant behaviour of these network activities.

2.2.1 Differentiated Services field definition

When a packet enters a DS domain, it experiences classification and traffic conditioning actions and it is assigned a value called Differentiated Services CodePoint (DSCP). Each DS node must use the DSCP to select the PHB which is to be experienced by each packet it forwards.

The Differentiated Services Field is made of the six most significant bits of the former IPv4 ToS octet or the former IPv6 Traffic Class octet [14] and it encodes the DSCP.

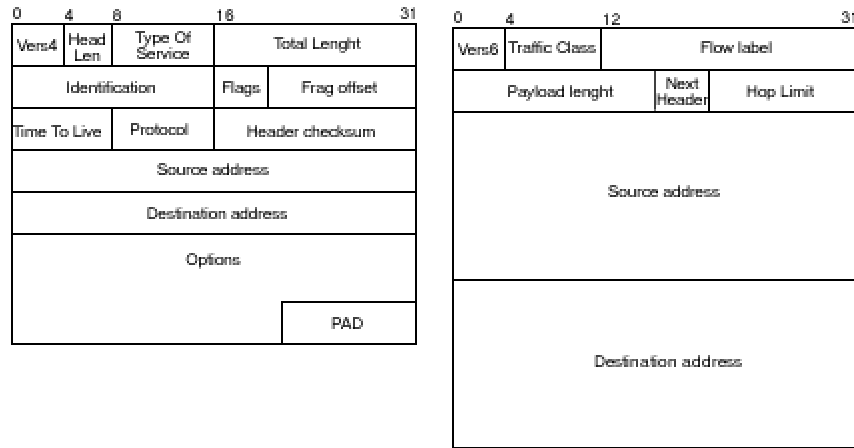


Figure 2.2 IPv4 and IPv6 headers

The two least significant bits of the IPv4 TOS octet and of the IPv6 Traffic Class octet are not presently used by DiffServ. An experimental use for an explicit congestion notification scheme is described in [15]. DS-compliant nodes must select PHB's by matching against the entire 6-bit DS field.

2.2.1.1 Backwards compatibility issues for the IPv4 ToS field

The structure of the DS field described above is incompatible with the previous definition of the IPv4 ToS octet. In the first definition [16], two ToS sub-fields were meaningful, the 3-bit Precedence field and the 3-bit DTR (Data, Throughput and Reliability). The former was intended to allow different levels of importance for a datagram, while the latter was defined for a primitive services differentiation. In the [17] and then [18], the DTR field became a 4-bit field called TOS.

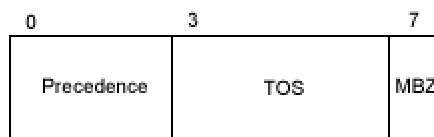


Figure 2.3 The original IPv4 ToS octet

While the Precedence field is widely deployed, even if not in the exact manner intended in [16], the TOS field has not found an effective usage.

To preserve a partial backwards compatibility with most of the deployed forwarding treatments selected by the Precedence field, a minimum requirement on a set of PHB's has been described in [10]. The related codepoints are called *Class Selector Codepoints* and correspond to any of the eight DSCP values equal to XXX000. A class selector codepoint with a larger numerical value than another class selector codepoint is said to have a higher relative order. The higher one should be associated to a PHB that treats packets with a non-lower probability of timely forwarding, if compared to the PHB associated to the lower one. In addition, PHB's selected by codepoints 11x000 must give packets a preferential forwarding treatment by comparison to the PHB selected by codepoint 000000 to preserve the common usage of IP Precedence values 110 and 111 for routing traffic [10].

No attempt is made to maintain backwards compatibility with the DTR/MBZ or TOS bits of the IPv4 ToS octet.

2.2.1.2 The DS field structure

In Figure 2.4, the complete structure of the DS field is shown. The default codepoint value 00000 is defined for the default Best Effort forwarding behaviour. This value meets the class selector PHB requirements and therefore preserves backwards compatibility with the current use of the 000 value for the Precedence field in the IPv4 ToS octet.

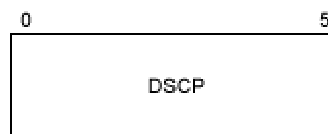


Figure 2.4 DS field definition

The DS field is capable of conveying 64 distinct codepoints. The codepoint space is divided into three pools for the purpose of codepoint assignment and management as showed in Table 2.1 [10]:

Table 2.1 The DSCP Pools

Pool	Codepoint space	Assignment Policy
1	XXXXX0	Standards Action (EF, AF _{xy} , Default, Class-Selector Codepoints)
2	XXXXX1	Experimental/Local Usage
3	XXXXX01	Experimental/Local Usage/Future Standards

2.2.2 Traffic classification and conditioning

In the first chapter, we introduced the concept of Service-Level Agreement (SLA) as the formal part that describes the type of service to be provided from a service provider to a customer. This agreement addresses both technical (e.g. traffic conditioning, service availability) and business aspects (e.g. pricing, contractual rules). A Traffic Conditioning Agreement (TCA) is a sub-set of an SLA and it addresses traffic classification and conditioning rules to be applied to a traffic stream entering the network.

A new terminology has been introduced to describe those elements of SLA and TCA that are addressed by DiffServ. *Service-Level Specification (SLS)* describes a set of parameters and their values which together define the service offered to a traffic stream by a DS domain. *Traffic Conditioning Specification (TCS)* is used to describe a set of parameters and their values which together specify a set of classifier rules and a traffic profile.

Traffic classification is used to describe the activity of splitting a single incoming traffic stream into multiple outgoing streams. The purpose is to steer packets matching some specified rule to an element of a traffic conditioner for further processing. The selection is based on the content of some portion of the packet header. Two types of classifiers have been defined. The BA (Behaviour Aggregate) Classifier classifies packets based on the DS codepoint only. The MF (Multi-Field) classifier selects packets based on the value of a combination of one or more header fields, such as source address, destination address, DS field, protocol ID, source port and destination port numbers, and other information such as incoming interface.

Traffic conditioning is used to describe the activity of ensuring that the traffic entering the DS domain conforms to the rules specified in the TCS, in accordance with the domain's service provisioning policy. This activity is performed through the following mechanisms:

- Meters. They measure the temporal properties of the stream of packets selected by a classifier against a traffic profile specified in a TCS. A meter passes state information to other conditioning functions to trigger a particular action for each packet which is either in- or out-of-profile trigger
- Markers. They set the DS field of a packet to a particular codepoint, adding the marked packet to a particular DS behaviour aggregate
- Shapers. They delay some or all of the packets in a traffic stream in order to bring the stream into compliance with a traffic profile
- Droppers. They discard some or all of the packets in a traffic stream in order to bring the stream into compliance with a traffic profile. This process is known as "policing" the stream.

To facilitate the definition of specific traffic conditioning functions that an incoming traffic stream can experience, the Traffic Conditioner Block has been introduced. It can be thought of as an entity with one input and one or more outputs and a set of control parameters.

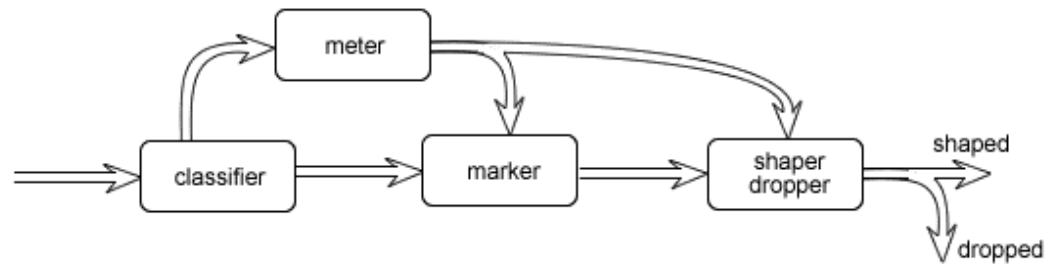


Figure 2.5 DiffServ Traffic Conditioner Block (TCB)

Note that a TCB may not necessarily contain all four elements. It is usually located within DS ingress and egress boundary nodes, but may also be located in nodes within a DS domain, or within a non-DS-capable domain [12].

2.2.3 Per-Hop Behaviors

The DiffServ WG defined the per-hop behaviour (PHB) as the externally observable forwarding treatment applied to a behaviour aggregate by a DS-compliant node. At each network node, mapping the DSCP of any packet to a particular PHB is the way to realize differentiated services along the packet forwarding path.

PHB's may be specified individually, or as a group (a single PHB is a special case of a PHB group). A PHB group usually consists of a set of two or more PHB's that can only be meaningfully specified and implemented simultaneously, due to a common constraint applying to each PHB within the group, such as a queue servicing or queue management policy [10].

The following set of PHB's has been defined by the DiffServ WG: Default PHB, Class Selector PHB's, Expedited Forwarding PHB and Assured Forwarding PHB Groups. A brief description is presented in the next paragraphs.

2.2.3.1 A default PHB

Each DS-compliant node must provide a default PHB and the recommended codepoint is 000000. The Best Effort forwarding behaviour might be its externally observable treatment.

A reasonable implementation of this PHB would be a queueing discipline that sends packets of this aggregate whenever the output link is not required to satisfy another PHB. A reasonable policy for constructing services would ensure that the aggregate will never starve. This could be enforced by a mechanism in each node that reserves some minimal resources (e.g. buffers, bandwidth) for Default behaviour aggregates. This permits senders that are not differentiated services-aware to continue to use the network in the same manner as today [10].

2.2.3.2 Class Selector PHB's

To preserve backwards compatibility with any IP precedence scheme currently in use on the network, the XXX000 values of DSCP have been reserved. These DSCP values are called Class Selector Codepoints and map on PHB called Class Selector PHB's. For this, PHB's requirements have been defined in [10] to ensure that DS-compliant nodes can coexist with IP Precedence-based nodes.

2.2.3.3 Expedited Forwarding PHB

The Expedited Forwarding PHB (EF PHB) [19] is intended to provide a building block for low-delay, low-jitter and low-loss services by ensuring that the EF aggregate is served at a certain configured rate [20]. This building block can be used to supply robust services as the Guaranteed Service in the IntServ model was designed.

An intuitive definition of this PHB is given in [20]: the rate at which EF traffic is served at a given output interface should be at least the configured rate R , over a suitably defined interval, independent of the offered load of non-EF traffic to that interface.

The dominant causes of delay in packet networks are fixed propagation delays (e.g. those arising from speed-of-light delays) on wide area links and queueing delays in switches and routers. Since propagation delays are a fixed property of the topology, delay and jitter are minimized when queueing delays are minimized. In this context, jitter is defined as the variation between maximum and minimum delay, while in general it is used to describe the absolute delay variation among a consecutive pair of packets.

The intent of the EF PHB is to provide a PHB in which suitably marked packets usually encounter short or empty queues. Furthermore, if queues remain short relative to the buffer space available, packet loss is also kept to a minimum.

To ensure that queues encountered by EF packets are usually short, it is necessary to ensure that the service rate of EF packets on a given output interface exceeds their arrival rate at that interface over long and short time intervals, independent of the load of other (non-EF) traffic.

A node that supports the EF PHB behaviour must satisfy the formal/mathematical definition given in [20].

Packets marked for EF PHB may be remarked at a DS domain boundary only to other codepoints that satisfy the EF PHB. Packets marked for EF PHB's should not be demoted or promoted to another PHB by a DS domain.

2.2.3.4 Assured Forwarding PHB Groups

The Assured Forwarding (AF) PHB [21] is nearly equivalent to Controlled Load Service available in the integrated services model. It is a means to offer different levels of forwarding assurances for IP packets crossing a DS Domain. For instance, PHB groups can be used to differentiate the level of urgency of behaviour aggregates (e.g. Telnet, SMPT and FTP can be assigned a greater level of urgency than HTTP). Inside a PHB group, the single PHB's can treat packets with different levels of importance (e.g. when a node experiences congestion, it can drop packets related to FTP applications more than packets related to Telnet applications).

The DiffServ WG defined four different AF PHB groups called AF classes and having the same structure. This structure can be named AF PHB Group Type and the four classes can be named AF PHB Group Instances of the AF type. The packets in one AF class must be forwarded independently from the packets belonging to another AF class. It means a DS node must not aggregate two or more AF classes together. Moreover, a minimum amount of forwarding resources has to be allocated to each implemented AF class.

An AF PHB Group (AF_x, x=1,2,3,4) provides three levels of drop precedence level, and therefore three levels of loss probability. At least two of these have to be implemented. In this case, the lower loss probability has to be assigned to the lower drop level, and the higher loss probability has to be assigned to the two remaining drop levels.

The following notation has been defined: AF_{xy} is the AF PHB in the group x with the drop precedence level y. In Table 2.2, the recommended codepoints are shown.

Table 2.2 DiffServ AF codepoints table

Drop precedence	Class 1	Class 2	Class 3	Class 4
Low	(AF11)	(AF21)	(AF31)	(AF41)
	001010	010010	011010	100010
Medium	(AF12)	(AF22)	(AF32)	(AF42)
	001100	010100	011100	100100
High	(AF13)	(AF23)	(AF33)	(AF43)
	001110	010110	011110	100110

The AF codepoint mappings recommended above do not interfere with the local use spaces nor with the recommended Class Selector codepoints.

3. DIFFSERV SIMULATIONS USING NS

In this chapter, the implementation issues of the DiffServ architecture are inspected and the modelling capabilities of the NS are considered. After a proof analysis, advantages and drawbacks are identified and solutions are given.

The purpose is to have a valuable simulation module that enables both significant service model simulations and performance evaluations.

3.1 The need for simulation

Formally, simulation can be defined as [22]:

“...the process of designing a model of a real system and conducting experiments with this model for the purpose either of understanding the behaviour of the system or evaluating various strategies (within the limits imposed by a criterion or a set of criteria) for the operation of the system”

From this definition, the primary advantages of simulations are understandable. First of all, with simulations it is possible to duplicate the behaviours of real systems through hardware and software. This leads to understand the sensitivities of parameters and their best setting-up without deploying several scenarios in a real network.

Secondly, it allows to study a problem at several different levels of abstraction. By approaching a system at a higher level of abstraction, a better understanding of behaviours and interactions of all the high level components within the system is achievable. The lower level components may then be designed and subsequently simulated for verification and performance evaluation. The entire system may be built based upon this top-down technique and alternative solutions can be investigated without actually physically building the systems. The drawback of this point is that the design and implementation of simulators are almost as complex as the systems being simulated.

Finally, simulators can be used as an effective means for presenting and demonstrating concepts. This is particularly true of simulators that make intelligent use of computer graphics and animation.

Despite the advantages presented above, it is important to remark that it is sometimes very difficult to find correct models for the real world. For example, extensive studies have shown that the Poisson assumption is not valid for modelling Internet traffic [2]. In particular, the aggregate arrival process of packets is not Poisson, but it contains a long-term correlation process that essentially changes the characteristics of traffic process. It is said that the traffic process is self-similar. Self-similarity in this context means that there are similar traffic variations on every time scale from milliseconds to weeks. Even on Internet traffic it is important to notice that there is no such thing as traffic process independent of the network resources (e.g. TCP and congestion-control mechanism). One of the ways of combating the aforementioned difficulty is to introduce simplifying assumptions or heuristics into the model. These elements have to be introduced as clear hypotheses and results have to be evaluated on their basis to avoid wrong conclusions.

3.2 Conceptual model for a DiffServ router

For the purpose of this work, it is worthy to introduce a conceptual model for DiffServ routers as described in [23]. The logical view of a DiffServ router architecture allows a better comprehension of how the basic building block of the DiffServ framework can be implemented. Furthermore, the deployment of mechanisms to implement the DiffServ building block will lead us to a better design and implementation of the networks and scenarios on the Network Simulator.

The presented model is suitable for both edge and core DiffServ routers. This is because the latter one can be implemented by a subset of components needed for the former one.

3.2.1 Hierarchical view of the model

In the conceptual model, a hierarchical organization is given. Starting from the topmost one, we will descend through the following levels reaching the granularity of mechanisms. In Figure 3.1, the major functional blocks of the model are illustrated. This analysis mostly focus on the central part of the diagram where interfaces to the links and router core are modelled.

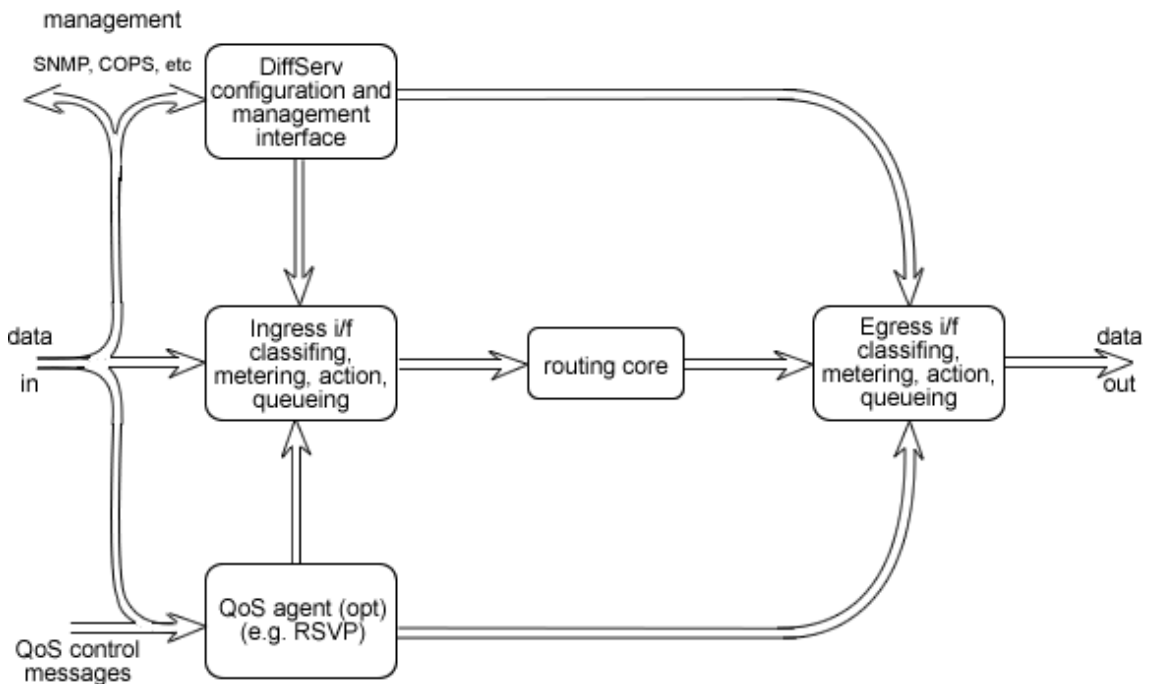


Figure 3.1 DiffServ router major functional blocks

The *Configuration and Management Interface* is intended to monitor and to provide the DiffServ operating parameters. Monitored parameters include statistics regarding traffic carried at various DiffServ service levels. These statistics may be important for accounting purposes and/or for tracking compliance to Traffic Conditioning Specifications (TCS's) negotiated with customers. Provisioned parameters are

primarily the TCS parameters for Classifiers and Meters and the associated PHB configuration parameters for Actions and Queueing elements.

The *QoS Agent Module* can add features for integration with other deployed QoS models (e.g. snooping of RSVP messages may be used to learn how to classify traffic without actually participating as a RSVP protocol peer [24]).

The *routing core* represents the normal routing and switching functionality, it moves packets between interfaces according to policies outside the scope of DiffServ. This block can be thought of as an infinite bandwidth, zero-delay backplane connecting interfaces. Actual queueing delay and eventual packet loss behaviour related have to be modelled inside the interface involved.

Before presenting the next block, two basic concepts are depicted. *Datapath* is used to define a conceptual path taken by packets with particular characteristics through a DiffServ router. *Functional datapath elements*, such as Classifiers and Meters, choose as packets take a specific path. Functional datapath elements are the basic building blocks of the conceptual router and represent the lowest level of the hierarchy.

Now the *interfaces* are introduced. Each interface has ingress and egress functionalities. Each of these components can be expressed as one or more datapath entities called Traffic Conditioning Block (TCB). In Figure 3.2, the logical view of a TCB is illustrated.

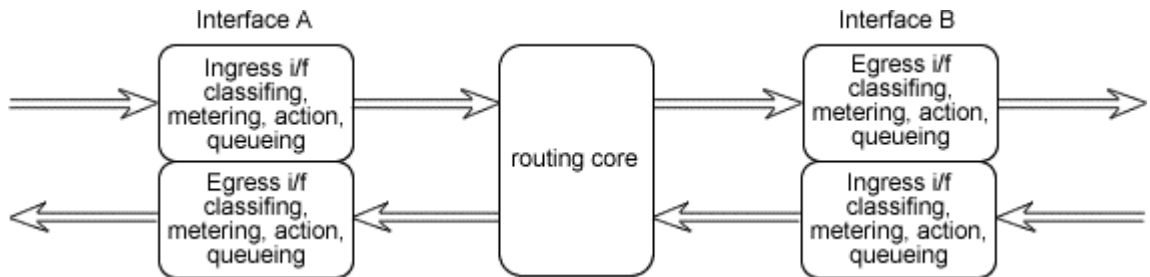


Figure 3.2 Traffic conditioning and queueing elements

Four QoS control functions can be deployed in a datapath of traffic crossing the router in one direction:

- Classification
- Metering
- Action (involving markers, absolute droppers counters and multiplexors)
- Queueing/Shaping (involving algorithmic droppers, queues and schedulers)

It is not mandatory that each of these functional datapath elements are implemented at both ingress and egress; equally multiple sets of these elements may be placed in series and/or in parallel at ingress or at egress.

Two basic assumptions are made: the first is that the set of DiffServ functions to be carried out on traffic in a given interface are independent of those functions on all other interfaces; the second is that in this analysis only two-port routers are considered.

3.2.2 Basic building blocks of a DiffServ router

The basic building blocks of a conceptual router are called *functional datapaths*. It is possible to classify these blocks in four main categories: classifier elements, meter elements, action elements and queueing elements.

Classifiers are 1:N (fan-out) devices; the packets from the input stream are sorted into various output streams by *filters* which match the contents of the packet or possibly match other attributes associated with the packet. A filter consists of a set of conditions on the component values of a packet's classification key.

Classifiers can be cascaded in sequence and have to be unambiguous (every possible classification key matches at least one filter and any ambiguity between overlapping filters is resolved by precedence).

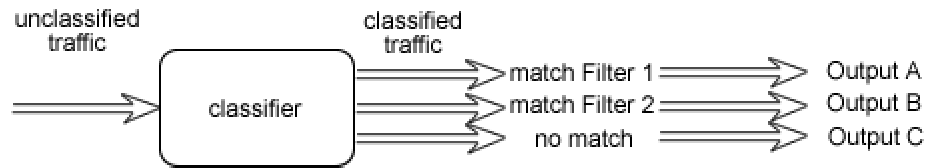


Figure 3.3 An example classifier

Two main classifier types have been selected to steer packets. The Behaviour Aggregate (BA) classifier uses the DSCP value encoded in the IP header, while the Multi-Field (MF) classifier uses a set of attributes such as source and destination addresses, source and destination ports or application type. An example of BA classifier configuration is given in Table 3.1, while an example of MF classifier configuration is provided in Table 3.2.

Table 3.1 A BA classifier configuration example

Filter	DSCP
Filter 1	101010
Filter 2	111111
Filter 3/no match	*****

Table 3.2 An MF classifier configuration example

Filter	IPv4 Src Addr	IPv4 Dest Addr	TCP SrcPort	TCP DestPort
Filter 1	172.31.8.1/32	172.31.3.X/24	X	5003
Filter 2	172.31.8.3/32	172.31.3.X/24	X	5003
Filter 3/no match	*.*.*.*/*	*.*.*.*/*	*	*

Meters are logically 1:N (fan-out) devices; packets from the input stream are selected against a traffic/temporal profile based on a TCS and are assigned to one of the several conformance levels (meter's outputs). This definition differs slightly from that described in [12] and consequently in paragraph 2.2.2, but does not change the function of a meter.

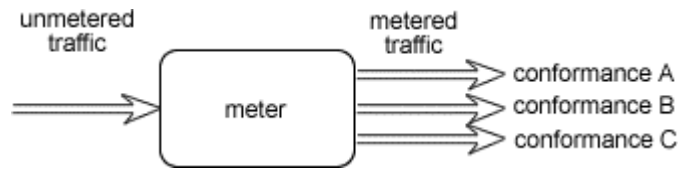


Figure 3.4 A general meter

A meter, according to this model, measures the rate at which packets making up a stream of traffic pass it, compares the rate to some set of thresholds and produces some number of potential results (two or more): a given packet is said to be "conformant" to a level of the meter if, at the time that the packet is being examined, the stream appears to be within the rate limit for the profile associated with that level.

Possible mechanisms to implement a meter are Average Rate Meter, Exponential Weighted Moving Average Meter (EWMA), Token Bucket Meter (TB), Single Rate Three Colours Marker (srTCM) [25], Two Rate Three Colours Marker (trTCM) [26] and Time Sliding Windows Three Colours Marker (TSWTCM) [27].

The action elements are:

- *DSCP Markers* is 1:1 element which sets a codepoint in the DS field of the packet header
- *Absolute Dropper* simply discards packets
- *Multiplexor* is a M:1 (fan-in) element which merges traffic streams (the output could be connected to a classifier or a meter)
- *Counter* is a 1:1 element which updates a counter by L and a packet counter by 1 every time a L-byte sized packet passes through them; it can be used to collect statistics for the purpose of customer billing, service verification or network engineering
- *Null Action* is a 1:1 element which performs no action on the packet; such an element is useful to define when the configuration or management interface does not have the flexibility to omit an action element in a datapath segment

The *queueing elements* perform three distinct, but related functions; they store packets, they modulate the departure of packets belonging to various traffic streams and they selectively discard packets. Hence, queueing elements can be decomposed in three functional datapath elements: *Queues*, *Schedulers* and *Algorithmic Droppers*.

The queue element stores packets coming from one or more input and serves them to the output. Packets are served in the same order they are received (FIFO). Operations of removing or examining packets can be implemented, but the order among packets belonging to the same microflow must not change. FIFO queues are theoretically without limits on their depth. In the real world, resources are limited, hence a limit size has to be modelled. For instance, an algorithmic dropper cascaded to a FIFO queue implements a real queue often called Drop Tail Queue.

The Scheduler element gates the departure of each packet that arrives at one of its inputs, based on a service discipline. This service discipline is an algorithm which might take any of the following as its input:

1. static parameters such as relative priority associated with each of the scheduler's inputs
2. absolute token bucket parameters for maximum or minimum rates associated with each of the scheduler's inputs
3. parameters, such as packet length or DSCP, associated with the packet currently present at its input
4. absolute time and/or local state

Possible service disciplines fall into a number of categories including, but not limited to, First Come First Served (FCFS), strict priority, weighted fair bandwidth sharing (e.g. WFQ), rate-limited strict priority and rate-based.

The Algorithmic Dropper is an element which selectively discards packets that arrive at its input, based on a discarding algorithm. It can be modelled as a 1:1 element or as an invoking operation on a FIFO queue (e.g. to implement a Drop Tail Queue).

For EF PHB, Drop-on-threshold or Drop-out-of-profile packets are suitable to maintain a short queue, hence, a short queueing delay. For AF PHB, the most widely used mechanisms are Random Early Detection (RED) [28] and Multi-level RED family. The latter applies multiple sets of RED parameters against different packets in the same queue. In [29] variants of RED are classified in four categories, depicted in Figure 3.5.

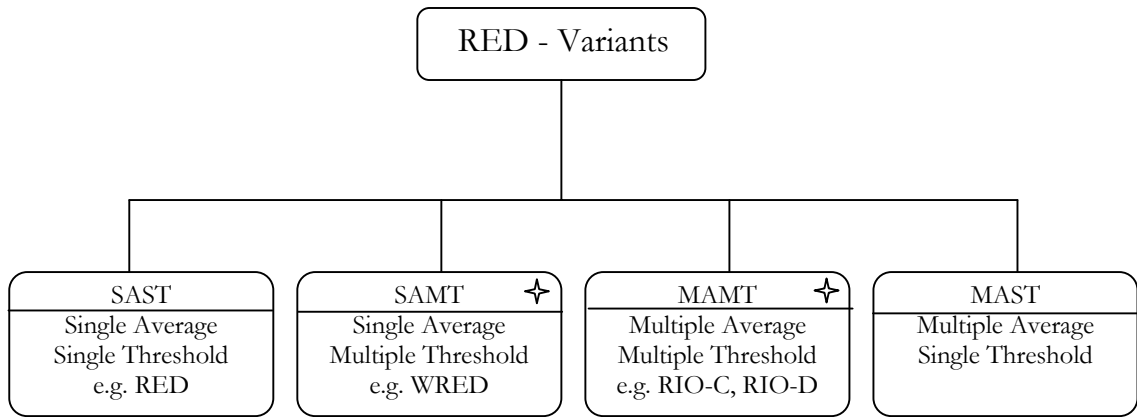


Figure 3.5 Variants of RED (✦ = MRED)

In a DiffServ Model, the use of queueing systems composed by several queueing elements enables interesting features. For instance, the shaping of behaviour aggregates, that can be done limiting the variation in delay or imposing a maximum rate. Another example is the load sharing, it is realized by allowing several streams to share a link in a semi-predictable fashion. This can be implemented through a set of FIFO queues steered by a weighted queueing algorithm such as the WFQ scheme. This form of queueing system tends to move delay and variation in delay from under-used classes of traffic to heavier users, as well as managing the rate of the traffic streams.

The Classifier, Meter, Action, Algorithmic Dropper, Queue and Scheduler functional datapath elements described above can be combined into Traffic Conditioning Blocks (TCBs).

All these elements are arranged arbitrarily according to the policy being expressed, but always in the order here. Traffic may be classified; classified traffic may be metered; each stream of traffic identified by a combination of classifiers and meters may have some set of actions performed on it, followed by drop algorithms; packets of the traffic stream may ultimately be stored into a queue and then be scheduled out to the next TCB or physical interface. It is permissible to omit elements or include null elements of any type, or to concatenate multiple functional datapath elements of the same type.

3.3 The Network Simulator (*NS*)

The prominent paradigm currently being used to implement simulators is the ubiquitous *object-oriented* paradigm, in which software entities closely model their real world counterparts. This paradigm has been successfully employed to implement a wide variety of simulators such as *NS* [30].

NS is a discrete event simulator targeted at networking research, which provides substantial support for simulation of TCP, routing, and multicast protocols over wired and wireless, local and satellite networks.

Currently, the *NS* Network Simulator development is supported through DARPA with SAMAN [31] and through NSF with CONSER [32], both in collaboration with other researchers including ACIRI [33].

3.3.1 *NS* architecture

As mentioned in the previous paragraph, *NS* is based on object-oriented paradigm. This leads to a better reusability and, coupled with the open source nature, it speeds up extensions coming from third parts.

The simulator is written in C++ and uses OTcl as a command and configuration interface. The underlying reasons of the need for two languages (one scripted, another compiled) are that:

- compiled languages are suitable for efficiently manipulating bytes, packet headers and for algorithms implementation that run over large data sets. For these tasks run-time speed is important
- scripting languages are suitable for those tasks which need to be run only once in a simulation, but which need to be tuned very often (e.g. topology configuration, setting parameters). For these tasks, a short turn-around time is important

NS is event-driven and four kinds of schedulers are provided. Among these, the real-time one provides the emulation capability. It means that the simulator can be introduced in a live network, get traffic from it and inject traffic into it.

NS is single-thread, only one event is in execution at any given time. If more events are scheduled at the same time, they are managed in a first dispatched manner. Neither partial execution is allowed nor preemption is supported.

Arbitrary network topologies, composed of routers, links and shared media can be defined. A rich set of protocols such as TCP and UDP is available and various types of applications can be simulated. Among them are FTP, Telnet, and HTTP, which use TCP as the underlying transport protocol, and applications requiring a Constant Bit Rate (CBR) traffic pattern, which use the UDP transport protocol. Multiple

queueing and scheduling policies can be configured such as drop-tail, random early detection, priority and fair queueing. Packet losses are simulated by buffer overflows in routers, which is also the dominant way packets get lost in the Internet. There is also support for error models other than losses through buffer overflow. The routing model can be static or dynamic. The link state (i.e. Dijkstra's SPF) and distance vector (i.e. Bellman-Ford) routing algorithms are supported.

3.3.1.1 Network topology

For a better understanding of what NS can model it is necessary to look into the topology building blocks design. Two elements are the basic entities of a network model: nodes and links.

Nodes (Figure 3.6) contain at least the following components:

- an address, monotonically increasing by 1 (from initial value 0) across the simulation namespace as nodes are created
- a list of neighbors
- a list of agents
- a node type identifier
- a routing module

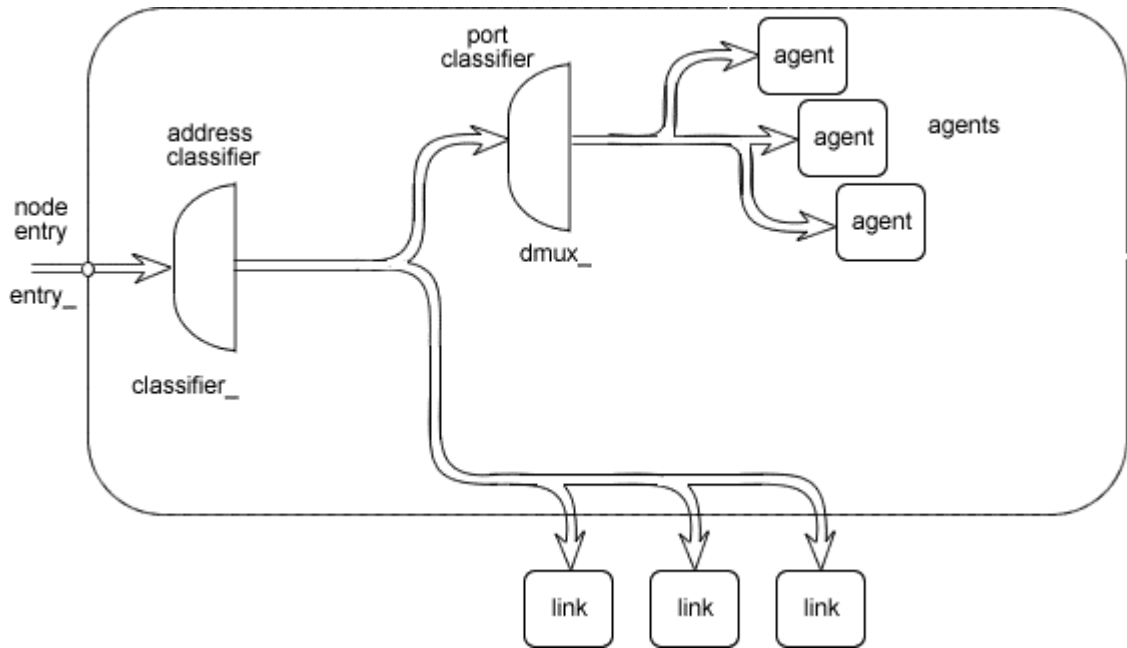


Figure 3.6 The Node structure

Links connect nodes, model output interfaces of a node and link characteristics. Here a brief description of the composing elements showed in Figure 3.7 is given:

- `head_`: entry point to the link, it points to the first object in the link
- `queue_`: reference to the main queue element of the link. Simple links usually have one queue per link. Other more complex types of links may have multiple queue elements in the link
- `link_`: a reference to the element that actually models the link, in terms of the delay and bandwidth characteristics
- `ttl_`: reference to the element that manipulates the ttl in every packet
- `drophead_`: reference to an object that is the head of a queue of elements that the process link drops

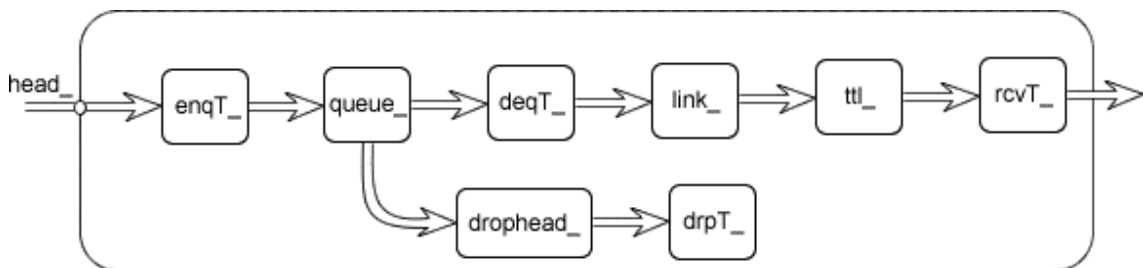


Figure 3.7 The Link structure

3.3.1.2 The delay model

Another important design choice on which it is worthy to focus is the delay model. In a real network, four components compose the end-to-end delay a packet experiences:

- processing delay: time needed to process a packet on each node of the network
- transmission delay: time a node needs to put the packet on the link (packet size/link bandwidth)
- propagation delay: time a packet needs to cover the link
- queueing delay: time a packet waits in a queue to be served

All these delay elements can be modelled except for the processing delay, which is not supported.

3.3.2 DiffServ support

DiffServ support has been added to *NS* on 2 November 2000. The module has been developed by Nortel Networks, but it is no longer supported by them. DiffServ functionalities are captured in a Queue class. Since instances of this class are deployed in the link element (refer to paragraph 3.3.1.1.), it models the output interface of a router. Therefore, it is possible to simulate DiffServ functionalities deployed only in output interfaces.

A Unified Modeling Language (UML) Class Diagram here depicts the DiffServ module design and a brief description is given:

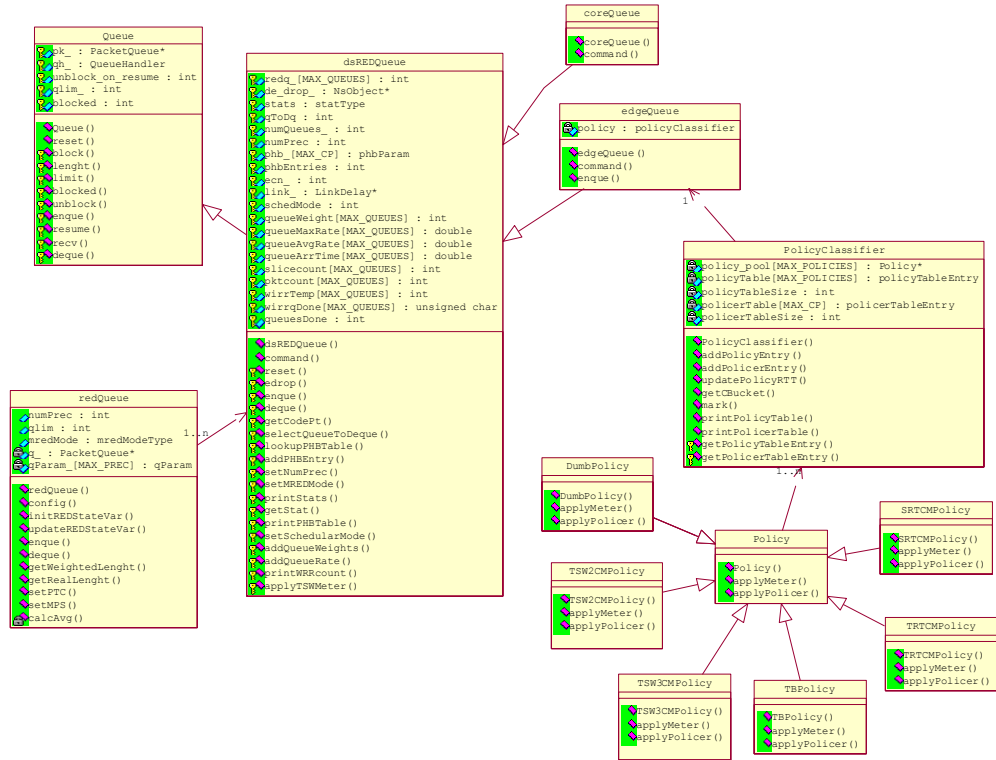


Figure 3.8 DiffServ module UML Class Diagram

The dsREDQueue class models a multiple queue, each one for a class of service. Each single queue with its congestion avoidance mechanisms is provided by the class redQueue.

EdgeQueue and CoreQueue classes are a specialization of the dsREDQueue and model the output interface of a router. The PolicyClassifier class is part of the first one and provides metering and marking functionalities.

Referring to Figure 3.2, the logical structure of the TCB deployed in edge and core router output interfaces is given:

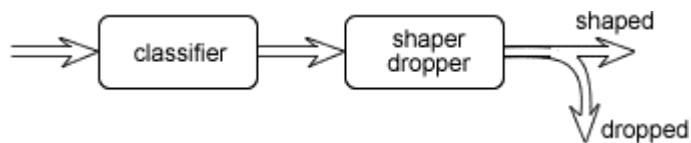


Figure 3.9 DiffServ Traffic Conditioner Block in the NS core router object

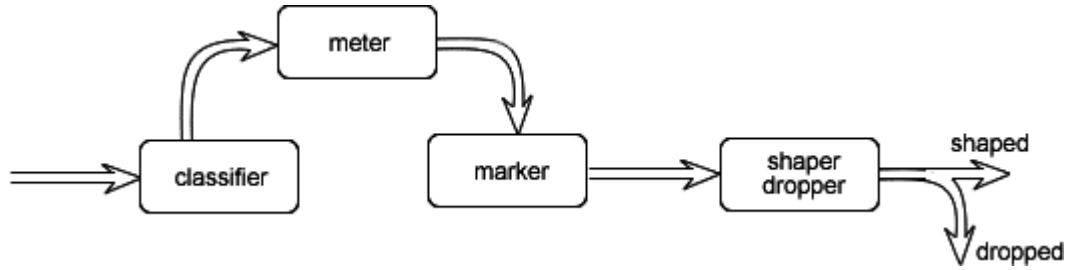


Figure 3.10 DiffServ Traffic Conditioner Block in the NS edge router object

Based on the former analysis presented in this chapter, the mechanisms provided by the DiffServ module are listed:

- Classifier: a MF classifier based on source address and destination address for meter selection and a DSCP classifier for queue selection
- Meter: TSW2CM, TSW3CM, Token Bucket, srTCM and trTCM
- Marker: cascaded to meter
- Queue: Drop Tail
- Scheduler: Round Robin (RR), Weighted Round Robin (WRR), Weighted Interleaved Round Robin (WIRR) and Priority Queueing (PQ)
- Algorithmic Dropper: RIO Coupled (RIO-C), RIO De-coupled (RIO-D), Weighted RED (WRED) and Drop-on-Threshold

Preliminary simulations have been conducted to focus on merits and shortcomings. The result analysis is here presented, first at the designer level and second at the user level.

At the designer level, a basic object-oriented structure is visible. Inside each new class a higher abstraction level is desirable and achievable. Both legibility and extensibility are poor due to a *flat* design. For instance, functions related to algorithmic droppers are grouped by method instead of being hidden in a class for each dropper type. (see dsREDq.h/cc file, redQueue:enqueue and redQueue:calcAvg methods). A more elegant solution could be to define an Abstract Data Type called,

for example, `dsDropper`. This class should provide public methods that a `dsREDqueue` needs to manage congestion avoidance. In particular, a *config* method to set dropper parameters and an *enqueueEvent* method to ask a dropper which action should be applied to the incoming packet.

The same considerations can be applied to schedulers (in file `dsRED.h/cc`) and meters (in file `dsPolicy.h/cc`). In general, the information hiding is not well implemented (e.g. struct `policyTableEntry`) and a better encapsulation is achievable.

A confusing use of the field `packet_type` in the common header of each packet has been noticed. This field is filled from time to time with a different semantic value depending on the deployed traffic agent. For instance, http traffic fills it with the `PT_HTTP` constant, Telnet and FTP traffic fill it with the `PT_TCP` constant, cbr traffic fills it with the `PT_CBR` constant. Hence transport protocol, application protocol and application type are alternatively used on the same field. This is an undesirable limitation against the need for packet classification and marking for differentiated services purpose.

At the user level, a lack of schedulers suitable for DiffServ is the first noticed shortcoming. In the official distribution, only PQ is eligible, especially for EF aggregate treatment [34], hence low delay, low jitter and low loss traffic requirement. On a SLS base, the EF aggregate has to be correctly policed to avoid the following drawbacks:

- a. starvation of other services
- b. high queue latency

A rate limiter for PQ is provided by the module, but it is based on a departure rate estimator. This means that if the arrival rate is greater than the departure rate, the priority queue is always full and every packet experiences a high queueing delay. Moreover, congestion is managed by buffer overflow. Informal simulations firstly

pointed out a bug and secondly, after having fixed it, confirmed the hypothesis above explained.

A better solution is to use a token bucket as an arrival rate meter, a marker for out-of-profile traffic and a drop-out-of-profile dropper. This way, it is possible that EF aggregate respects the traffic contract (SLS) without introducing a relevant queueing delay in case of congestion.

Classification, marking, and metering rules definition are not well designed. The following example shows a set of restrictions:

```
0: $qE1C setMREDMode DROP
1: $qE1C addPolicyEntry [$src id] [$dst id] TokenBucket 20 200000 10000
2: $qE1C addPolicerEntry TokenBucket 20 21
3: $qE1C addPHBEntry 20 0 0
4: $qE1C addPHBEntry 21 0 1
5: $qE1C configQ 0 0 20
```

`$qE1C` identifies the `edgeRouter` object; on line 0, a drop-on-threshold dropper is set; on line 1, marking and metering rules are defined: packets coming from node `src` and going to node `dst` are marked with DSCP 20. Moreover, they are metered with a token bucket whose parameters are 200kpbs for Committed Information Rate (CIR) and 10.000 bytes for Maximum Burst Size (MBS); on line 2, out-of-profile packets are marked with DSCP 21; on lines 3 and 4, there are rules for the DSCP classifier and codepoints are mapped on a specific queue and a drop precedence level is assigned; on line 5, a threshold parameter for the dropper is set.

In this example, several restrictions arise:

- a. it is not possible to mark traffic on a per-packet-type basis; for instance, it is not possible to mark Telnet traffic with a certain codepoint and HTTP traffic with another codepoint
- b. it is not possible to mark traffic among several source-destination nodes and to apply the same meter to marked traffic; therefore, it is not possible to define a meter for aggregate traffic

- c. it is not possible to drop out-of-profile traffic; in fact, the deployed dropper is on a threshold basis, and this threshold refers to a single queue where packets with different precedence levels can be injected (e.g. a drop precedence level can be assigned to in-profile traffic and another drop precedence level can be assigned to out-profile traffic).

3.3.3 DiffServ module improvements

The former analysis raises the need for a partial redesign of the module to provide a substantial set of mechanisms suitable for service models simulation. The following changes will be applied to improve router modelling capabilities:

- schedulers: the targets are to speed the scheduler addition by encapsulating the scheduler mechanism in its own class and to increase the number of available schedulers
- marker and meter: the target is to enable marking on a per-packet basis and to decouple marking from metering
- dropper: the target is to enable a drop out-of-profile traffic capability on a drop precedence level basis

Moreover, the following set of functionalities for measurement and performance analysis will be added:

- One-Way Delay (OWD): the target is to enable end-to-end one-way delay computation as defined in [35]; instantaneous, average, minimum and frequency distributed OWD will be provided for UDP-based traffic
- IP Packet Delay Variation (IPDV): the target is to enable the delay variation computation in a destination node for packets belonging to the same micro-flow; the definition of IPDV is based on [36]; instantaneous, average, minimum and frequency distributed IPDV will be provided for UDP-based traffic

- Queue length: the target is to enable queue length checking at a script language level; it will be provided on a per queue and per drop level precedence basis
- Maximum burstiness for queue 0: the target is to enable maximum enqueued packets checking for queue 0; this queue is typically used for priority traffic
- Departure rate on a per queue basis and on a per queue and per drop level precedence basis
- Received packets, transmitted packets, early dropped and late dropped packets on a DSCP basis, both absolute and percentage values
- TCP Goodput on a DSCP basis, instantaneous and frequency distributed TCP Round-Trip Time on a DSCP basis, instantaneous and frequency distributed TCP Windows Size on a DSCP basis: the target is to enable computation of performance parameters for TCP-based traffic to understand the level of differentiation on an aggregate level

In the last years, several *schedulers* for buffer management have been proposed and some of these are suitable for differentiated services implementation. In this section, the first aim is to redesign the dsREDqueue class encapsulating scheduler functionalities in a different class called dsScheduler. This class represents only an abstract scheduler and real schedulers can be implemented by specialization. The following UML class diagram shows the new design:

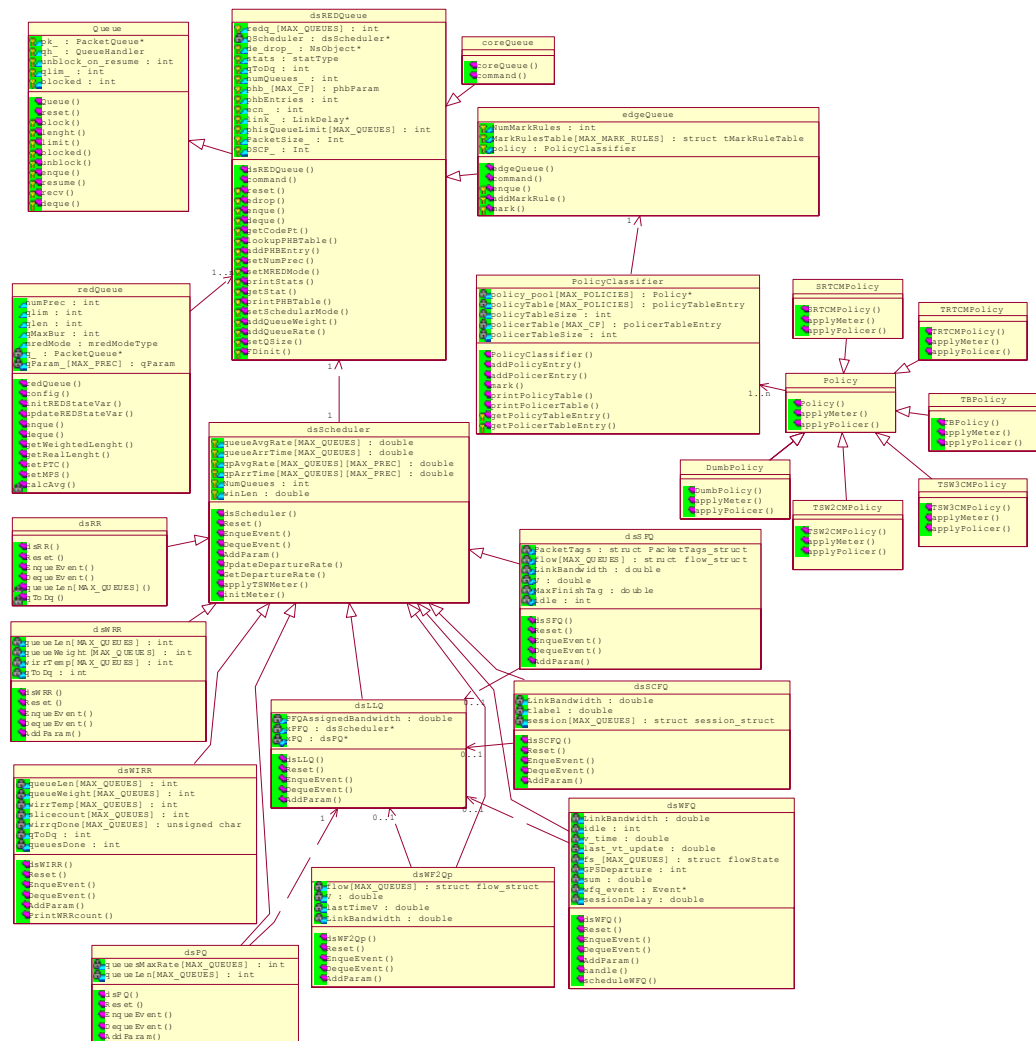


Figure 3.11 DiffServ+ module UML Class Diagram

The following step is the porting of schedulers coming with the module, that is RR, WRR, WIRR and PQ. For backwards compatibility, the rate limit option in the PQ has been preserved. The departure rate estimator is present in each class and can also be used for statistic purposes.

Finally, new schedulers suitable for differentiated services implementation will be added. The decision is to implement the following schedulers: WFQ [37], WF²Q+ [38], SCFQ [39], Start-time Fair Queueing (SFQ) [40] and Low Latency Queueing (LLQ) [41].

WFQ, also known as Packet-by-Packet Generalized Processor Sharing (PGPS), has been the first attempt to emulate a Generalized Processor Sharing (GPS) system on a per-packet basis. Currently, it is not considered suitable for DiffServ implementation due to a high implementation complexity and to a weak fairness property. In [42], it is pointed out that the difference between the amount of service provided to each session by WFQ and the GPS is not merely one packet, as thought first, but $N/2$ packets, where N is the number of sessions sharing the link. It is here implemented for its historical importance.

WF²Q+ has been introduced to overcome the limits of WFQ. Moreover, it is less complex than WF²Q [43] even if the delay bound, fairness and worst-case fair properties are the same.

SCFQ was proposed as a simpler alternative to WFQ. It avoids the needs of a GPS emulation, but as a drawback, it has a larger delay bound. SFQ was proposed to maintain the simplicity of SCFQ, but to provide a tighter delay bound.

LLQ has been introduced by Cisco and here there is a basic implementation. It is suitable for service models implementation that include a Premium Service for EF aggregate and several other services that require relative differentiation. The idea is to use a PQ scheduler to accomplish low delay, low jitter and low loss requirements

of Premium class and to use a PFQ scheduler to accomplish link sharing, fairness and efficiency for other services. A portion of link bandwidth is statically assigned to PQ, but further studies could get in deep on how to give Premium excess bandwidth to lower services.

More schedulers have been proposed in the last years (e.g. CBQ [44], LFQ [45], FFQ [46], SPFQ [46], BPR [47] and WTP [47]) and they are mentioned as a reference.

Decoupling of *marking* and *metering* rule definition is done adding the new command `addMarkRule`. This command also provides a marking rule on a per-transport-protocol basis and per-application-type basis. The last feature needed to add the new field `app_type_` in the common header of each packet and to update traffic agents to fill it with the proper value. The syntax of the command is:

```
$edgeRouterObject addMarkRule initialCodepoint srcNode dstNode
                    transportProtocolType applicationType
```

SrcNode and dstNode can be a specific node or any. TransportProtocolType can be udp, tcp, ack or any, while ApplicationType can be telnet, ftp, audio, realaudio, cbr or any. The list of possible values can be easily extended (refer to file packet.h for packet type list and to method `edgeQueue::addMarkRule` in file dsEdge.cc for marking code). Obviously, the code of the agent which generates the traffic has to be changed to fill packet header fields with the proper constants. Packets not matching any rules are marked with the default codepoint (DSCP=0).

The `addPolicyEntry` command changes as follows: the source address and destination address are not needed; the initial codepoint is not used for marking, but for classification, therefore for meter selection on a DSCP basis.

Drop out-of-profile packets are implemented by modifying the drop-on-threshold dropper. Referring to the previous example, here it is the new script configuration:

```

0: $qE1C setMREDMode DROP 0
1: $qE1C addMarkRule 20 [$s1 id] [$dest1 id] udp cbr
2: $qE1C addPolicyEntry 20 TokenBucket 200000 10000
3: $qE1C addPolicerEntry TokenBucket 20 21
4: $qE1C addPHBEntry 20 0 0
5: $qE1C addPHBEntry 21 0 1
6: $qE1C configQ 0 0 20
7: $qE1C configQ 0 1 -1

```

On line 7, parameter `-1` for queue 0 precedence level 1 means that the packets matching have to be dropped.

3.3.3.2 Measurements and performance analysis

One-Way Delay and IP Packet Delay Variation are two important parameters for EF PHB performance analysis. Since usually the traffic belonging to this aggregate is carried by UDP protocol, the measurement solution is deployed in it. The new attribute `sendtime_` has been added to the packet header (refer to file `packet.h`, `struct_hdr_cmh`). The purpose is to carry in it the packet send time. This attribute is filled by the `udp` agent when the `sendmsg` method is called (refer to `udp.h/cc` files).

The computation of these parameters has been added to the `LossMonitor` agent (refer to `LossMonitor.h/cc` files) and the following new variables can be accessed at the script level:

```

flowid_           : flow id to monitor
npktsFlowid_      : number of received packets
ipdv_             : current ipdv
owd_              : current owd
sumIpdv_          : sum of all |ipdv|
sumOwd_           : sum of all owd
minOwd_           : minimum owd

```

Therefore current and average owd and ipdv can be computed, moreover minimum owd that can be used to normalize diagrams.

Even owd and ipdv frequency distribution computation was added because it enables a better understanding of mechanisms behaviors, in particular schedulers.

The following command enables this capability:

```
$LossMonitorObject FrequencyDistribution owdTick ipdvTick fileName
```

An object called tFD has been defined. It provides attributes and methods for data storage and frequency distribution computation. The tick parameter is the width of intervals where time values are mapped. When the command `FrequencyDistribution` is executed, two instances of the object tFD are allocated, one for the one-way delay parameter and another for the IP delay variation parameter. There is no additional overhead if this feature is not enabled.

```
$LossMonitorObject flushFD owdFactor ipdvFactor owdThreshold ipdvThreshold
```

Using this command, collected data are written in two files called `owdfileName` and `ipdvfileName`. In every file, two columns of values are provided. In the first column the interval upper bound divided by the factor is written (normalization), while in the second column the percentage of packets fitting that interval is written, but only if greater than the threshold.

Other performance parameters are accessible in both `$edgeRouterObject` and `$coreRouterObject` using the following syntax:

```
set paramVar [$edge|coreRouterObject getStat param]
```

where param can be:

pkts	:	number of all received packets	
drops	:	number of all dropped packets due to buffer overflow	
edrops	:	number of all dropped packets due to droppers	
%drops	:	percentage of all dropped packets due to buffer overflow	
%edrops	:	percentage of all dropped packets due to droppers	
pkts x	:	number of received packets	(DSCP=x)
drops x	:	number of dropped packets due to buffer overflow	(DSCP=x)
edrops x	:	number of dropped packets due to droppers	(DSCP=x)
%drops x	:	% of dropped packets due to buffer overflow	(DSCP=x)
%edrops x	:	% of dropped packets due to droppers	(DSCP=x)

Moreover, useful aggregate parameters information for TCP traffic are accessible on a DSCP basis:

```
TCPcwnd x : current window size used by the sender (DSCP=x)
```

TCPrtt x : current RTT measured by the sender (DSCP=x)
 TCPbGoTX x : current number of transmitted bytes (DSCP=x)
 TCPbReTX x : current number of retransmitted bytes (DSCP=x)
 TCPnReTX x : current number of retransmitted packets (DSCP=x)

An important parameter that can be computed with previous collected data is the goodput for TCP traffic. There is not a unique definition, but in this work, the following has been used:

$$Goodput(x) = \frac{TCPbGoTX(x)}{TCPbGoTX(x) + TCPbReTX(x)}$$

x is the DSCP for which goodput is supposed to be computed. Goodput equal to 1 means, for DSCP x, that each byte has been sent only once, no retransmissions occurred.

The following commands provide a departure rate for queue x or for queue x and drop precedence level y:

```

$edge|coreRouterObject getDepartureRate x
$edge|coreRouterObject getDepartureRate x y

```

while the following commands provide the current number of enqueued packets for queue x or for queue x and drop precedence level y:

```

$edge|coreRouterObject getQueueLen x
$edge|coreRouterObject getVirtQueueLen x y

```

and finally, the following parameters can be accessed:

```

$edge|coreRouterObject set MBS0_ : maximum burstiness for queue 0
$edge|coreRouterObject set DSCP_ : DSCP of last enqueued packet
$edge|coreRouterObject set packetSize_: packet size of last enqueued packet

```

To compute TCP RTT frequency distribution and TCP Windows Size frequency distribution on a DSCP basis, the following commands can be used:

```

$edge|coreRouterObject frequencyDistribution x

```

this enables data collection for both RTT and Window Size frequency distribution for DSCP equal to x ,

```
$edge|coreRouterObject flushFD x
```

while this writes RTT frequency distribution on file `FDrtt_DSCP $x$ .tr` and Window Size frequency distribution on file `FDcwnd_DSCP $x$ .tr` .

A very rich set of parameters is now accessible to better understand the behaviour of the simulated network and the level of services differentiation.

It is important to remark that these changes to the simulator allow computation of the above parameters without the need for a post processing of a trace file.

4. SIMULATION SCENARIOS AND RESULTS

In this chapter, the simulation capabilities of the improved module will be explored. The first scenario is an attempt to compare the performance analysis of a real IP DiffServ enabled network and a simulated one and, hence, to verify how much the simulator well models a real network. The second scenario aims to explore congestion avoidance capabilities and selective packet discard among traffic classes belonging to the same AF PHB group. The third scenario aims to set up a complete service model where Premium Service is targeted for conversational services while an Olympic Services sub-model is targeted for relative data traffic differentiation and the Best Effort Service is targeted for backwards compatibility. The focus is more on service model implementation versatility than service model design.

4.1 Scenario 1: schedulers comparison for delay and jitter sensitive traffic

This scenario is based on a performance analysis conducted by TF-TANT on a real IP DiffServ-enabled network; in particular, the results coming from phase 2 of the Differentiated Services experiments, group 1 [48], are considered.

Aims: in the TF-TANT experiments, a test bed is set up to study the behaviour of both WFQ (SCFQ Cisco implementation) and PQ schedulers with delay and jitter sensitive traffic. The following tests are considered:

- A. WFQ: the relationship between EF service rate and average OWD vs. packet size (par 5.1.2 [48])
- B. PQ: the impact of BE packet size on EF OWD and IPDV (par 5.1.3 [48])
- C. WFQ vs. PQ: the comparison of average OWD, average IPDV, OWD frequency distribution and IPDV frequency distribution (par 5.1.4 [48])

Set-up: the simulated test bed in Figure 4.1 differs from the real one on the following points:

- In the real scenario, the traffic generator and sink are connected to a switched LAN to reduce the synchronization error. Here, the choice is to not model the LAN because the simulator program runs in a stand-alone machine and a general system clock is available to the whole network.
- In the real scenario, the DiffServ enabled router has a Transmission queue (TX queue) cascaded to the queueing system; when a scheduler chooses a packet to send, it sends the candidate packet to this queue. In the simulated scenario, TX queue is not implemented because this feature is not available in the current NS release. Therefore, the influence of its queue length on performances cannot be checked.

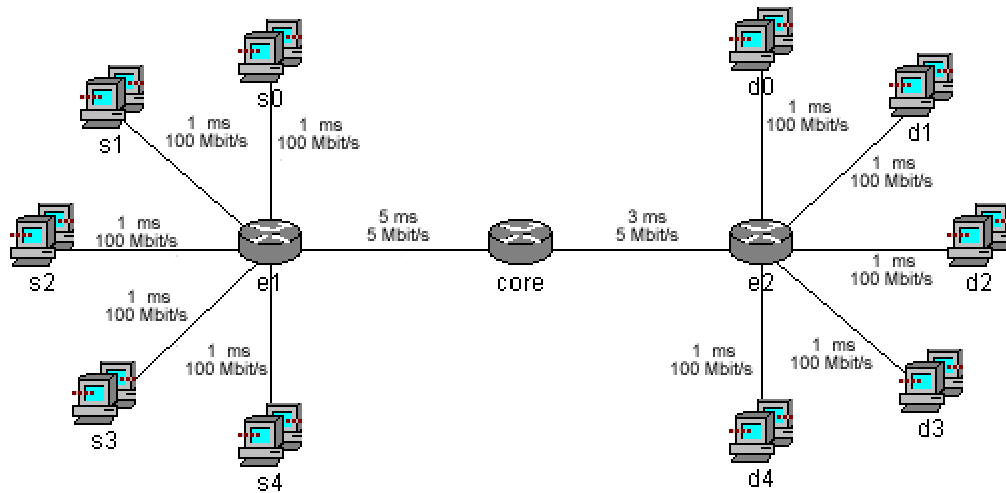


Figure 4.1 Scenario 1: test bed

Test A - settings: in this test, the WFQ scheduler and two queues are configured in the e1->core egress interface to support both EF and BE aggregates. The hypothesis for the EF aggregate is to have traffic entering the EF queue at a constant rate of 300 Kbps. For this reason, the EF aggregate is generated by a unique CBR source and is policed with a token bucket meter and a dropper as described in Table 4.1. To avoid synchronization problems, background traffic is generated with several CBR sources whose rate is chosen from a uniform random

distribution in the range [10 Kbps, 100Kbps], while the starting time is chosen from a uniform random distribution in the range [0s,5s]. The BE packet size is always 1000 bytes.

Table 4.1 TCS for e1->core egress interface

Aggregate	DSCP value	Assigned bandwidth	Arrival traffic rate	Meter	Dropper
EF	46	Variable, depending on STAR	300 Kbps	Token Bucket CIR 300 Kbps CBS 1 packet	Drop out of profile
BE	0	remaining	2.3 Mbps	×	Drop on threshold

In [19] a Service-To-Arrival-Ratio (STAR) is defined as:

$$S_r * l_r = A_r * STAR$$

where S_r is the service rate expressed as a fraction of the line rate, l_r is the link rate and A_r is the EF arrival rate. The impact of STAR parameter on OWD is analyzed.

A single simulation conducted for a 50-second period is characterized by a certain EF packets size and a certain STAR parameter according to the TF-TANT settings. The average OWD is computed on the whole period.

Test A - results: looking at Figure A.1 and Figure A.2, performance analysis results can be compared. The simulated network provides behavior similar to that of the real one. Increasing the STAR parameter significantly reduces the OWD, in particular for large packet sizes. The gain is relevant until the STAR parameter equals 4. A difference is perceivable for STAR parameter greater than 1 and short packets size. The simulation experiment shows a stable OWD, while the experiment on the real network shows a raising OWD. Finally, in the simulated network, as well as in the real one, the IPDV is not sensitive to the STAR parameter.

Test B - settings: in this test, the PQ scheduler and two queues are configured in the e1->core egress interface to support both EF and BE aggregates. The hypothesis for the EF aggregate is to have traffic entering the EF queue at a

constant rate of 300 Kbps. For this reason, the EF aggregate is generated by a unique CBR source and is policed with a token bucket meter and a dropper as described in Table 4.2. To avoid synchronization problems, background traffic is generated with several sources whose rate is chosen from a uniform random distribution in the range [10 Kbps, 100Kbps], while the starting time is chosen from a uniform random distribution in the range [0s, 5s]. BE packet size varies from 100 bytes to 1450 bytes according to the 'TF-TANT' settings. Only the deterministic distribution is considered because in *NS*, there is no availability for traffic agent generating packets whose size is chosen from a geometric distribution.

Table 4.2 TCS for e1->core egress interface

Aggregate	DSCP value	Assigned bandwidth	Arrival traffic rate	Meter	Dropper
EF	46	300 Kbps	300 Kbps	Token Bucket CIR 300 Kbps CBS 1 packet	Drop out of profile
BE	0	remaining	2.3 Mbps	×	Drop on threshold

A single simulation conducted for a 50-second period is characterized by a certain EF packets size and a certain BE packet size according to the 'TF-TANT' settings. The average OWD and average IPDV are computed on the whole period.

Test B - results: looking at Figure A.3 and Figure A.4, performance analysis results can be compared. The average OWD increases with BE packet size as expected, but in the simulated network, this parameter raises less than in the real network. To have a more faithful comparison, the OWD should be normalized to the optimal one in both networks. About the average IPDV, the results present more discrepancies. To better understand this parameter behavior in the simulated network, simulations have been repeated for a wider set of BE packet size values and a diagram is presented in Figure A.4. For 1024 bytes, the EF packet size presents a sawtoothed behavior. More research should be dedicated to this difference.

Test C - settings: in this test, two queues are configured in the e1->core egress interface to support both EF and BE aggregates. The hypothesis for the EF aggregate is to have traffic entering the EF queue at a constant rate of 300 Kbps. For this reason, the EF aggregate is generated by a unique CBR source and is policed with a token bucket meter and a dropper, same as Test B (Table 4.2). To avoid synchronization problems, the background traffic is generated with several sources whose starting time is chosen from a uniform random distribution in the range [0s, 5s]. Moreover, each source has a 100 Kbps rate, but the packet size varies from 64 to 1472 bytes (64 bytes increment) for a total of 23 flows.

A single simulation conducted for a 50-second period is characterized by a certain EF packet size and a certain scheduler according to the TF-TANT settings. The average OWD, the average IPDV, the OWD frequency distribution and the IPDV frequency distribution are computed on the whole period.

Test C - results: Figure A.5 and Figure A.6 compare the average OWD for different EF packet sizes and the OWD frequency distribution for both 128 and 1518 byte EF packet size. For these parameters, the simulator well models the real network. PQ always performs better than WFQ, even if for a small packet size the advantage is minor. There is no overlapping for 64-byte EF packet size and this is the main difference against the real network. The OWD frequency distribution shows the same relative behavior, the more the EF packet size increases, the more significant values on x axis become dense. Furthermore, the distance between higher value of PQ and WFQ scheduler increases. As well as in the real network, in the simulation experiment, the WFQ profile moves to higher delays faster than the PQ profile.

In Figures A.7 and A.8, the average IPDV for different EF packet sizes and the IPDV frequency distribution for 128 and 1518 byte EF packet sizes are shown. In the simulation experiment, the average parameter shows a different behavior than the same in the real network. But looking at the frequency distribution behavior, an

important property is confirmed: PQ and WFQ in this kind of scenario give the same performance.

Encountered difficulties and possible improvements: the first problem was to understand how to properly model the test bed and to find the right balance between simplification and being true to reality. Then implementing the SCFQ scheduler and collecting performance parameters required a good understanding of both the *NS* architecture and DiffServ module and the following coding.

Some trial has been done to explore the possible motivation of IPDV behaviour with PQ scheduler (refer to Figure A.4). An attempt to create a real EF aggregate with several sources pointed out an incorrect behaviour of token bucket with MBS set to one packet. In some simulation, the rate of in-profile EF packets was 200 Kbps even if the token bucket was set with a CIR equal to 300 kbps. The reason could be found in the constant parameter used in the algorithm to meter the traffic. This parameter, usually set to 1, gives more or less sensitiveness to rate oscillation. It is interesting to notice that in the real network where Cisco routers were deployed, the minimum MBS parameter could be set at 8000 bytes.

Another noticed fact is that using the WFQ, 128-byte EF packets size and big background packets size, the EF queue showed a maximum burstiness equals to 2. Informal simulations with more active queues showed a bigger MBS, up to 6 for 5 queues. This behaviour is confirmed by the theory. Using WFQ (SCFQ implementation), a packet entering the queue i can experience the following maximum latency due to the scheduler:

$$Latency = \frac{L_i}{\rho_i} + \frac{L_{\max}}{r}(V-1) \quad [49]$$

L_i is the maximum packet size of session i , $L_{\max}$ is the maximum packet size among all sessions, ρ_i is the allocated rate to session i , r is the total service rate and V is the number of active sessions. Hence, queue latency due to the WFQ scheduler increases with the number of active queues/sessions and with the packet size.

Bigger maximum burstiness has been recorded for small EF packet size than for big EF packet size. This does not contrast the theory because the measurement unit is packet number and because a CBR source, for a small packet size, generates more packets per time unit.

Future work could focus on modelling the Transmission Queue and on building a traffic agent with packet size geometrically distributed. Moreover, simulations could be repeated for other kinds of schedulers implemented in this thesis work.

4.2 Scenario 2: the level of importance differentiation in the AF PHB group

The Assured Forwarding PHB group aims to support the level of importance differentiation among packets crossing a DS domain. MRED (Multi-level RED) is a generic term used to describe any scheme where the drop probability for packets with a different drop precedence needs to be calculated independently for each drop precedence.

Aims: this scenario is targeted as a test bed for RED mechanisms evaluation in a large-scale internet traffic where Telnet, FTP and HTTP aggregates are injected in the same AF PHB group. The TCP protocol carries all three traffic categories and its congestion management future is considered. WRED [50] is the selected mechanism for admission queue policy in the AF queue because it is suitable for TCP congestion avoidance and because it is widely deployed.

By randomly dropping packets prior to periods of high congestion, WRED tells the packet source to decrease its transmission rate. Since the packet source is using TCP, it will decrease its transmission rate until all the packets reach their destination, which indicates that the congestion is cleared. WRED reduces the chances of tail drop by selectively dropping packets when the output interface begins to show signs of congestion. By dropping some packets early rather than waiting until the queue is full, WRED avoids dropping large numbers of packets at once and minimizes the chances of global synchronization. Thus, WRED allows the transmission line to be used fully at all times. Finally, WRED allows simultaneous, but differentiated packet dropping using packet header information (drop precedence level encoded in the DSCP).

The purpose is to find the best parameter set that minimizes the number of TCP retransmissions due to packet loss and, at the same time, gives the expected level of importance differentiation.

Set-up. the starting point of the test bed set-up is the large-scale web traffic scenario provided as example in the current *NS* release (ns2.1b8a/tcl/ex/large-scale-web-traffic.tcl). In Figure 4.2, the topology of the complete network is depicted. 469 nodes composed of 6 routers and 463 hosts are deployed. Link characteristics are shown except for the following nodes where described parameters are chosen at run-time:

- the n6-n45 nodes delay is a distributed uniform variable (10 ms to 100 ms range)
- the n46-n465 nodes bandwidth is a distributed uniform variable (22 Mbit/s to 32 Mbit/s range)
- the n46-n465 nodes delay is a distributed uniform variable (10 ms to 100 ms range)

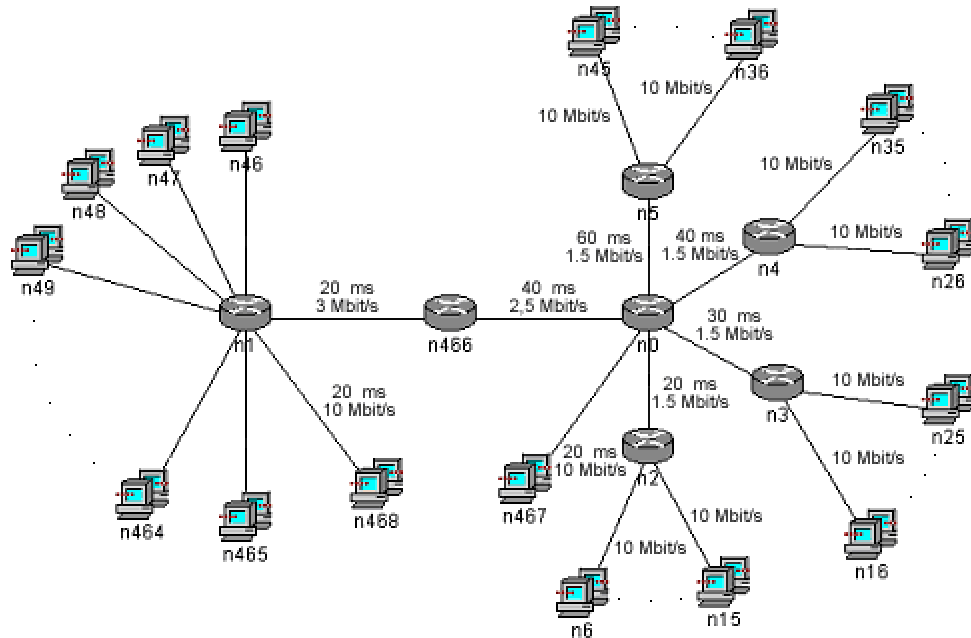


Figure 4.2 Scenario 2: test bed

Router n0 is DiffServ enabled on its outgoing interface connected to router n466 and in Table 4.3, the settings can be checked. The chosen scheduler is SFQ because it has been the best performing during informal simulations. Assigned weights are 17 for the AF PHB and 3 for the Default PHB, which meets the recommendation of the DiffServ WG for which a minimum amount of bandwidth should be always available to the Default PHB.

Table 4.3 TCS for n0->n466 egress interface

PHB	Traffic type	DSCP value	Drop precedence level	Bandwidth	Meter	Dropper
AF11 AF12 AF13	Telnet FTP HTTP	10 12 14	0 1 2	85 %	×	WRED
Default	other	0	×	15 %	Token Bucket CIR 500 kbps CBS 10k bytes	

Traffic description: concerning HTTP traffic, no changes have been made to the provided example. The web servers are n6-n45 hosts, while the web clients are n46-n465 hosts. The following parameters describe the HTTP traffic model:

- number of HTTP sessions: 400
- inter-session time: exponential distributed variable of 1 s average
- number of page per session: constant random variable of 250 average
- inter-page time: exponential distributed variable of 15 s average
- page size: constant random variable of 1 (Kbyte) average
- inter-object time: exponential distributed variable of 0.01 s average
- object size: Pareto distributed variable of 12 (Kbytes) average and 1.2 shape

In [51], new web traffic analyses are presented and can be used to update this traffic model. FTP traffic is generated using default values and 50 FTP connections are established using the same web client and server nodes. Telnet traffic is generated using the tcplib distribution (see tcplib-telnet.cc) for the inter-arrival time and 50 Telnet connections are established among the same nodes as FTP. Both FTP and Telnet traffics are activated during the first 50 seconds of simulation. Background

traffic is sent as CBR flow from n467 node to n468 node in a way that the Default queue is always full.

The first simulations were targeted to understand the behaviour of each AF aggregate without a congestion situation. Hence, for each traffic type, parameters like throughput, TCP window size frequency distribution, TCP RTT frequency distribution and queue length were monitored without the presence of the other traffic types inside the AF group. The results are shown in Figures B.1, B.2 and B.3.

With further simulations, the same parameters were collected (except for queue length) in the absence of Services Differentiation (Figure B.4).

And finally, simulations have been conducted with all the enabled DiffServ capabilities and for several WRED parameters sets. Global result evaluation is made comparing the following parameters: goodput, packet loss due to buffer overflow and packet loss due to WRED mechanism, all of them collected on a DSCP basis.

WRED parameter settings: three general parameter set structures are usually considered. They are staggered, partially overlapped and overlapped. Two different settings are tested for each structure. The drop probability does not change and is 0.1 for DP0 (Drop Precedence level 0), 0.2 for DP1 and 0.5 for DP2. In Figure 4.3, all parameter sets are shown.

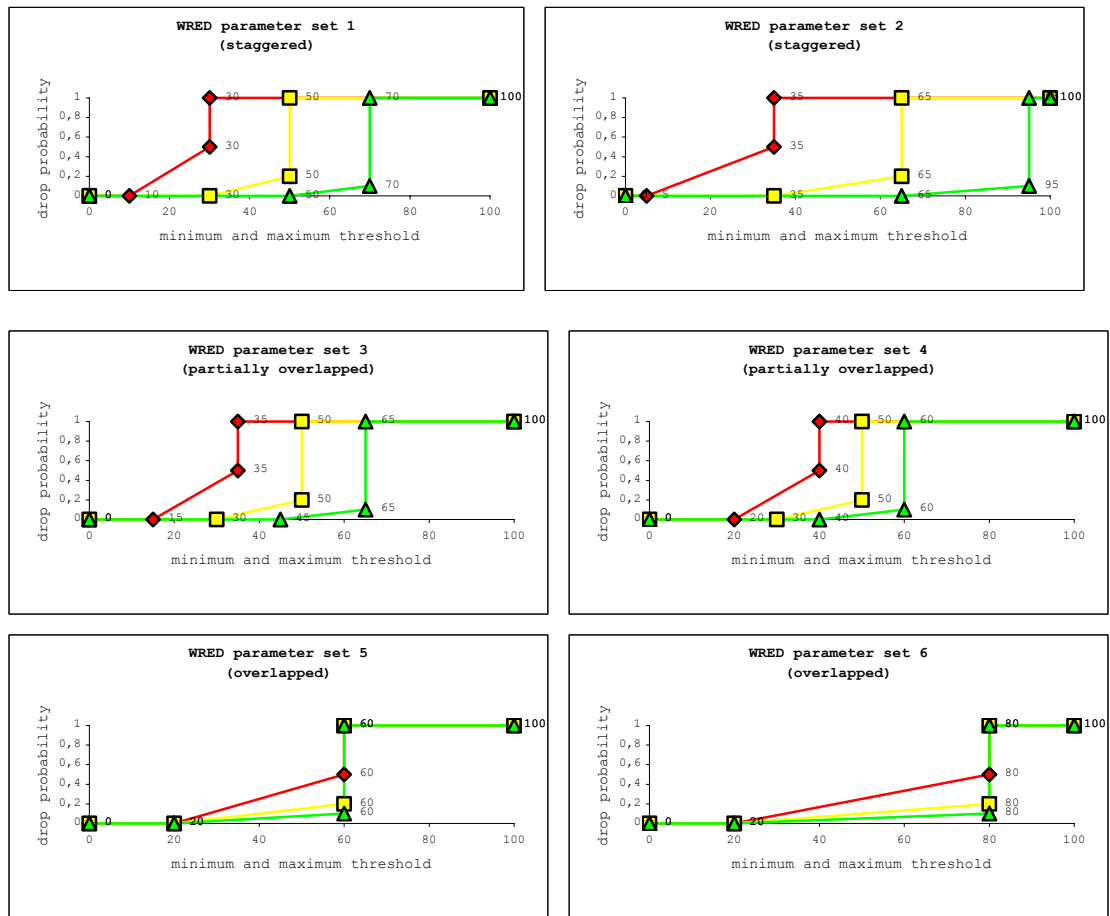


Figure 4.3 WRED parameter sets

For each simulation, the following parameters were also monitored:

- *Departure rate per traffic category and per aggregate:* these rates are measured at the n0 router egress interface
- *Instantaneous number of enqueued packets per traffic category:* the AF queue in the n0 router egress interface is injected with Telnet, FTP and HTTP traffic; the number of packets for each category in this queue is monitored
- *Number of received ack:* this represents the number of ack that crosses the n0 router from the n466 router to the TCP senders
- *TCP Round-Trip Time Frequency Distribution* on a DSCP basis
- *TCP Window Size Frequency Distribution* on a DSCP basis

Results: in Table 4.4, numerical results are shown, while in Figure B.11, graphical global results are presented. The WRED staggered parameter set structure gives more protection to the DP0 packets (Telnet traffic) in case of congestion. The WRED overlapped parameter set structure gives instead the best level of importance differentiation among the three traffic categories sharing the packet loss in a proportional fashion.

The highest level of goodput is detected for the overlapped parameter set structure even though differences among the whole sets are not so relevant.

Table 4.4 Numerical results

		<i>Parameter set 1</i>	<i>Parameter set 2</i>	<i>Parameter set 3</i>	<i>Parameter set 4</i>	<i>Parameter set 5</i>	<i>Parameter set 6</i>
caPL	DP0	0%	0%	0%	0.03%	2.91%	2.39%
	DP1	0.01%	0%	0.54%	1.18%	4.84%	4.40%
	DP2	25.12%	23.61%	27.45%	23.02%	20.50%	18.95%
boPL	DP0	0%	0%	0%	0%	0%	0.03%
	DP1	0%	0%	0%	0%	0%	0.01%
	DP2	0%	0%	0%	0%	0.02%	0.09%
Goodput	DP0	0.87	0.87	0.88	0.89	0.89	0.89
	DP1	0.91	0.91	0.91	0.92	0.91	0.91
	DP2	0.78	0.79	0.78	0.81	0.83	0.83

As it is expected, simulations with only one kind of active AF traffic each time, showed the biggest window size and the smallest RTT (Figures B.1, B.2 and B.3). In fact, during these simulations, no packet loss was detected, enabling the TCP senders to work with wider window sizes.

Conversely, in the simulation with disabled DiffServ capabilities, the worst RTT have been measured for all the AF traffic categories. The following packet losses due to buffer overflow have been recorded: 6.83% for Telnet aggregate, 4.80% for FTP aggregate and 17.61% for HTTP aggregate. Comparing these percentages with the percentages showed in Table 4.4, it is evident how congestion avoidance mechanisms are effective on a giving level of importance differentiation.

It is important to remark that WRED works on a per-packet basis, therefore it cannot provide link sharing capabilities among the three TCP protocol-based classes of traffic.

4.3 Scenario 3: a complete service model

In this last scenario, a complete approach to services differentiation is done. Starting with the design of a service model suitable for most of the current applications and passing through the implementation, a final evaluation will be done to show both DiffServ architecture skills and NS simulation capabilities.

Aims: this scenario is an attempt to set up a complete service model. The aims are to check DiffServ module versatility in services definition and to provide a set of performance parameters suitable for mechanisms selection and parameters setting. The provided service model is just an example, the focus is more on service model implementation than on service model design. The purpose is to show how to build and how to measure the performances of a service model.

Set up: a complete service model including a Premium Service for conversational traffic, an Olympic Services sub-model for relative service differentiation and a Best Effort Service for backwards compatibility is presented.

In Figure 4.4, the topology is depicted. 771 nodes composed by 6 routers and 765 hosts are deployed. Link characteristics are shown except for the following nodes where the described parameters are chosen at run-time:

- the n6-n45 nodes delay is a distributed uniform variable (10 ms to 100 ms range)
- the n46-n465 nodes bandwidth is a distributed uniform variable (22 Mbit/s to 32 Mbit/s range)
- the n46-n465 nodes delay is a distributed uniform variable (10 ms to 100 ms range)

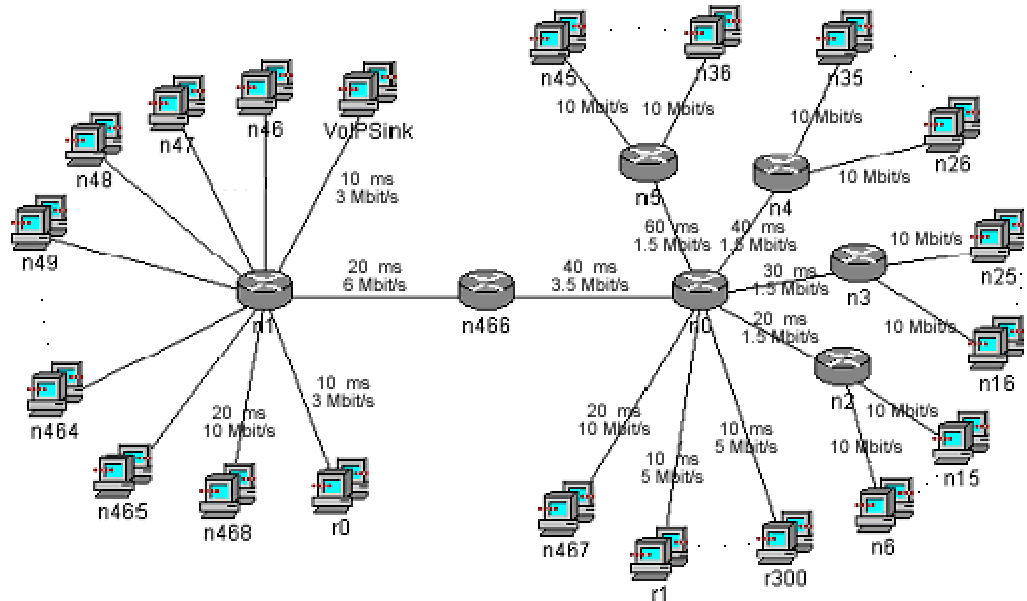


Figure 4.4 Scenario 3: test bed

Router n0 is DiffServ enabled on its outgoing interface connected to router n466 and, in Table 4.5, settings can be checked. The chosen scheduler is LLQ which provides a priority scheduler for the Premium Service. SFQ is the selected sub-scheduler for remains services. This choice is suggested by the need of a scheduler with link-sharing properties, fair excess bandwidth distribution and a low level of complexity. Moreover, informal simulations have showed that this is the best performing among the schedulers implemented in the *NS* DiffServ module.

Table 4.5 TCS for n0->n466 egress interface

Traffic class		Traffic type	PHB	DSCP value	Bandwidth		Meter	Dropper
Premium		VoIP and video telephony	EF	46	500 Kps		Token Bucket CIR 500 kbps CBS 10k bytes	Drop out of profile
O L Y M P I C	Gold	Audio and video streaming	AF11 AF12	10 12	R E M A I N I N G	30%	TSW2CM CIR 600 kbps	RIO-C Green: 60,110,0.02 Yellow: 30,60,0.6
	Silver	Telnet FTP	AF21 AF22	18 20		30%	×	WRED 30,50 0.1,0.2
	Bronze	HTTP	AF31	26		30%	×	WRED 30,60 0.5
Best Effort		other	Default	0		10%	Token Bucket CIR 400 kbps CBS ×	Drop out of profile

Traffic description: the selected VoIP traffic codec is ITU-T G.723.1. This codec is targeted for telephone applications. The algorithm is based on the CELP principle and the bit rate is 6.3 kbps (half-duplex). The VoIP traffic is modelled as an exponential ON/OFF distribution (1.004-second ON period, 1.587-second OFF period) and two frames (30 ms audio sample each frame) are carried in each packet (i.e. 24+24 bytes RTP payload size). 200 VoIP connections, that are 5K-second long, are activated in the first 200 seconds. The involved nodes are the r1-r200 as senders and the VoIPSink as receiver. A service rate limiter policy is applied to avoid starvation of other services. A 500 kbps CIR is set for the token bucket, this leads to a packet loss of almost 1% that is tolerated for this kind of application. In a real VoIP service implementation, global rate is also policed by the Call Admission Control mechanisms.

In the Gold Service audio streaming traffic is injected. Traffic model is based on RealAudio agent that generates ram-encoded traffic. Connections are set as the provided example with the current NS release (ns2.1b8a/tcl/ex/realaudio/ramodel.tcl). Parameters for the agent are :

- packet size: 245 bytes
- burst time: 0.05 ms
- idle time: 1800 ms

Number of sequential flows per user, flow duration and flow rate parameters are taken from empirical data files. The nodes involved are r1-r300 (senders) and r0 (receiver). A Time Sliding Window Two Color algorithm is deployed to measure the arrival rate and to mark packets when the rate overcomes a certain threshold (in this simulation, this threshold is set to 66% of the assigned bandwidth). The differentiation is used to early drop packets with different drop probability. The principle is that it is preferable to lose the same amount of packets on a longer period than to lose bursts that could belong to the same connection. The quality of the single connections should be higher, but no study has been dedicated to that.

The selected dropper is RIO-C because it protects packets with a lower drop level probability better than WRED [52]. In this scenario, it is desired to always drop out-of-profile packets if there are any.

The Silver Service is targeted for Telnet and FTP traffic. A relative differentiation is given by early dropping Telnet traffic with lower probability. FTP traffic is generated using default values and 50 FTP connections are established among n6-n45 nodes (senders) and n46-n95 nodes (receivers). Telnet traffic is generated using the tcplib distribution (see tcplib-telnet.cc) for the inter-arrival time and 50 Telnet connections are established among the same nodes of FTP. Both FTP and Telnet traffics are activated during the first 50 seconds of simulation.

The Bronze Service is targeted for HTTP traffic. Web servers are n6-n45 hosts, while web clients are n46-n465 hosts. The following parameters describe the traffic model:

- number of HTTP sessions: 400
- inter-session time: exponential distributed variable of 1 s average
- number of page per session: constant random variable of 250 average
- inter-page time: exponential distributed variable of 15 s average
- page size: constant random variable of 1 (Kbyte) average
- inter-object time: exponential distributed variable of 0.01 s average
- object size: pareto distributed variable of 12 (Kbytes) average and 1.2 shape

In [51], new web traffic analyses are presented and can be used to update this traffic model. An early detection mechanism is deployed (WRED) to avoid excessive loss of bursts and global synchronization.

The Best Effort Service is used for packets which do not match in any other service. A minimum bandwidth is reserved to avoid starvation in the accomplishment of

DiffServ WG recommendations for Default PHB. The Best Effort departure rate is limited to 400 kbps to favour other services in sharing excess bandwidth.

The monitored parameters are:

- *Maximum Burst Size for VoIP queue:* this is because VoIP is a real-time service that needs strictly delay bound; the worst queueing delay can easily be checked with this parameter
- *Instantaneous number of enqueued packets per traffic category and per service*
- *Packet loss percentage due to congestion avoidance (caPL) per traffic category*
- *Packet loss percentage due to buffer overflow(boPL) per traffic category*
- *Departure rate per traffic category and per service*
- *TCP Round-Trip Time Frequency Distribution* on a DSCP basis for TCP-based traffic
- *TCP window size* on a DSCP basis for TCP-based traffic
- *Goodput* on a DSCP basis for TCP-based traffic

Moreover, average and frequency distribution for OWD and IPDV are monitored for both a single VoIP connection and a single RealAudio connection.

Results: this scenario proves that a good level of versatility has been reached and even such complex service model has been implemented.

Looking at Figure C.1, link sharing properties and excessive bandwidth distribution among active queues due the SFQ scheduler can be observed. Gold, Silver and Bronze Services have an allocated bandwidth of 900 Kbps each, while the Best Effort Service has an allocated bandwidth of 300 kbps. Since injecting RealAudio traffic in the Gold Service does not exploit the whole bandwidth except in a range near 5000 s, the scheduler shares excessive transmission resources to the other services. In this scenario the Best Effort Service is bounded to 400 kbps. Therefore,

higher services are favoured in sharing excess bandwidth. This is just a design choice and other solutions could be taken.

In Figure C.3, instantaneous queue length per traffic category is shown. As expected, LLQ scheduler gives the right prioritization to the VoIP traffic. Premium queue length never overcomes 10 packets and, considering the small packet size, this fulfills expectations. When a VoIP packet enters the queue, the scheduler can be busy, serving a packet of a low priority queue. As these packets are usually bigger and no pre-emption can be deployed, several other VoIP packets can enter the priority queue during service time. Therefore, the queue size is affected by the size of the packets in other queues. A more detailed discussion is provided in Scenario 1.

In Figure C.5, the two monitored connections are compared, one VoIP and one RealAudio. Both OWD and IPDV are strictly bounded for Premium Service, moreover they fit the ITU G.114 recommendation that suggests maximum OWD for IP telephony of 150 ms. In this example the VoIP connection experiences a maximum of 80 ms OWD, while RealAudio connection experiences a maximum of 400 ms OWD. As a remark, it is important to remember that even if RealAudio is a real-time service, it can tolerate higher OWD using buffering on the receiver side. This can be done because of the non-interactive nature.

In Figure C.6, packet loss due to congestion avoidance mechanisms and packet loss due to buffer overflow are shown. The first consideration is about the RealAudio aggregate. Depending on the tolerated packet loss percentage, the assigned bandwidth should be increased. In the Silver Service, Telnet and FTP aggregates receive the correct level of importance differentiation in terms of packet loss and delay (Telnet queue shorter than FTP). Among the Olympic Services, as expected, the Bronze Service gets the worst goodput, the worst packet loss and the worst RTT.

Encountered difficulties and possible improvements: the traffic generator for VoIP is missing in the current *NS* and some effort has made to find a suitable one. For other traffic types, the provided example were used.

A wrong behaviour was firstly detected for RIO-C dropper. Even if correctly set, during the congestion situation most of the RealAudio packets were dropped due to buffer overflow instead because of the dropper. By looking into the mechanism and its implementation, a reason was found.

RIO-C is a RED-based dropper. RED algorithm [28] computes the average queue length of each packet arrival event as follows:

$$avg_i = avg_{i-1} + w_q(q - avg_{i-1})$$

where q is the current queue size, w_q is equal to 0.002 (in *NS* implementation).

In case the queue is found in idle state, the calculation is made as follows:

$$m = \frac{(time - q_time)}{s}$$

$$avg_i = (1 - w_q)^m avg_{i-1}$$

m represents the number of packets that might have been transmitted by the router during the time that the queue was idle. Therefore, $time$ is the arrival time of the current packet, q_time is the starting time of the queue idle status and s is defined as the typical transmission time for a small packet.

In the *NS* DiffServ module, s is expressed as $\frac{1}{PTC}$. Here PTC means Packet Time

Constant and is computed as follows:

$$PTC = \frac{outLinkBW}{8 \bullet MPS}$$

$outLinkBW$ is automatically assigned with the line rate (bit per second) and MPS is the mean packet size (has to be set by the user).

Since the queue is served with a rate smaller than the line rate, to use the total one brings to the wrong computation of the m parameter. In particular, it is overestimated and the computed average queue is consequentially underestimated.

And that is why the dropper does not work properly; configured thresholds seldom are overcome by the wrong avg , while the real queue length increases until it generates buffer overflow.

After having changed the code in a way that allows to set *outLinkBW* for each configured queue, and after having set it to the correct rate, the RIO-C dropper worked properly.

5. CONCLUSIONS

All through this work several issues have been taken into consideration and various solutions have been proposed. The final results are classified into three categories:

1. Service models simulation capabilities of the Network Simulator
2. Performance parameters monitoring
3. Mechanisms test beds

5.1 Service models simulation capabilities of the Network Simulator

The proof analysis conducted in chapter 3 raised several limitations in the *NS* DiffServ module and a sub-set of them have been identified and solved. The target was to enable a good versatility of the Service Models implementation and performances evaluation.

The following key contributions have been reached:

- Schedulers. The scheduler mechanisms have been encapsulated in their own class and moved into a separate file; now a clear interface speeds the addition of new schedulers with only a minimal understanding of the rest of the module; the following new schedulers have been added: SCFQ, SFQ, LLQ, WFQ and WF²Q+
- Marking and metering. These functionalities have been decoupled and the possibility of marking on a per source-destination address basis, per transport protocol-type basis or per application-type basis has been enabled; once packets are marked, it is possible to meter aggregates on a per-codepoint basis

- Dropping. A Drop-out-of-profile packets dropper has been added giving the opportunity to allocate a rate limiter for the purpose of admission queue policy; this can be done coupling a token bucket meter for arrival rate metering, a marker for in/out-of-profile marking and a drop-out-of-profile packets dropper

Two relevant bugs in the *NS* module have been detected and fixed. The first one was related to the wrong behavior of the rate limiter capability of the PQ scheduler. The second one was related to the wrong behavior of the RIO-C dropper (a description of this problem can be found in the last part of paragraph 4.2).

5.2 Performance parameters monitoring

When a new mechanism is invented and implemented, there is the need of collecting performance parameters to understand how valuable it is. Furthermore, in the real networks, performance parameters are also used in the Network Operation & Management activities. For these purposes, several new parameters can be monitored.

They can be divided in:

- global performance parameters. They give information about performances on the whole packet forwarding path
- local performance parameters. They give information about performances on a single node

In the first category, the chosen parameters are:

- For UDP-based traffic
 - Average, instantaneous, minimum and frequency distributed OWD
 - Average, instantaneous, minimum and frequency distributed IPDV

- For TCP-based traffic
 - TCP Goodput on a DSCP basis
 - TCP Round-Trip Time on a DSCP basis, both instantaneous value and frequency distribution
 - TCP Window Size on a DSCP basis, both instantaneous value and frequency distribution

In the second category, the chosen parameters are:

- Instantaneous and average queue length on a queue basis or on a queue and drop precedence level basis
- Maximum burstiness for queue 0
- Departure rate on a queue basis or on a queue and drop level precedence basis
- Received packets, transmitted packets, dropped packets due to droppers and dropped packets due to buffer overflow, all on a DSCP basis and for both absolute and percentage values

5.3 Mechanisms test beds

Once a mechanism is implemented and a set of performance parameters is selected, there is the need for a test bed where the mechanism behaviour can be analysed.

In this work three test beds have been set for different purposes:

- Scenario 1. A test bed is provided for scheduler performances comparison with delay and jitter sensitive traffic; the topology and traffic settings come from experiments on a real IP network [48] and a comparative study has been conducted

- Scenario 2. A test bed for mechanisms testing with both AF PHB group and TCP-based traffic is presented. Three different kinds of traffic (Telnet, FTP and HTTP) compose the AF aggregate and resources are allocated in a way to create a congestion situation
- Scenario 3. In this last test bed, a complete service model is set to meet the needs of the current applications; a Premium Service is set to support conversational applications like VoIP, an Olympic Services sub-model is set to support relative services differentiation and a Best Effort Service is set for backwards compatibility

For all three scenarios, simulations have been conducted using available mechanisms, and performance parameters have been collected.

6. FUTURE WORK

Many are the opportunities to continue with this work and here a brief list is given:

- Redesigning the dropper mechanisms and policy mechanisms as has been done for the schedulers
- Enabling the hierarchical scheduling
- Integrating the new module with the MPLS module for collaborative simulations
- Working on the Network Operations & Management side joining on-going projects related to Bandwidth Broker and to Management Information Base implementation
- Adding performance parameters to measure end-to-end quality of service for single connections of applications like VoIP or RealAudio

REFERENCES

- [1] **A Brief History of the Internet** 4th August 2000
<http://www.isoc.org/internet/history/brief.html>
- [2] Kilkki, Kalevi **Differentiated Services for the Internet** Macmillan 1999
Technology Series
- [3] **Official DiffServ Working Group Web Site**
<http://www.ietf.org/html.charters/diffserv-charter.html>
- [4] **RSVP Working Group Home Page**
<http://www.ietf.org/html.charters/rsvp-charter.html>
- [5] **Specification of Guaranteed Quality of Service** September 1997
<http://www.ietf.org/rfc/rfc2212.txt>
- [6] **Specification of the Controlled-Load Network Element Service** September 1997
<http://www.ietf.org/rfc/rfc2211.txt>
- [7] **RSVP Refresh Overhead Reduction Extensions** April 2001
<http://www.ietf.org/rfc/rfc2961.txt>
- [8] **Generalized MPLS Signalling - RSVP-TE Extensions** October 2001
<http://www.ietf.org/internet-drafts/draft-ietf-mpls-generalized-rsvp-te-05.txt> (work in progress)
- [9] **Information technology - Telecommunications and information exchange between systems - Local and metropolitan area networks -Common specifications - Part 3: Media Access Control (MAC) Bridges: Revision. This is a revision of ISO/IEC 10038: 1993, 802.1j-1992 and 802.6k-1992. It incorporates P802.11c, P802.1p and P802.12e** ISO/IEC 15802-3: 1998
- [10] **Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers** December 1998
<http://www.ietf.org/rfc/rfc2474.txt>
- [11] **Definition of Differentiated Services Per Domain Behaviors and Rules for their Specification** April 2001
<http://www.ietf.org/rfc/rfc3086.txt>
- [12] **An Architecture for Differentiated Services** December 1998
<http://www.ietf.org/rfc/rfc2475.txt>
- [13] **MPLS Support of Differentiated Services** April 2001
<http://www.ietf.org/internet-drafts/draft-ietf-mpls-diff-ext-09.txt> (work in progress)
- [14] **New Terminology for DiffServ** August 2001
<http://www.ietf.org/internet-drafts/draft-ietf-diffserv-new-terms-05.txt> (work in progress)
- [15] **A Proposal to add Explicit Congestion Notification (ECN) to IP** January 1999
<http://www.ietf.org/rfc/rfc2481.txt>
- [16] **INTERNET PROTOCOL** September 1981
<http://www.ietf.org/rfc/rfc791.txt>
- [17] **Requirements for Internet Hosts - Communication Layers** October 1989
<http://www.ietf.org/rfc/rfc1122.txt>
- [18] **Type of Service in the Internet Protocol Suite** July 1992
<http://www.ietf.org/rfc/rfc1349.txt>

- [19] **An Expedited Forwarding PHB** June 1999
<http://www.ietf.org/rfc/rfc2598.txt>
- [20] **An Expedited Forwarding PHB** September 2001
draft-ietf-diffserv-rfc2598bis-02.txt (work in progress)
- [21] **Assured Forwarding PHB Group** July 1999
<http://www.ietf.org/rfc/rfc2597.txt>
- [22] Shannon, Robert E. **System Simulation: The Art and Science** Prentice-Hall 1975
- [23] **An Informal Management Model for DiffServ Routers** February 2001
<http://www.ietf.org/internet-drafts/draft-ietf-diffserv-model-06.txt> (proposed standard)
- [24] **A Framework for Integrated Services Operation over Diffserv Networks** November 2000
<http://www.ietf.org/rfc/rfc2998.txt>
- [25] **A Single Rate Three Color Marker** September 1999
<http://www.ietf.org/rfc/rfc2697.txt>
- [26] **A Two Rate Three Color Marker** September 1999
<http://www.ietf.org/rfc/rfc2698.txt>
- [27] **A Time Sliding Window Three Colour Marker (TSWTCM)** June 2000
<http://www.ietf.org/rfc/rfc2859.txt>
- [28] Floyd, S. **Random Early Detection gateways for congestion avoidance** IEEE/ACM August 1993
Jacobson, V. Transaction on Networking, Vol. 1, No 4
- [29] Goyal, M. **Effect of number of drop precedences in Assured Forwarding** GLOBECOM 99 December 1999
et al. Rio de Janeiro
- [30] **The Network Simulator (NS) – Home Page**
<http://www.isi.edu/nsnam/ns/index.html>
- [31] **SAMAN Project Home Page**
<http://www.isi.edu/saman/index.html>
- [32] **CONSER Home Page**
<http://www.isi.edu/conser/index.html>
- [33] **ACIRI Home Page**
<http://www.aciri.org/>
- [34] **Supplemental Information for the New Definition of the EF PHB** June 2001
<http://www.ietf.org/internet-drafts/draft-ietf-diffserv-ef-supplemental-01.txt> (work in progress)
- [35] **A One-way Delay Metric for IPPM** September 1999
<http://www.ietf.org/rfc/rfc2679.txt>
- [36] **IP Packet Delay Variation Metric for IPPM** February 2001
<http://www.ietf.org/internet-drafts/draft-ietf-ippm-ipdv-07.txt> (expired draft)
- [37] Parekh, Abhay K. **A Generalized Processor Sharing Approach to Flow Control in Integrated Services Networks: The Single-Node Case** IEEE/ACM 1993
Gallager, Robert G. Transactions on Networking Vol. 1, No. 3

- | | | | | |
|------|--|--|--|--------------|
| [38] | Bennett, J.C.R.
Zhang, Hui | Hierarchical Packet Fair Queueing Algorithms | IEEE/ACM Transactions on networking, Vol. 5., No. 5 | October 1997 |
| [39] | Golestani, S. J. | A self-clocked fair queueing scheme for broadband applications | Proc. IEEE INFOCOM'94, pages 636-646, | 1994 |
| [40] | Cheng, Haichen
Goyal, Pawan,
Vin, Harrick M. | Start-time Fair Queueing: A scheduling algorithm for integrated services packet switching networks | Proc. SIGCOMM 1996 | 1996 |
| [41] | Low Latency Queueing
http://www.cisco.com/univercd/cc/td/doc/product/software/ios122/122cgcr/fqos_c/fqcprt2/qcfconmg.htm#1001280 | | 16 August 2001
(Last modified) | |
| [42] | Bennett, J.C.R.
Zhang, Hui | Why WFQ is not good enough for integrated services networks | | 1996 |
| [43] | Bennett, J.C.R.
Zhang, Hui | WF²Q: Worst-case fair weighted fair queueing | Proc. IEEE INFOCOM '96, San Francisco, CA | March 1996 |
| [44] | Floyd, S.,
Jacobson, V | Link-sharing and Resource Management Models for Packet Networks | IEEE/ACM Trans. On Netw., Vol. 3 No. 4, pp. 365-386, | August 1995 |
| [45] | Suri, S.
Varghese, G.
Chandranmenon, G | Leap Forward Virtual Clock | | October 1997 |
| [46] | Stidialis, D.
Varma, A. | Efficient Fair Queueing Algorithms for Packet-Switched Networks | IEEE/ACM Transactions on networking, Vol. 6, No. 2 | April 1998 |
| [47] | Dovrolis, C.
Stiliadis, D.
Ramanathan, P. | Proportional Differentiated Services: Delay Differentiation and Packet Scheduling | SIGCOMM | 1999 |
| [48] | Ferrari, T. | Differentiated Services: Experiment Report, Phase 2 | Tiziana Ferrari Editor, v 1.1 | May 2000 |
| [49] | Stiliadis, D.
Varma, A. | Design and Analysis of Frame-based Fair Queueing: A new Traffic Scheduling Algorithm for Packet-Switched Networks | SIGMETRICS 96-5/96 Philadelphia, PA, USA | 1996 |
| [50] | WRED specifications
http://www.cisco.com/univercd/cc/td/doc/product/software/ios122/122cgcr/fqos_c/fqcprt3/qcfconav.htm#11086 | | 16 August 2001
(Last Modified) | |
| [51] | Donelson Smith, F.
Campos, F. H.
Jeffay, K.
Ott, D. | What TCP/IP Protocol Headers Can Tell Us About the Web | ACM SIGMETRICS | June 2001 |
| [52] | Makkar, R.
Lambadaris, I.
<i>et al.</i> | Empirical Study of Buffer Management Scheme for DiffServ Assured Forwarding PHB | Las Vegas Nevada | October 2000 |

APPENDIX A – RESULTS DIAGRAMS FOR SCENARIO 1

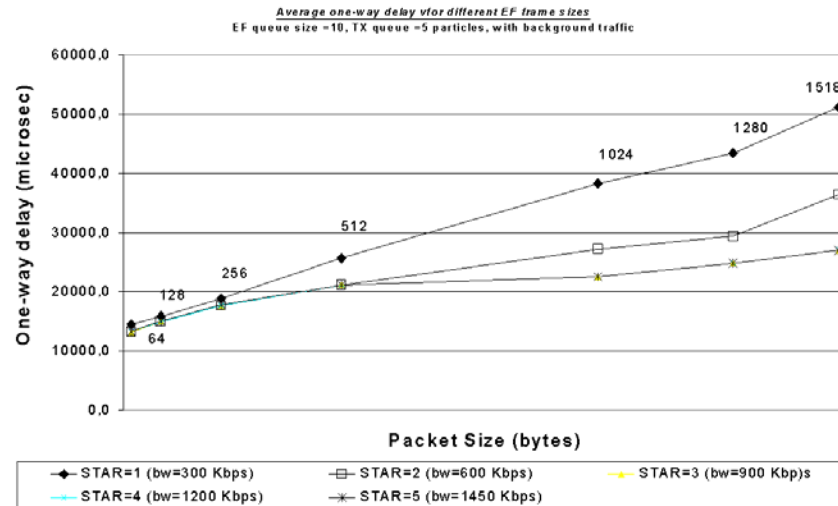


Figure A.1 Relationship between EF Service rate and average OWD vs. packet size for WFQ scheduler – real network (source [48])

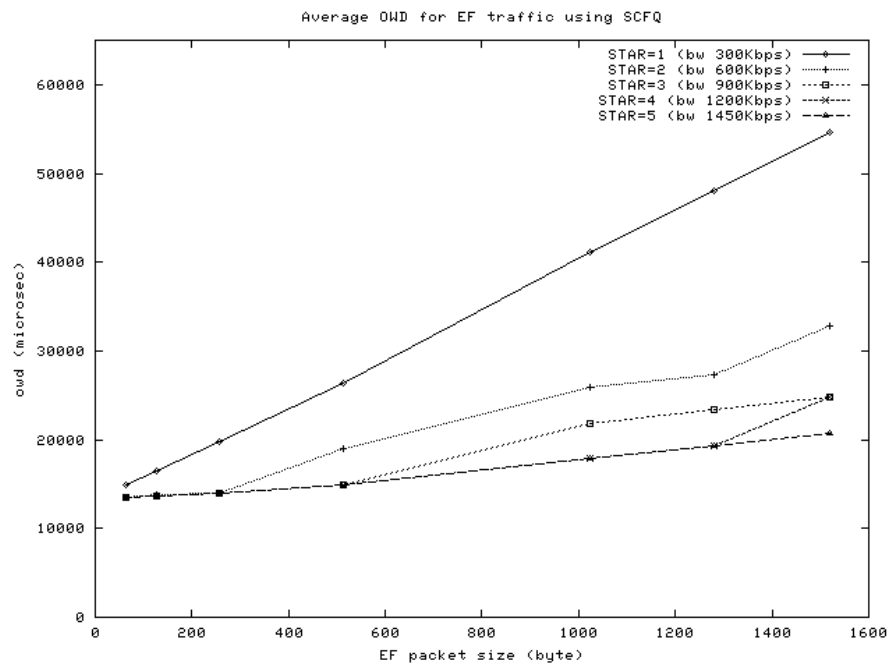


Figure A.2 Relationship between EF Service rate and average OWD vs. packet size for WFQ scheduler – simulated network

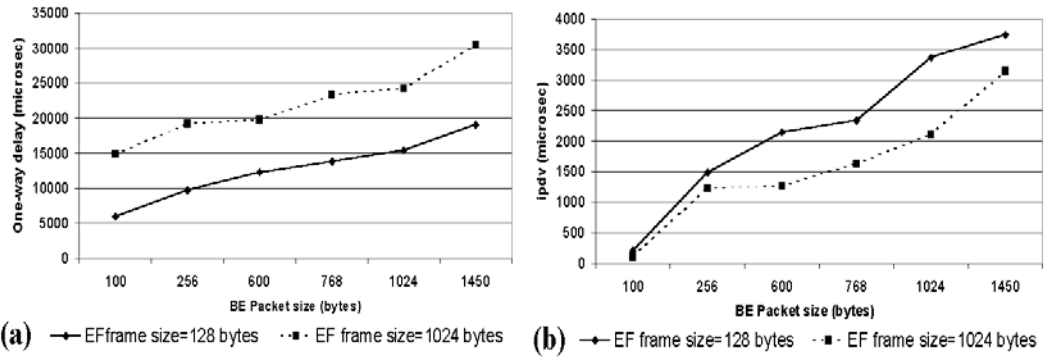


Figure A.3 Impact of BE packet size on EF OWD and IPDV for PQ scheduler - real network (source [48])

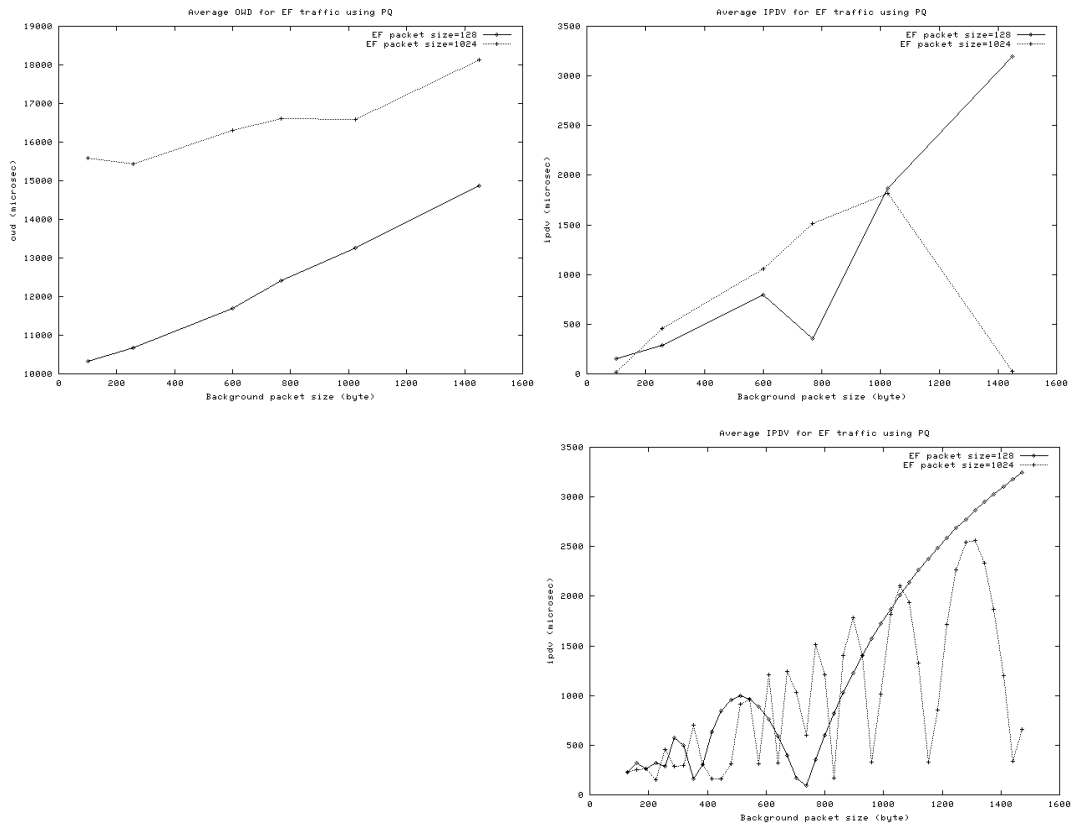


Figure A.4 Impact of BE packet size on EF OWD and IPDV for PQ scheduler - simulated network

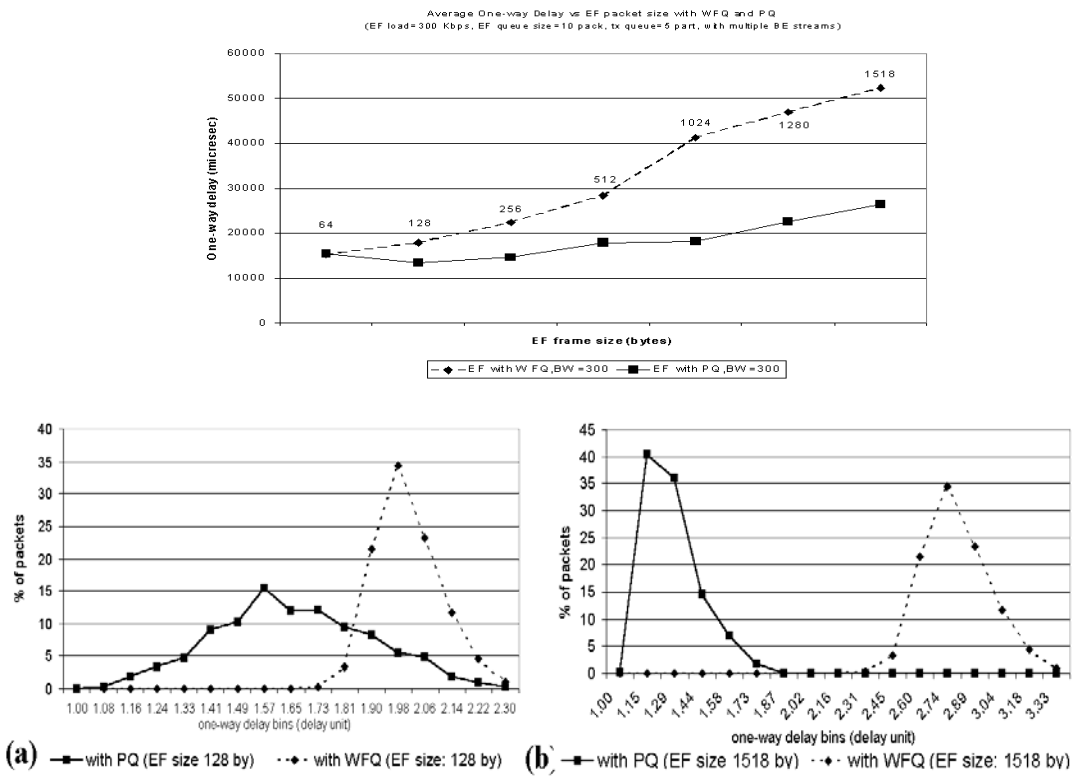


Figure A.5 Comparison of average and frequency distributed OWD with WFQ and PQ for different EF frame packet sizes - real network (source [48])

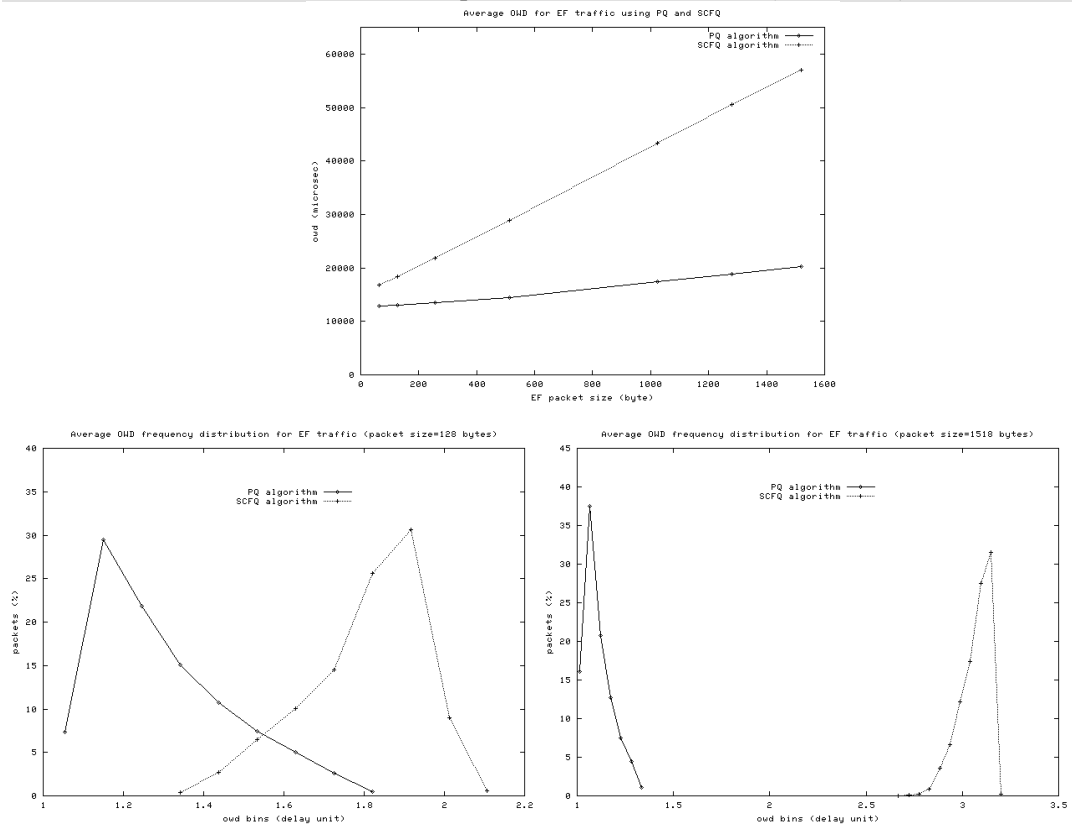


Figure A.6 Comparison of average and frequency distributed OWD with WFQ and PQ for different EF frame packet sizes – simulated network

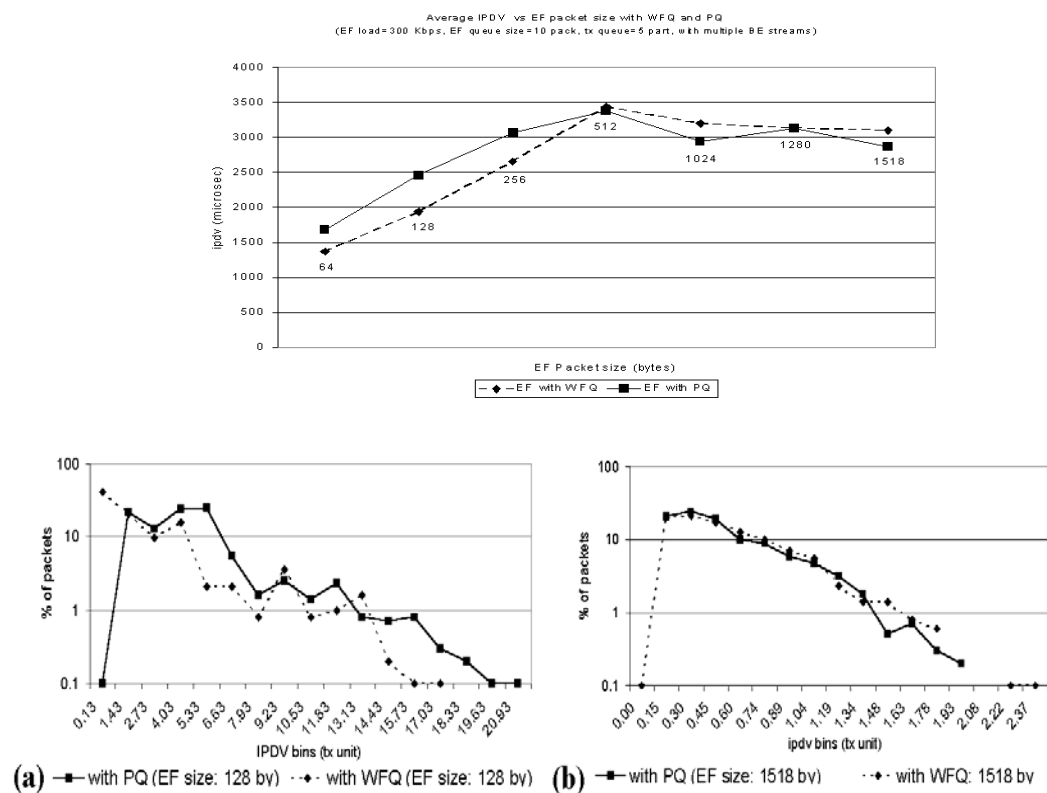


Figure A.7 Comparison of average and frequency distributed IPDV with WFQ and - real network (source [48])

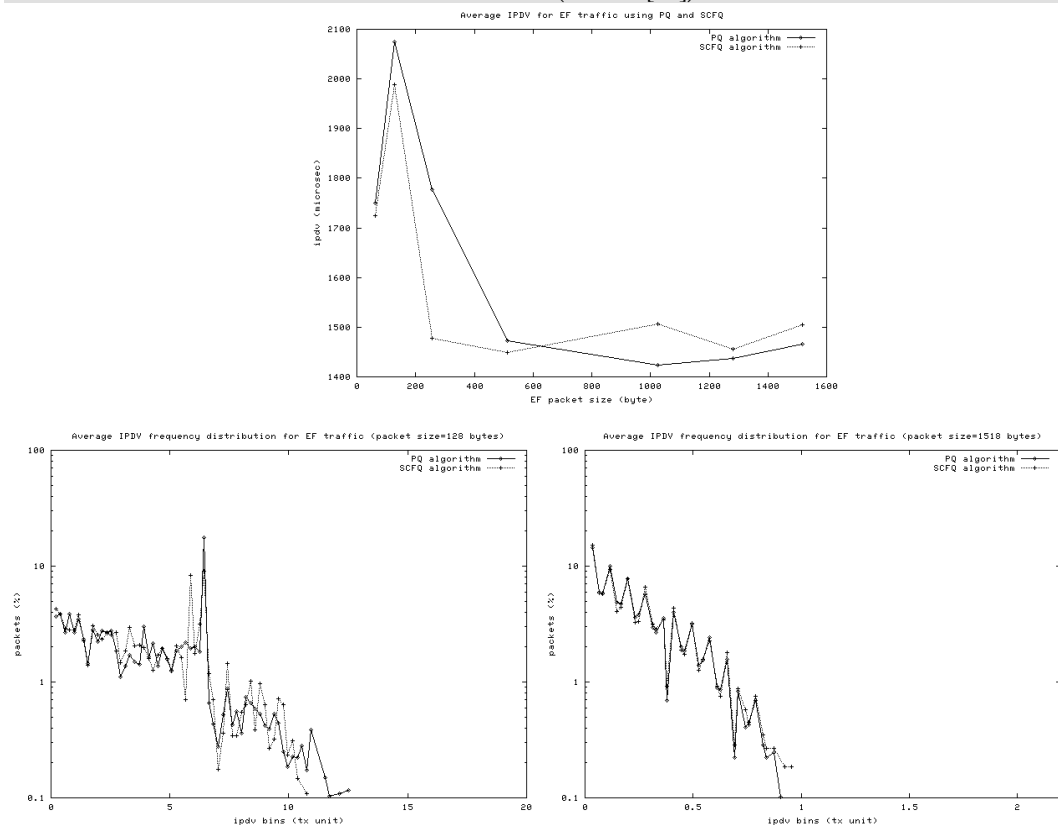


Figure A.8 Comparison of average and frequency distributed IPDV with WFQ and PQ – simulated network

APPENDIX B – RESULTS DIAGRAMS FOR SCENARIO 2

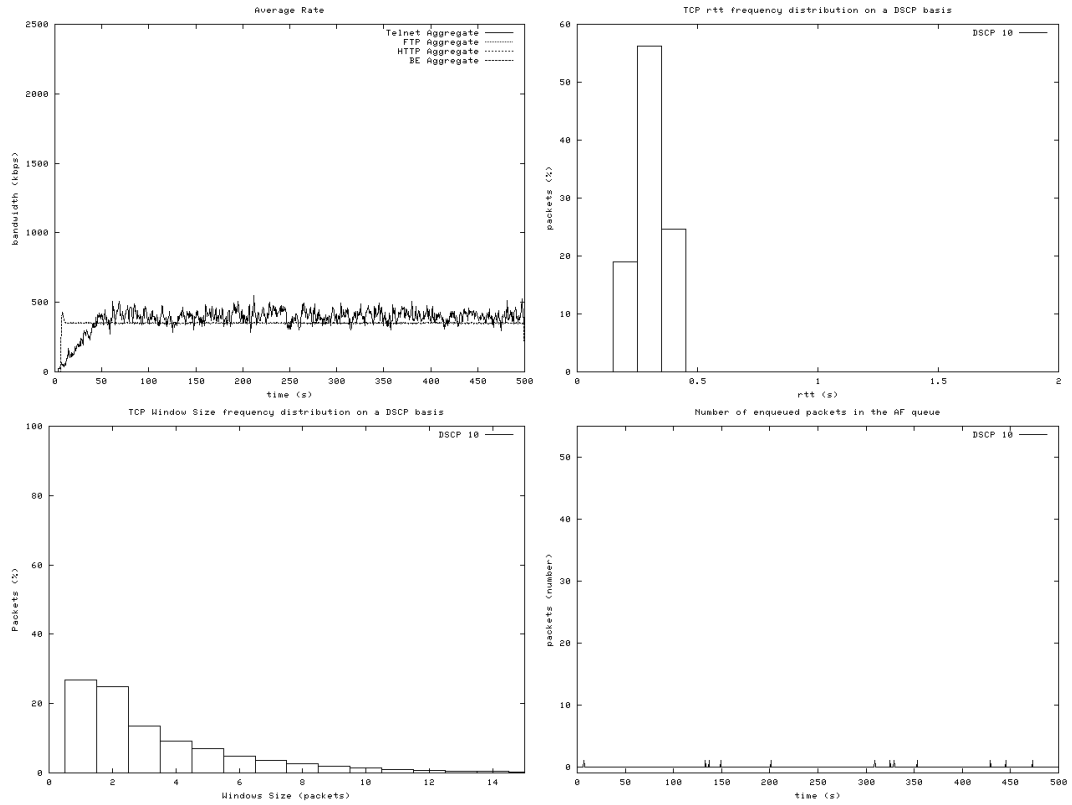


Figure B.1 Telnet traffic aggregate performance parameters in absence of other AF traffic

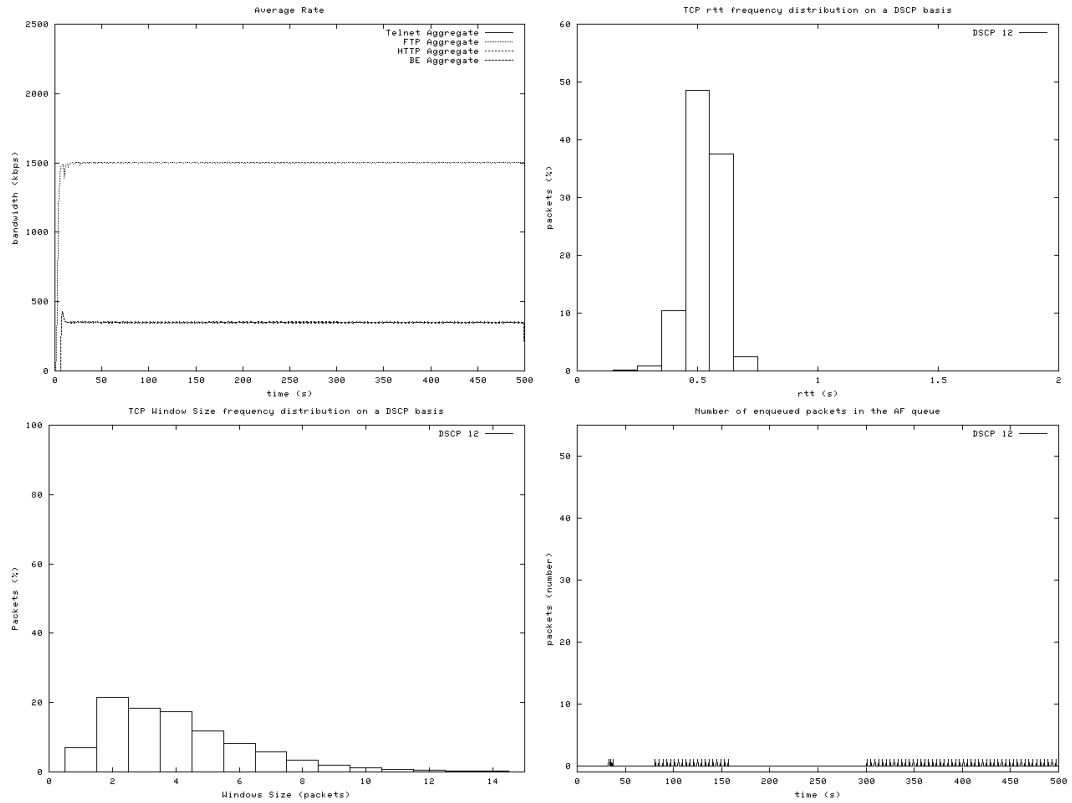


Figure B.2 FTP traffic aggregate performance parameters in absence of other AF traffic

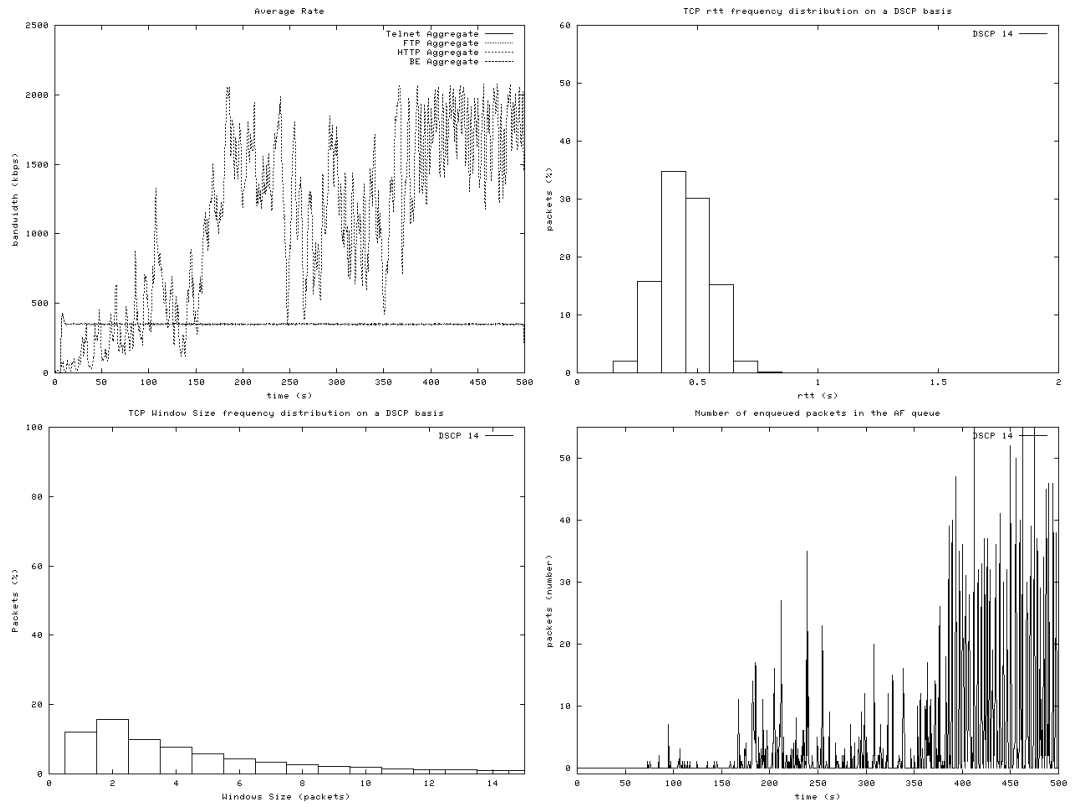


Figure B.3 HTTP traffic aggregate performance parameters in absence of other AF traffic

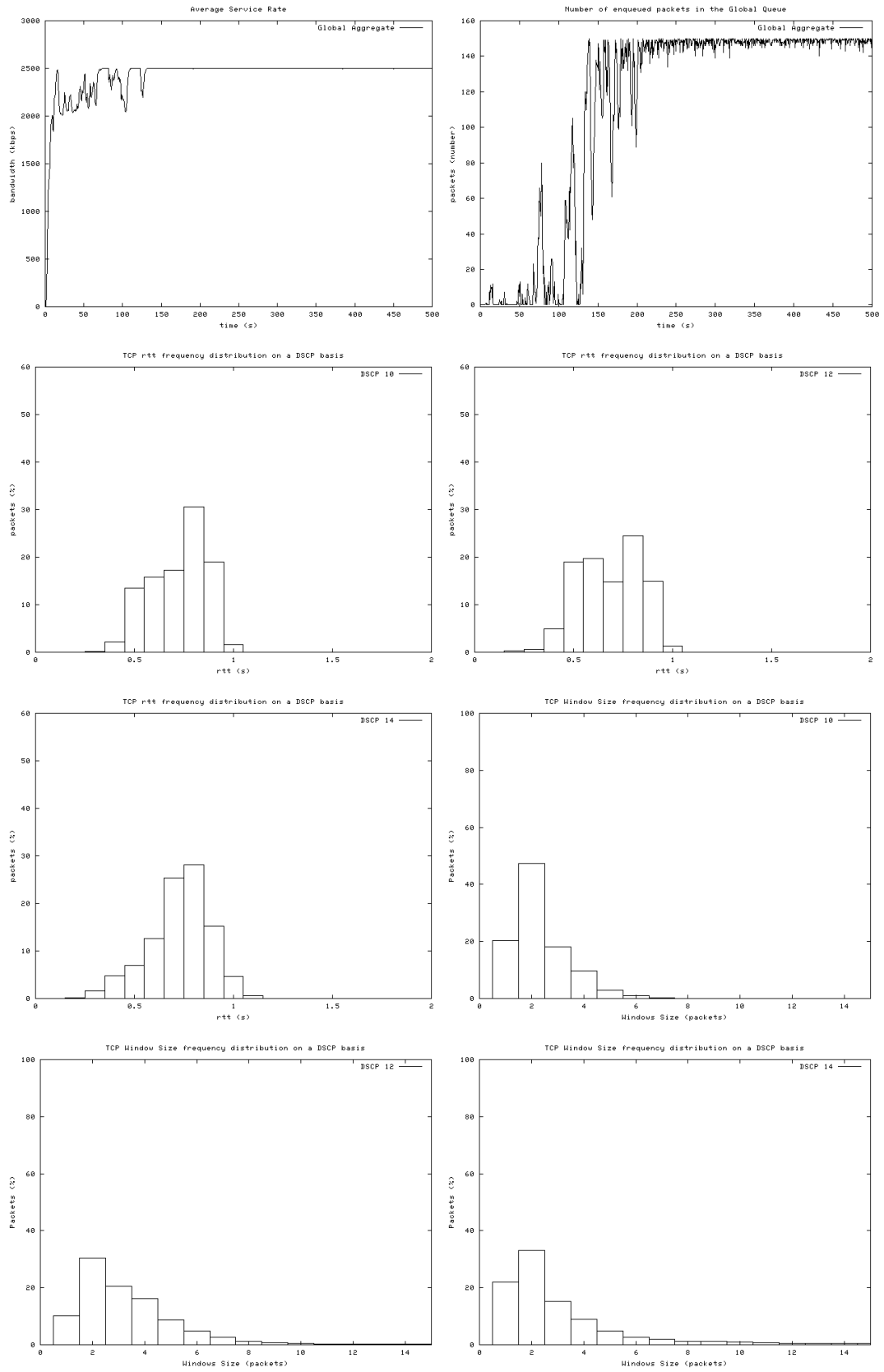


Figure B.4 Global parameters performance comparison – no DiffServ enabled

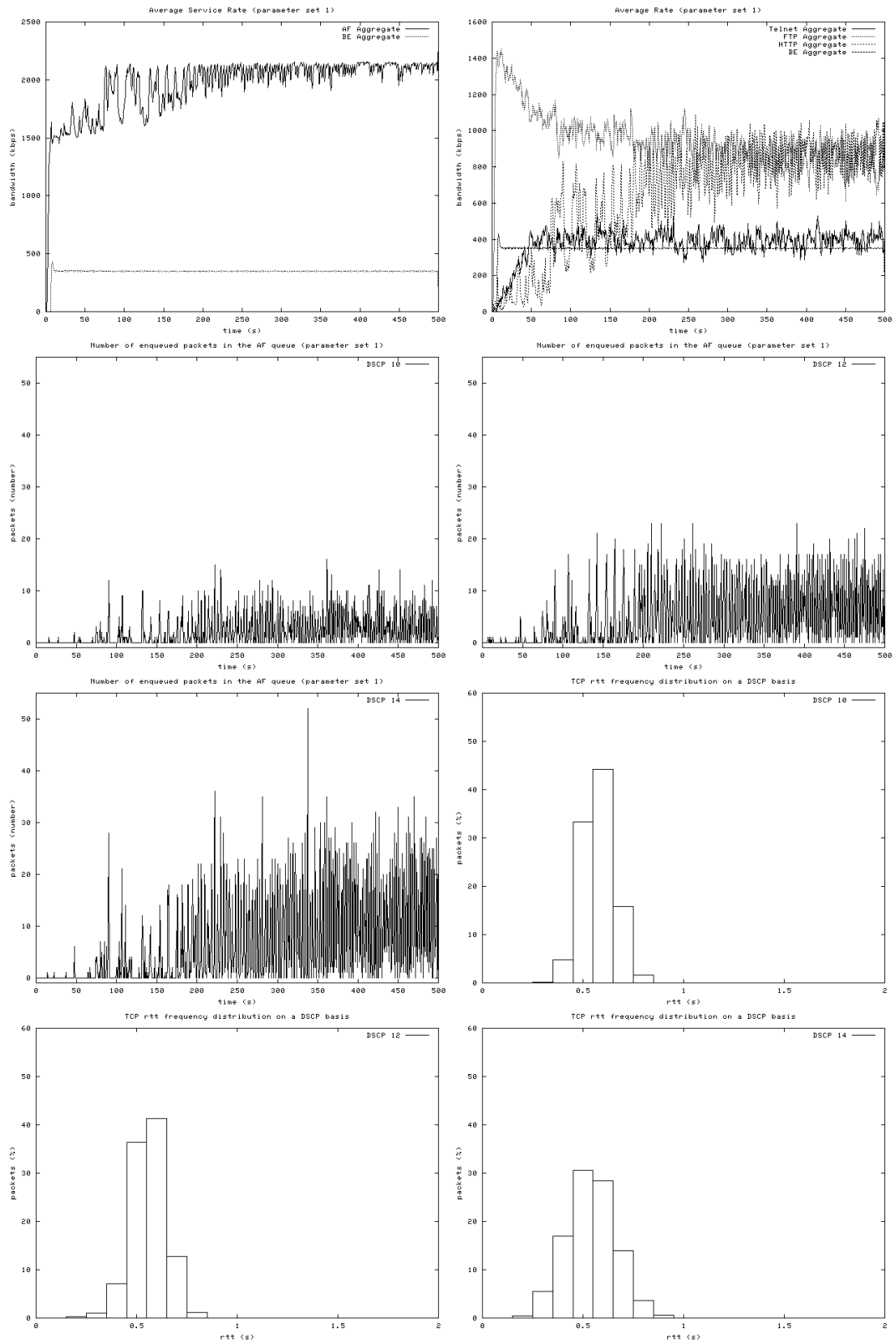


Figure B.5a WRED parameter set 1 (*staggered*)

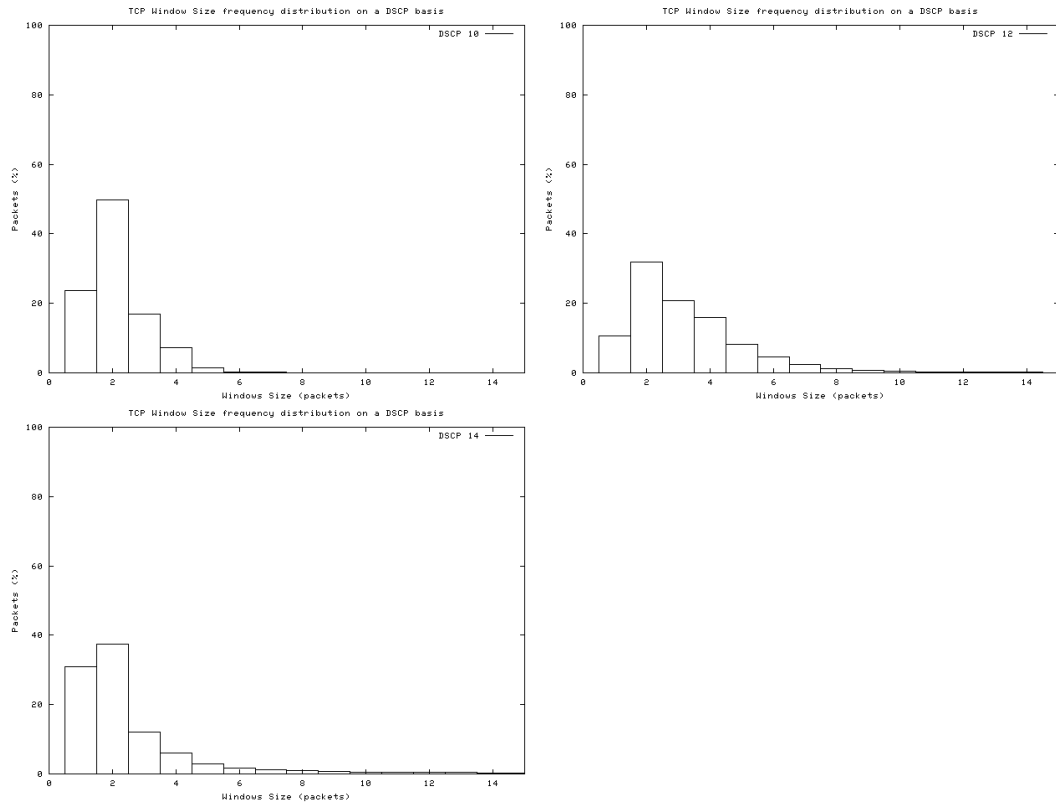


Figure B.5b WRED parameter set 1 (*staggered*)

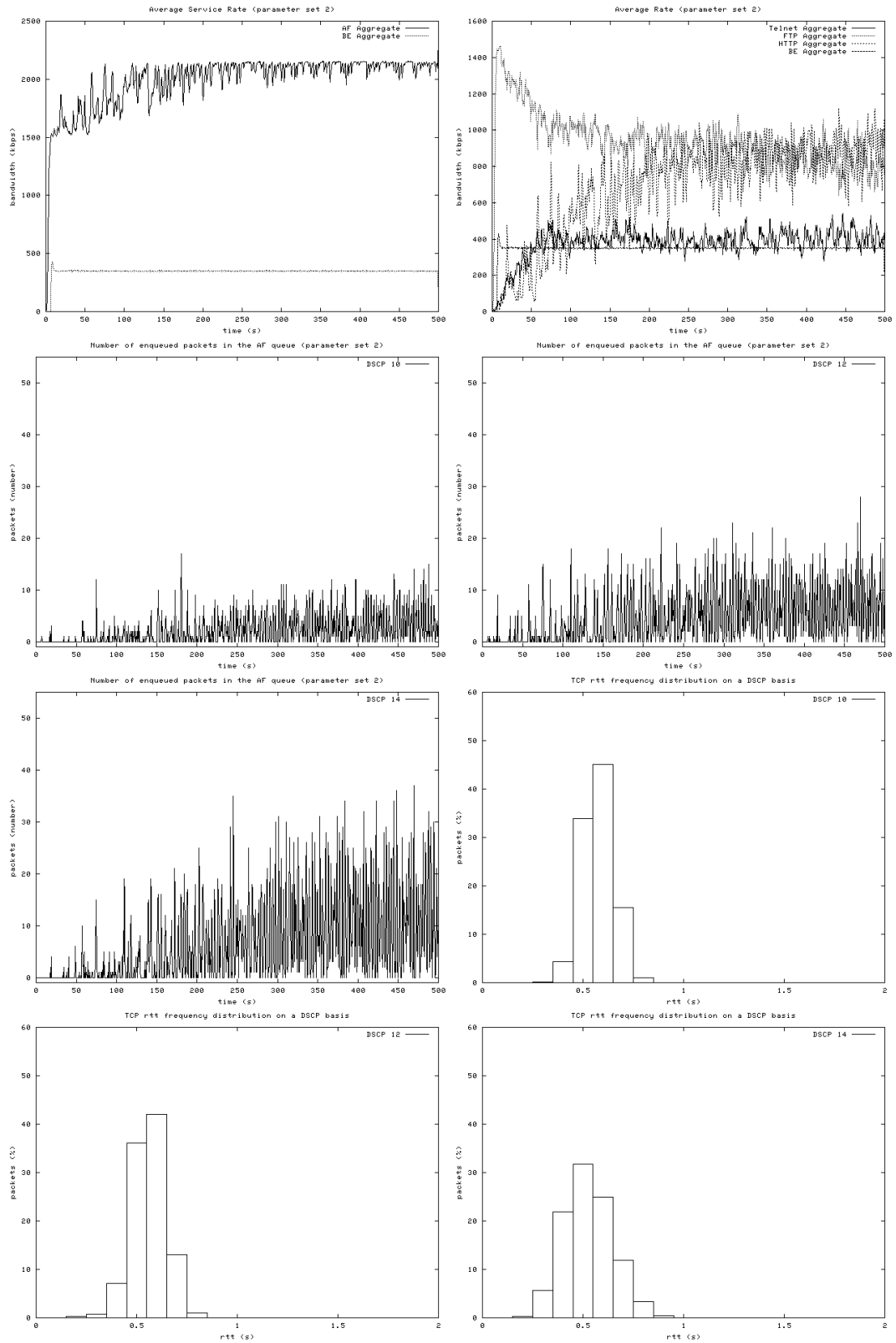


Figure B.6a WRED parameter set 2 (*staggered*)

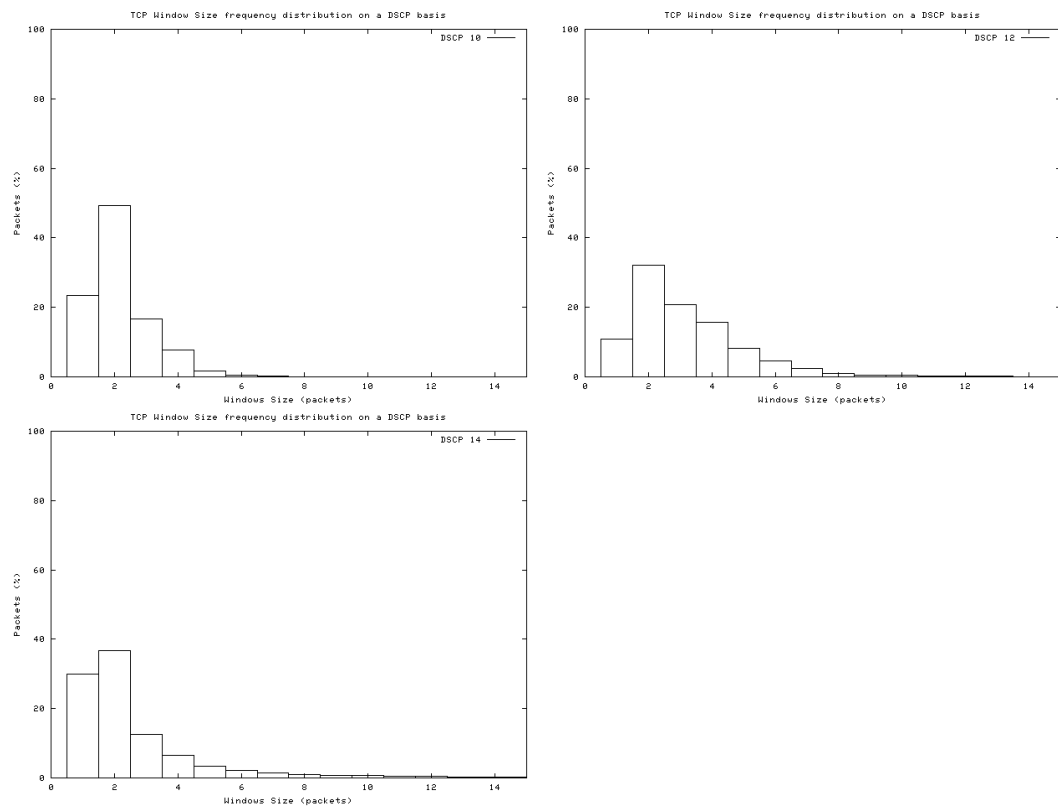


Figure B.6b WRED parameter set 2 (*staggered*)

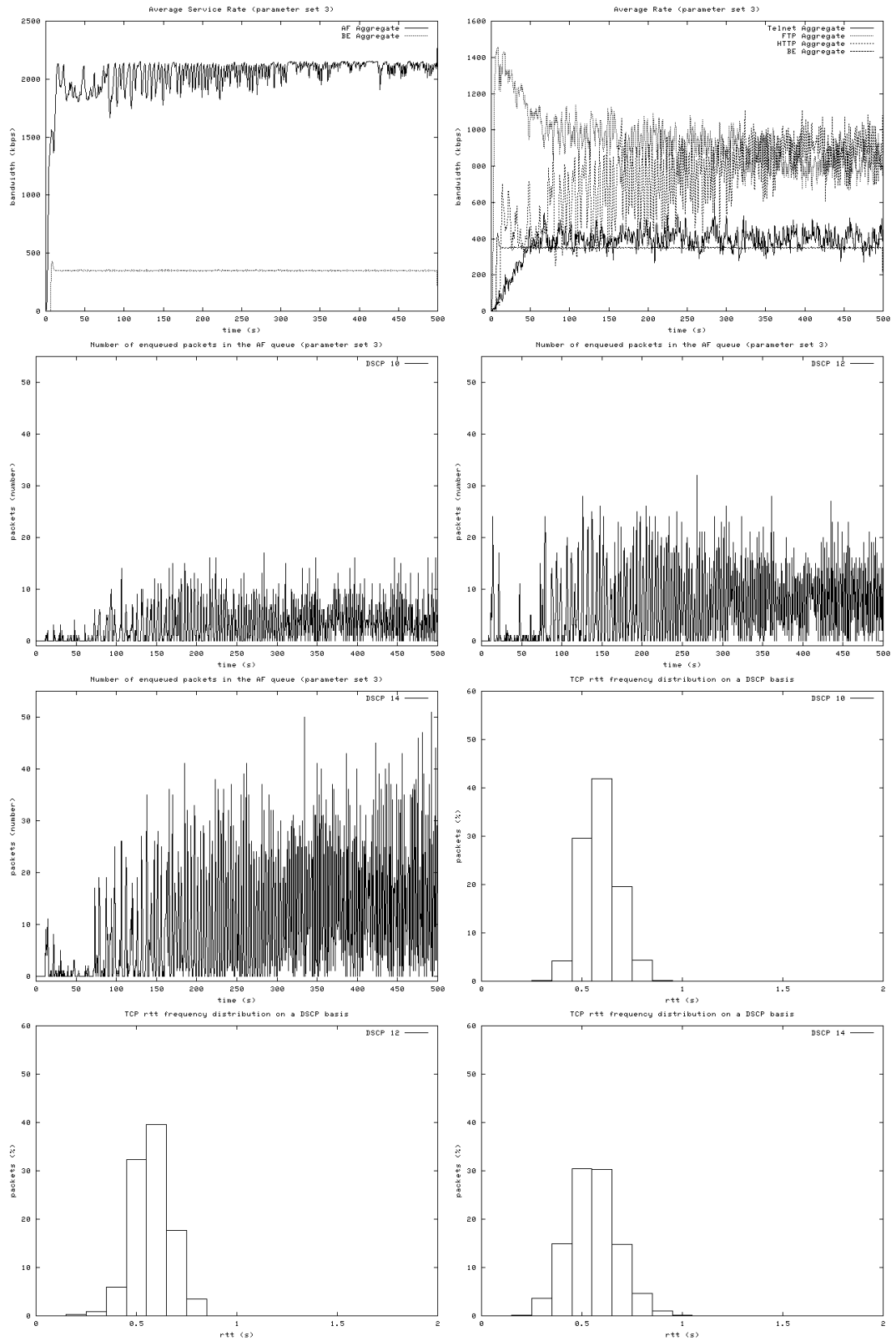


Figure B.7a WRED parameter set 3 (*partially overlapped*)

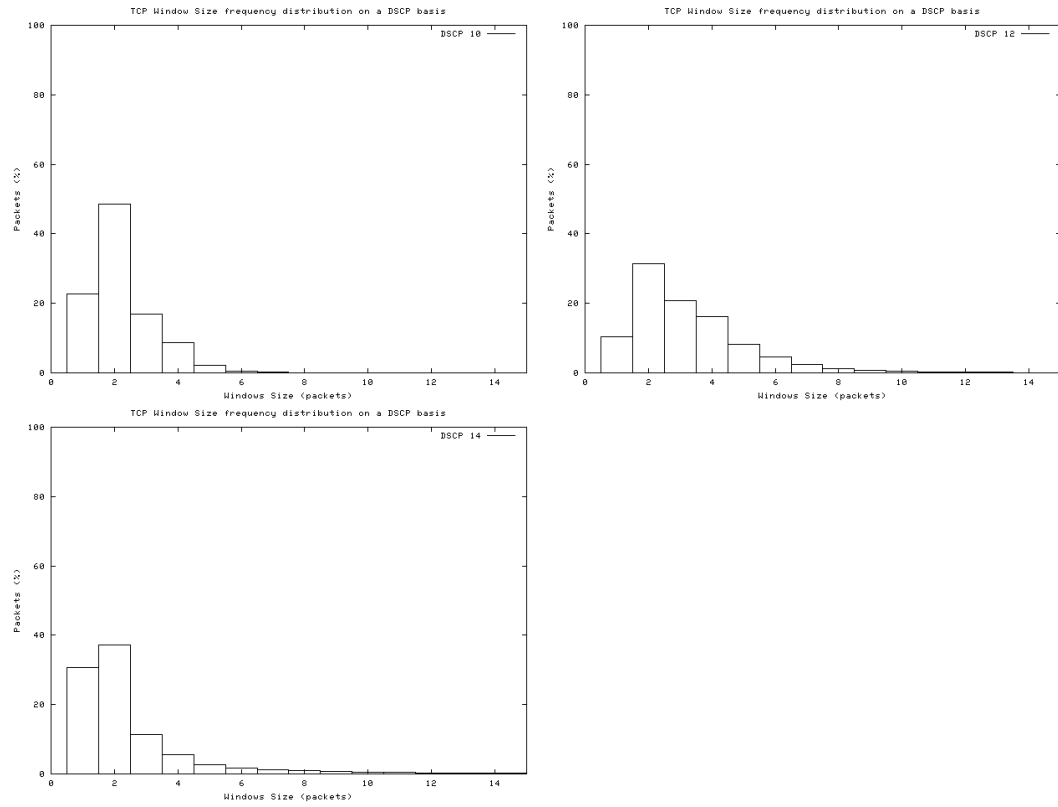


Figure B.7b WRED parameter set 3 (*partially overlapped*)

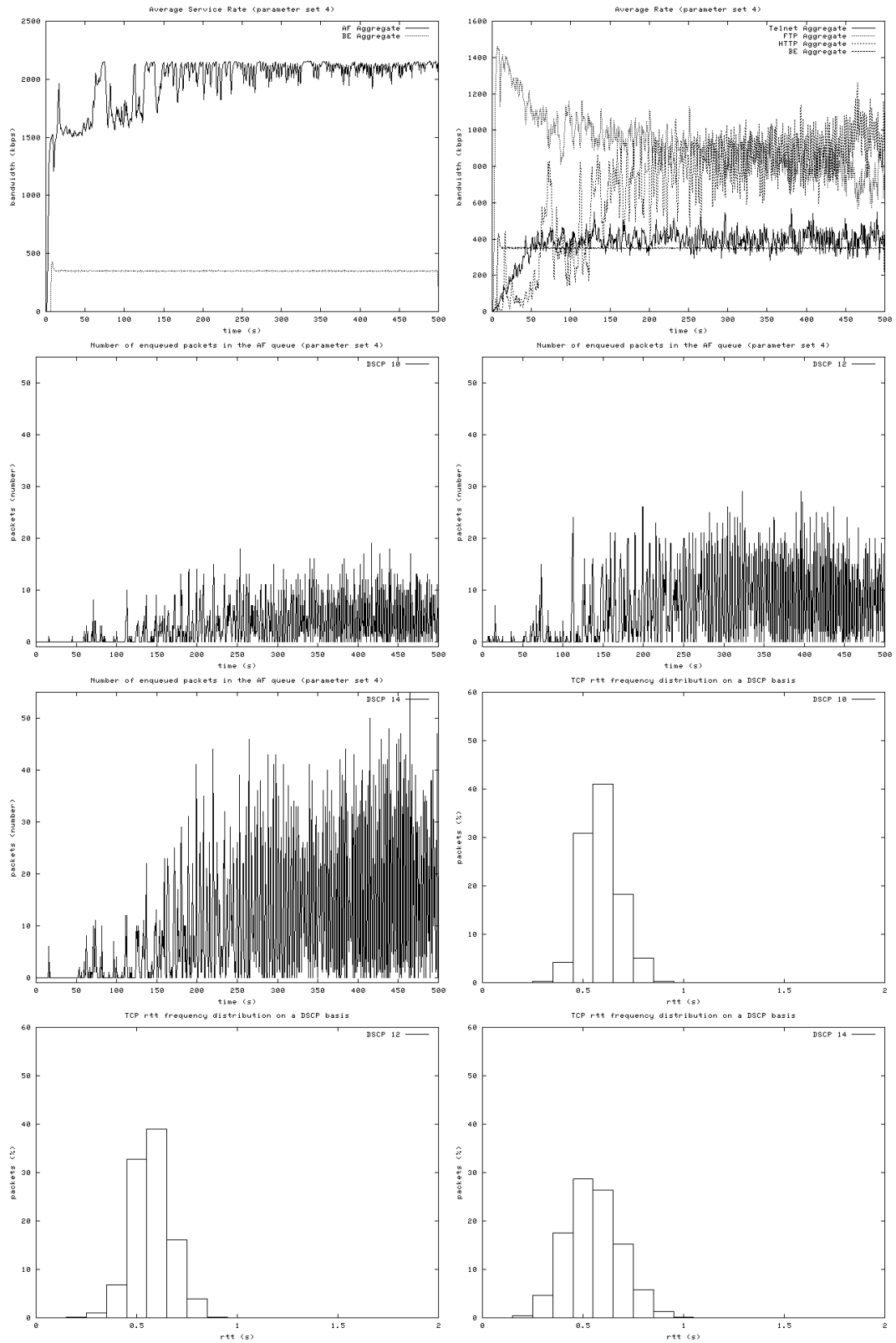


Figure B.8a WRED parameter set 4 (*partially overlapped*)

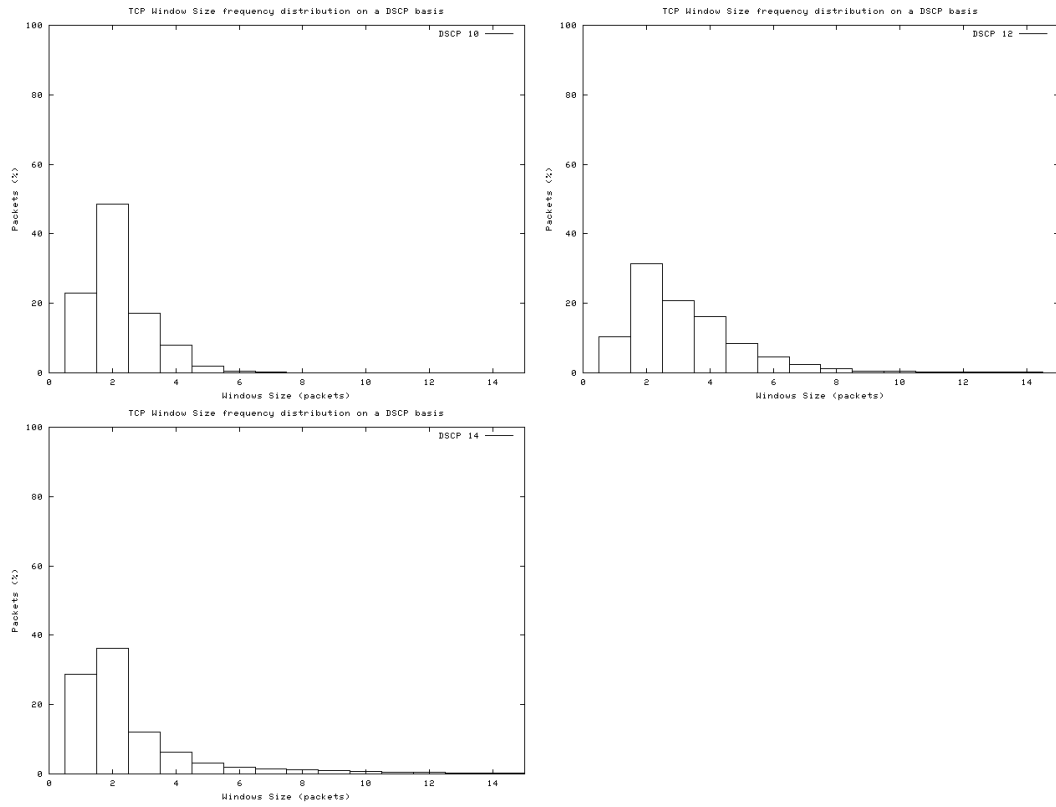


Figure B.8b WRED parameter set 4 (*partially overlapped*)

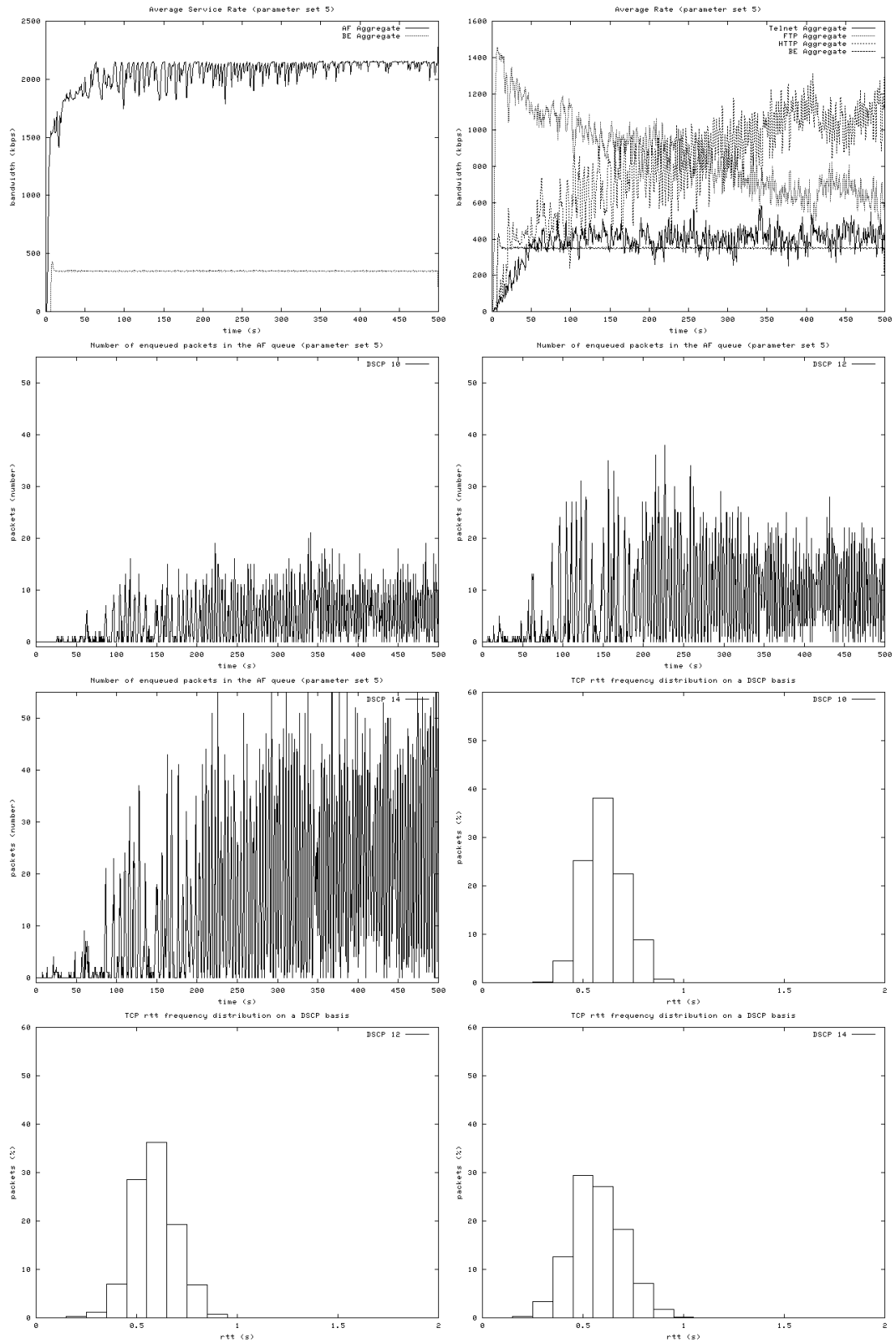


Figure B.9a WRED parameter set 5 (*overlapped*)

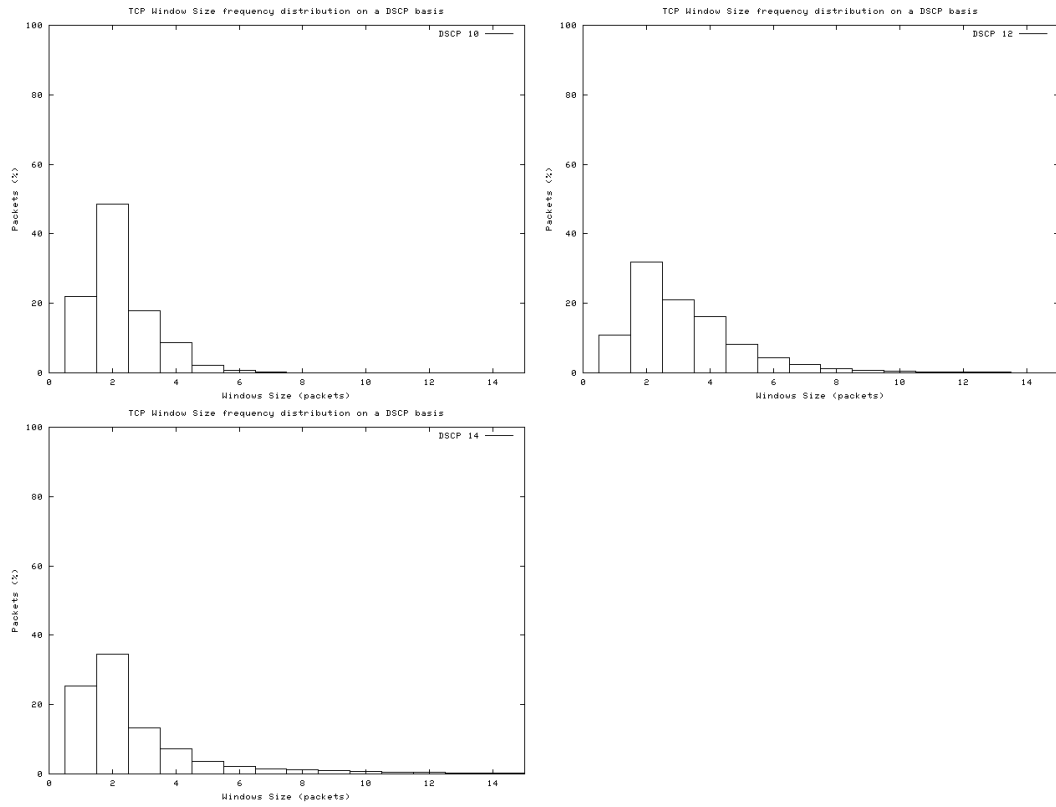


Figure B.9b WRED parameter set 5 (*overlapped*)

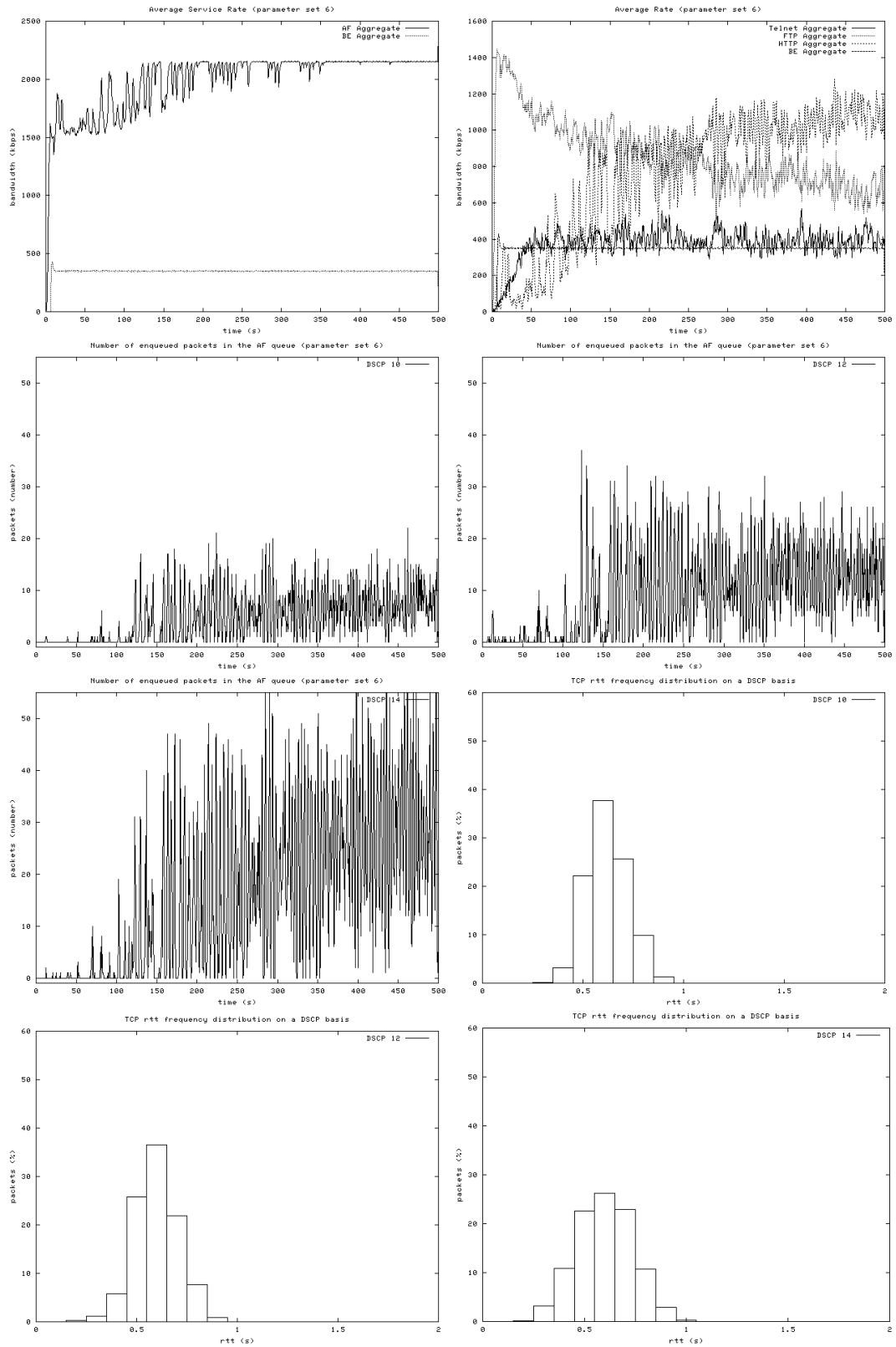


Figure B.10a WRED parameter set 6 (*overlapped*)

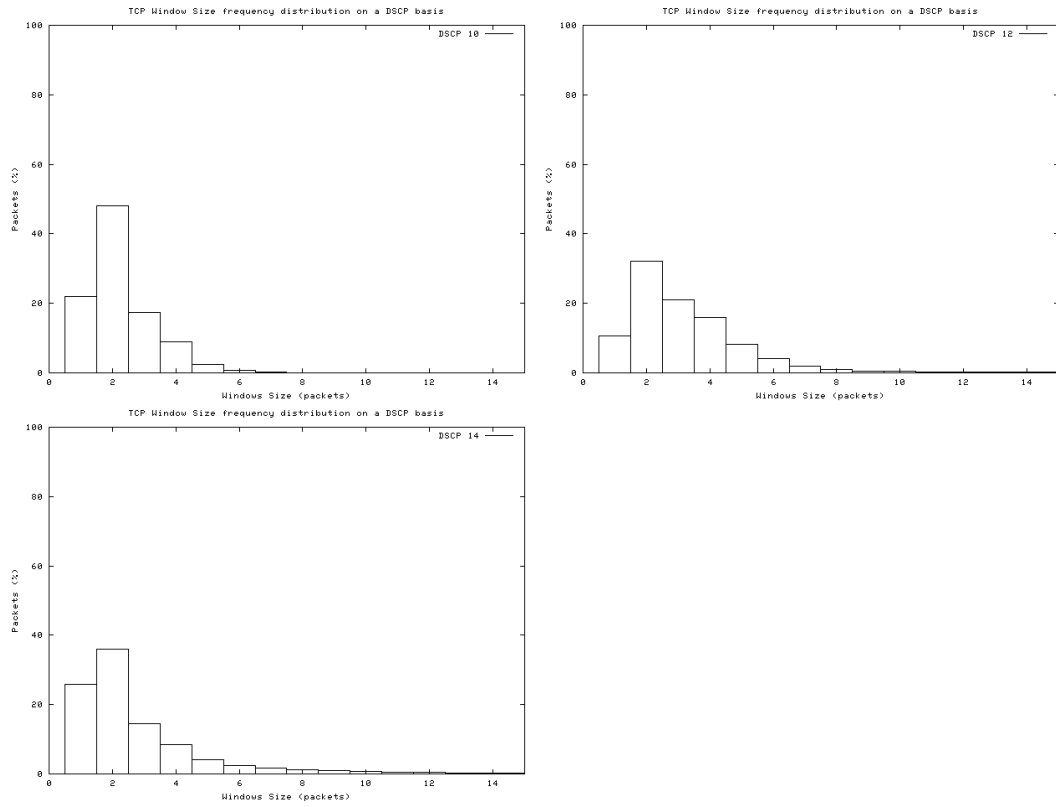


Figure B.10b WRED parameter set 6 (*overlapped*)

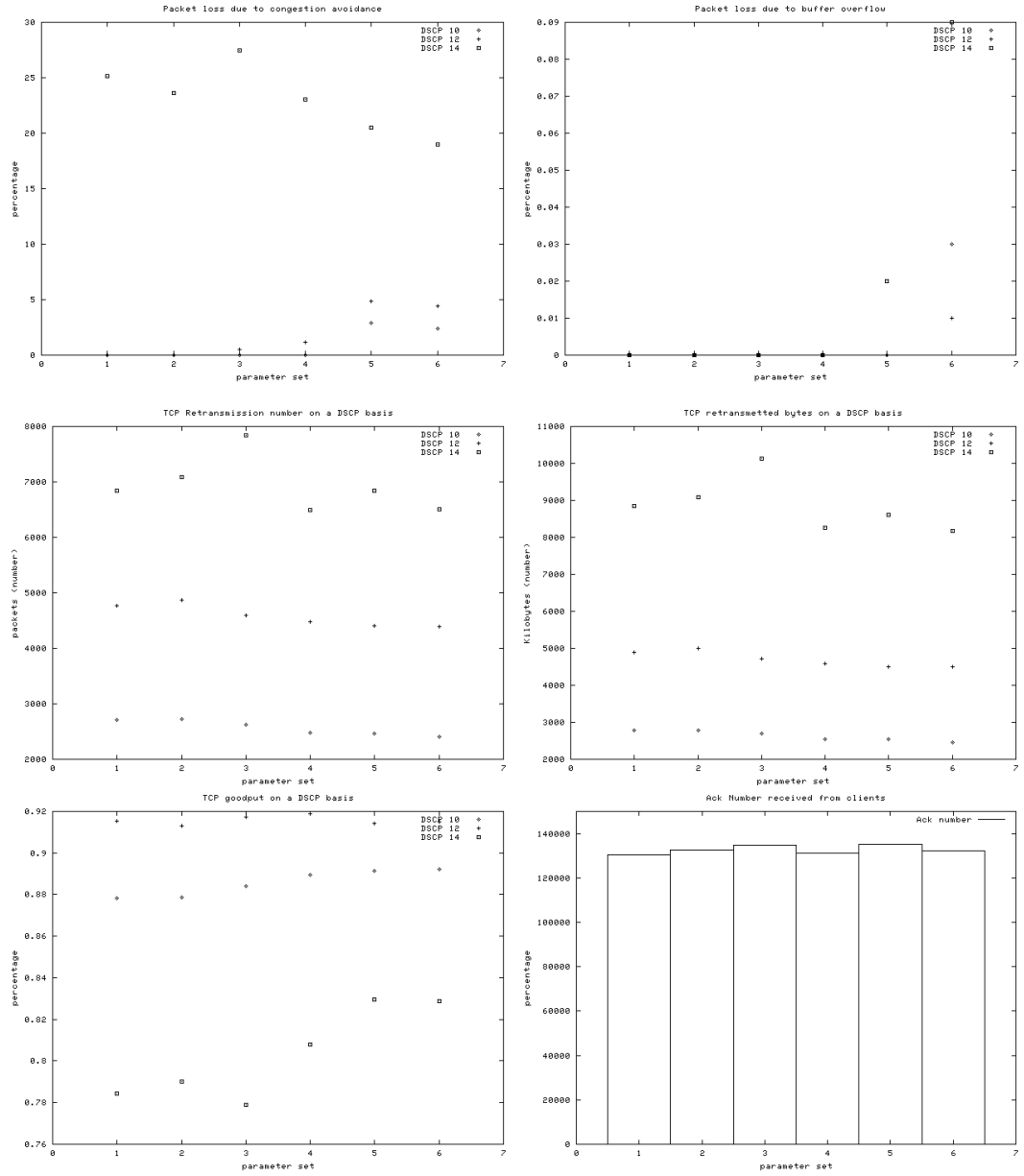


Figure B.11 Global parameters performance comparison

APPENDIX C – RESULTS DIAGRAMS FOR SCENARIO 3

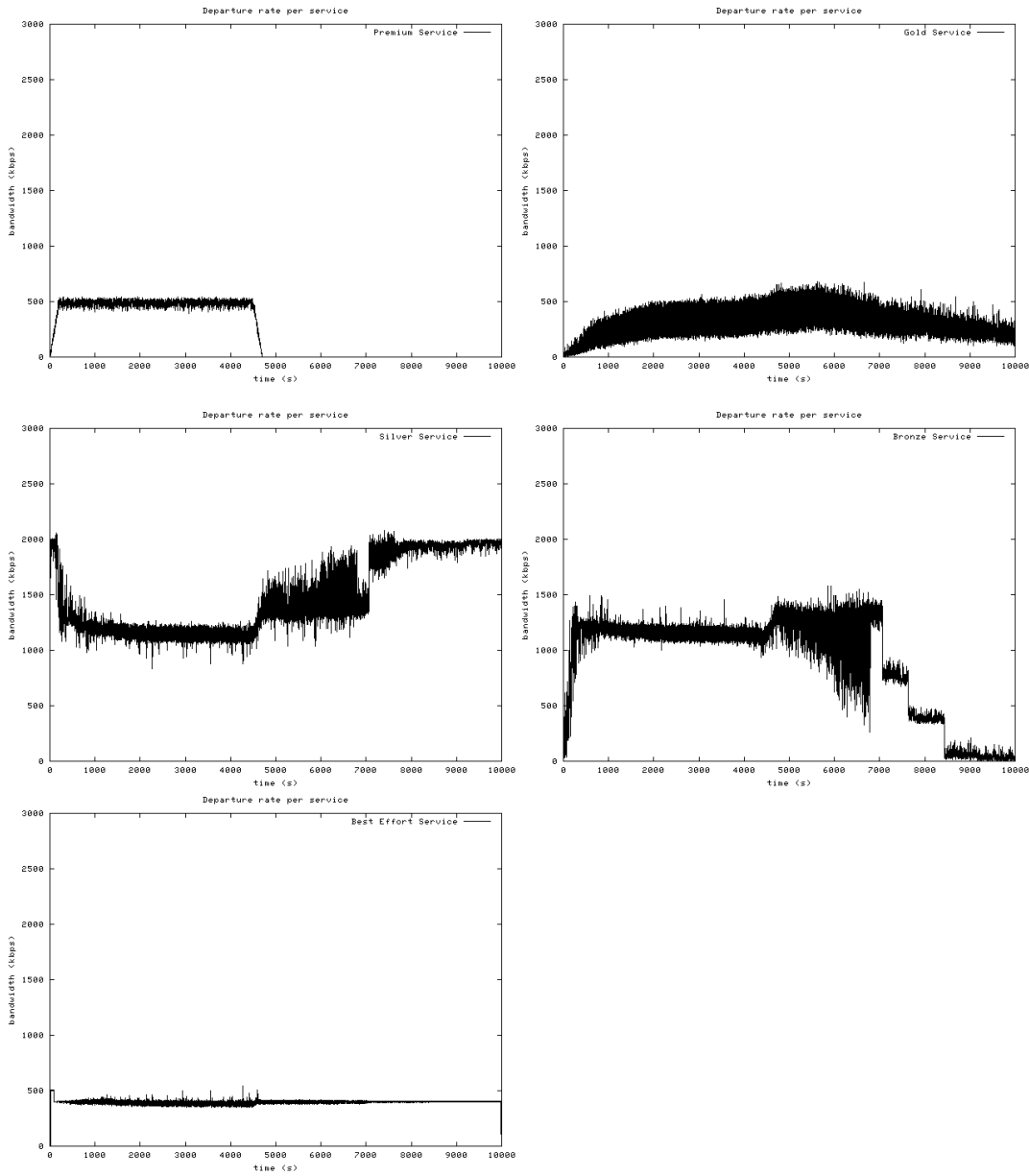


Figure C.1 Departure rate for n0->n466 egress interface per service category

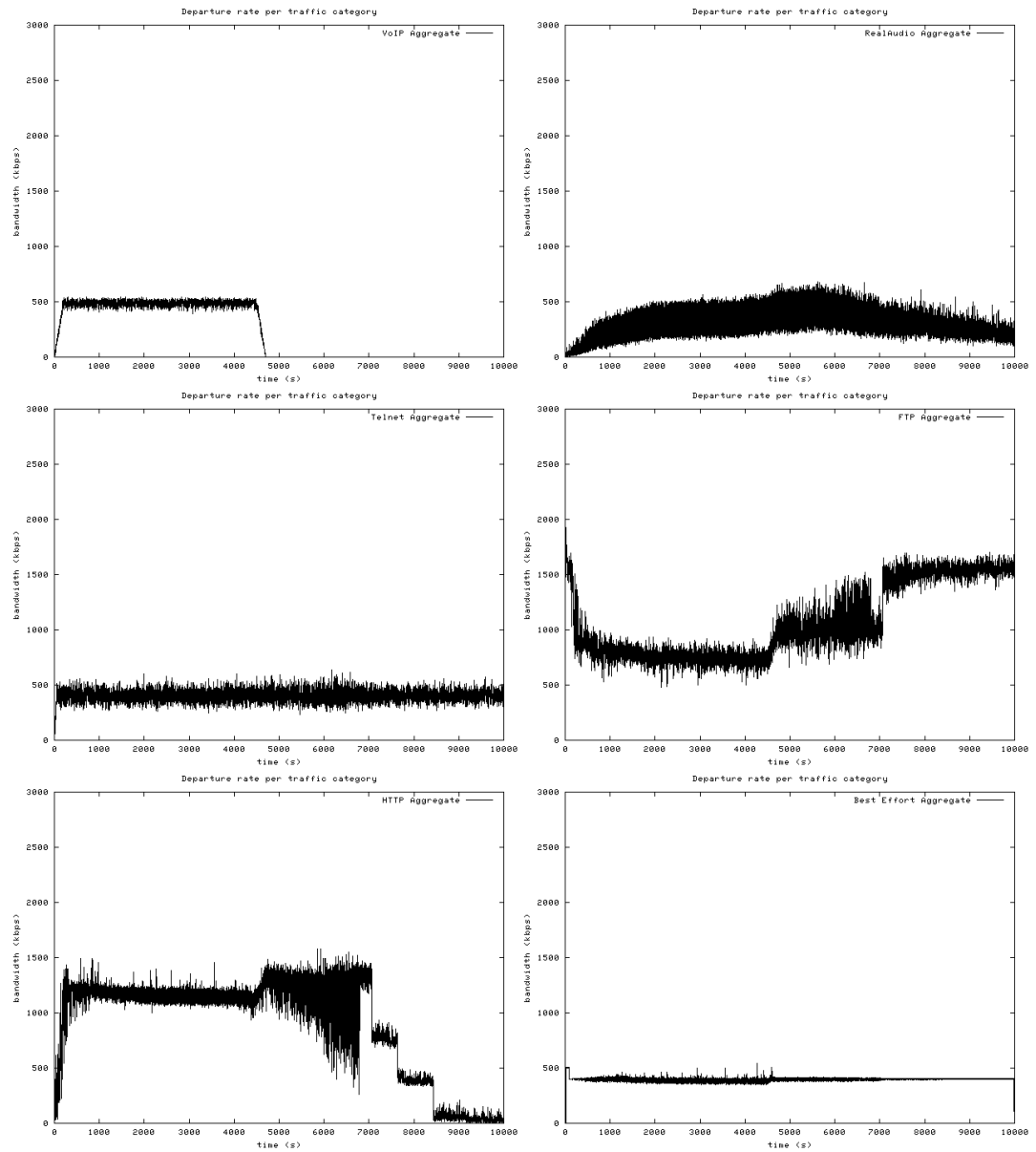


Figure C.2 Departure rate for n0->n466 egress interface per traffic category

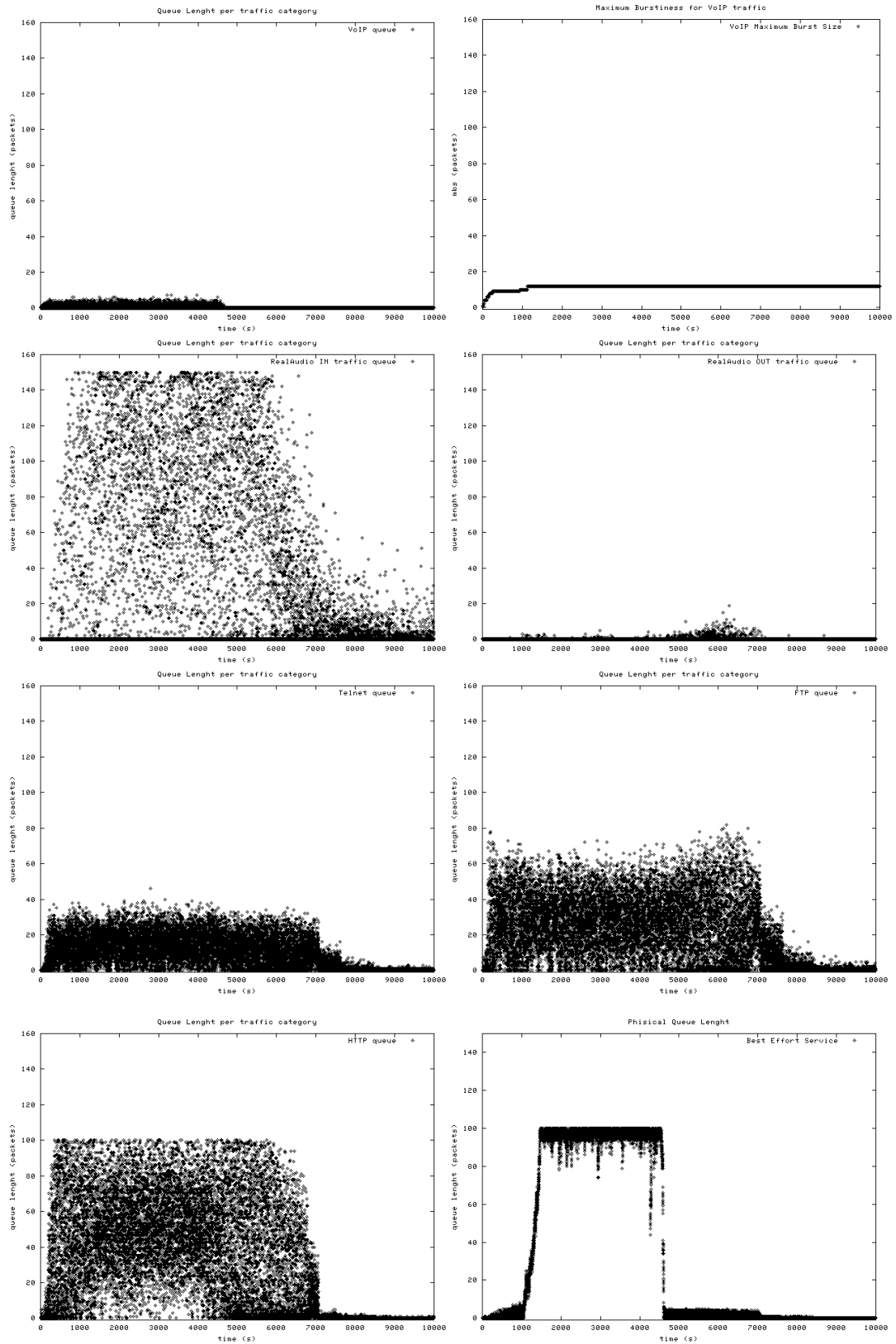


Figure C.3 Queue length per traffic category and MBS (only for VoIP)

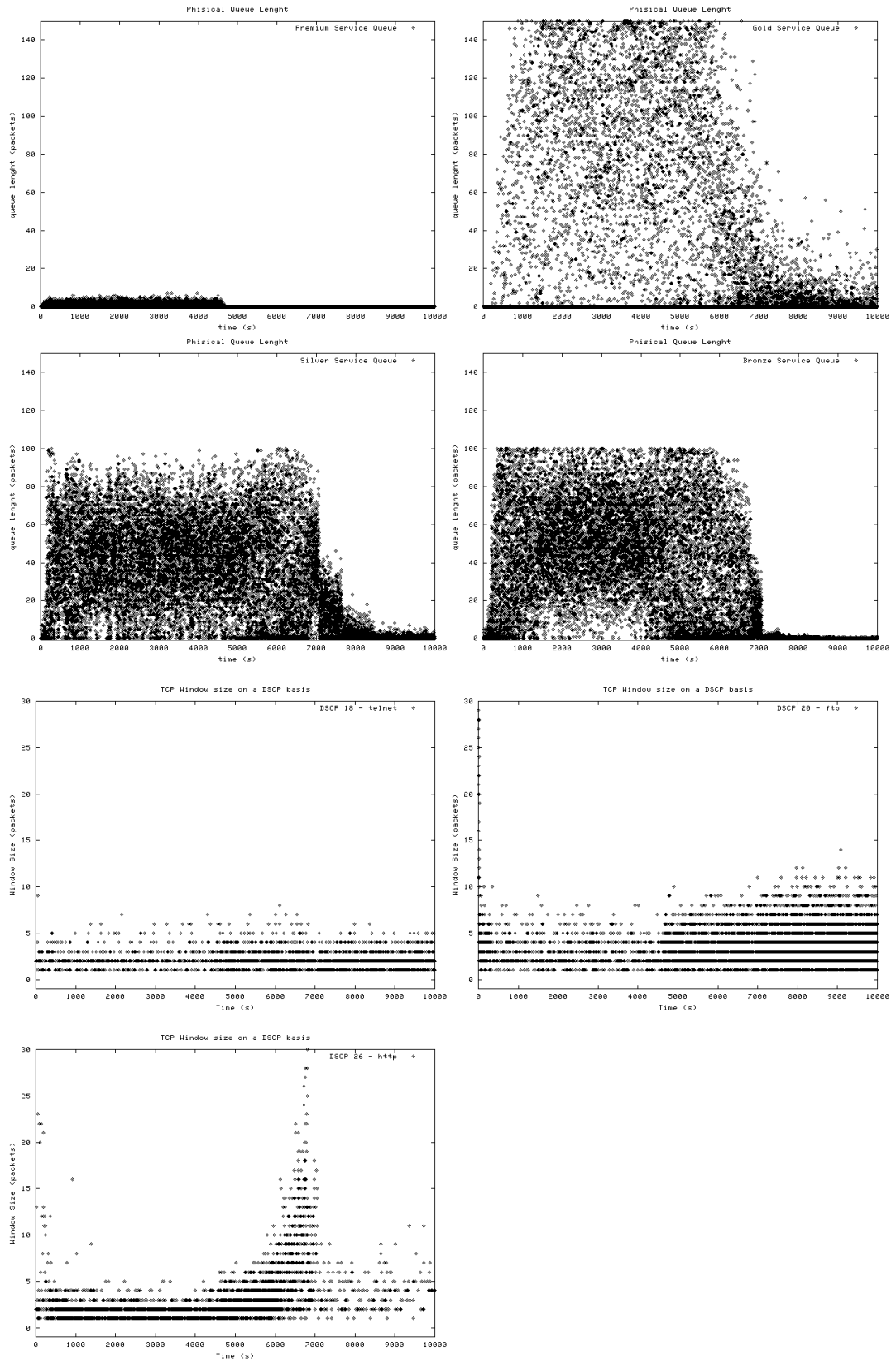


Figure C.4a Olympic Services sub-model performance parameters

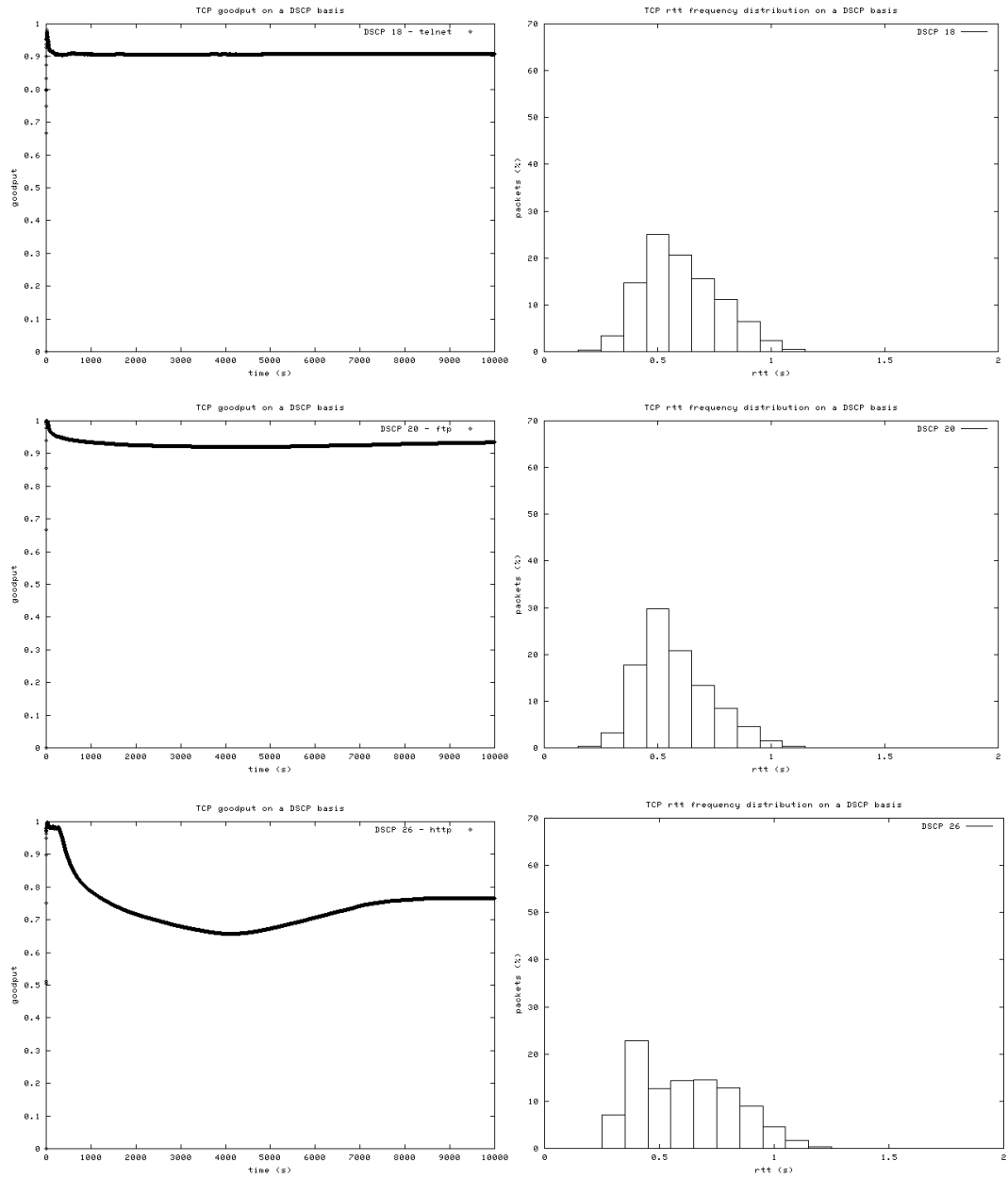


Figure C.4b Olympic Services sub-model performance parameters

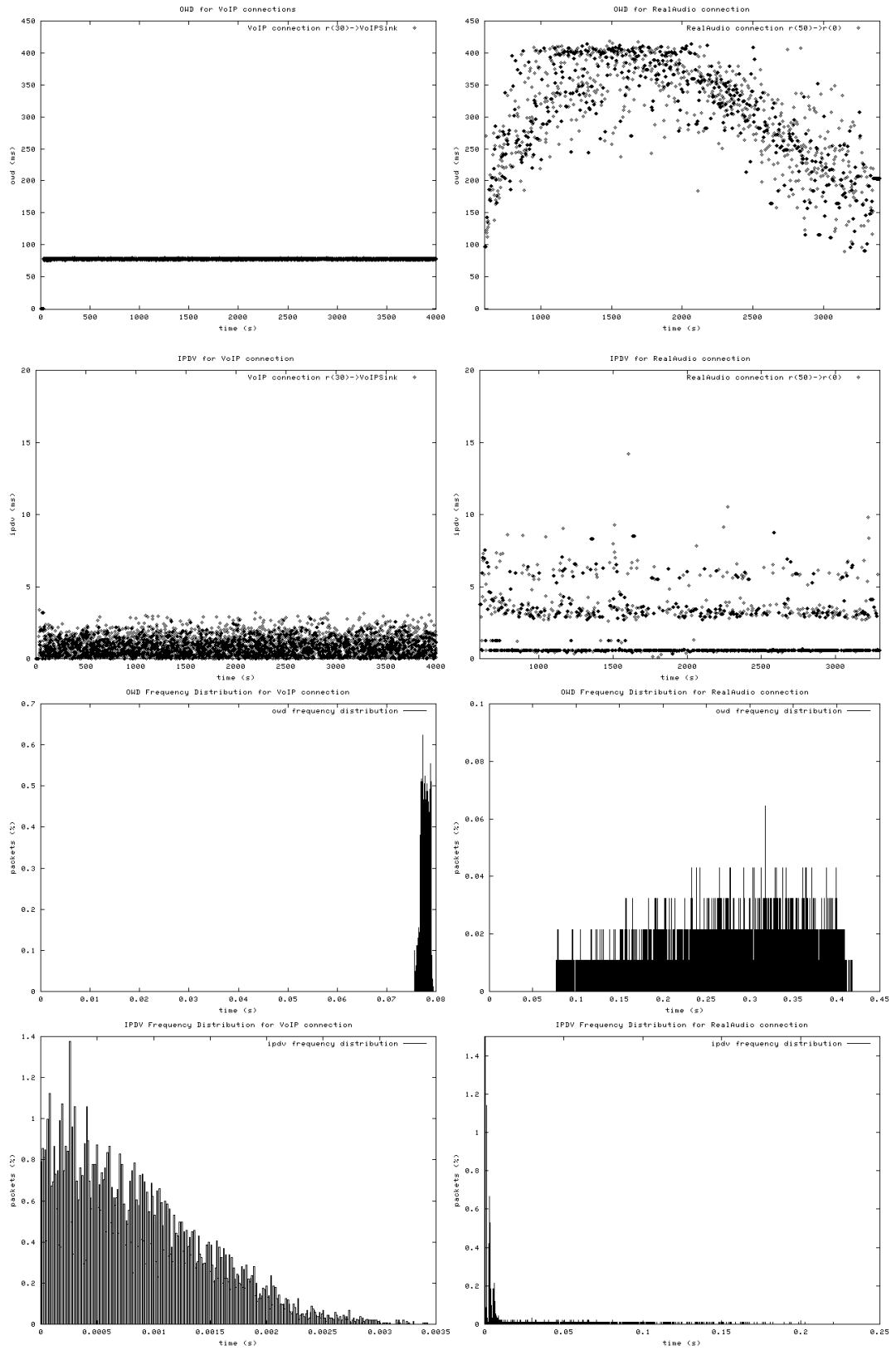


Figure C.5 OWD and IPDV comparison among VoIP and RealAudio (one monitored connection per traffic category)

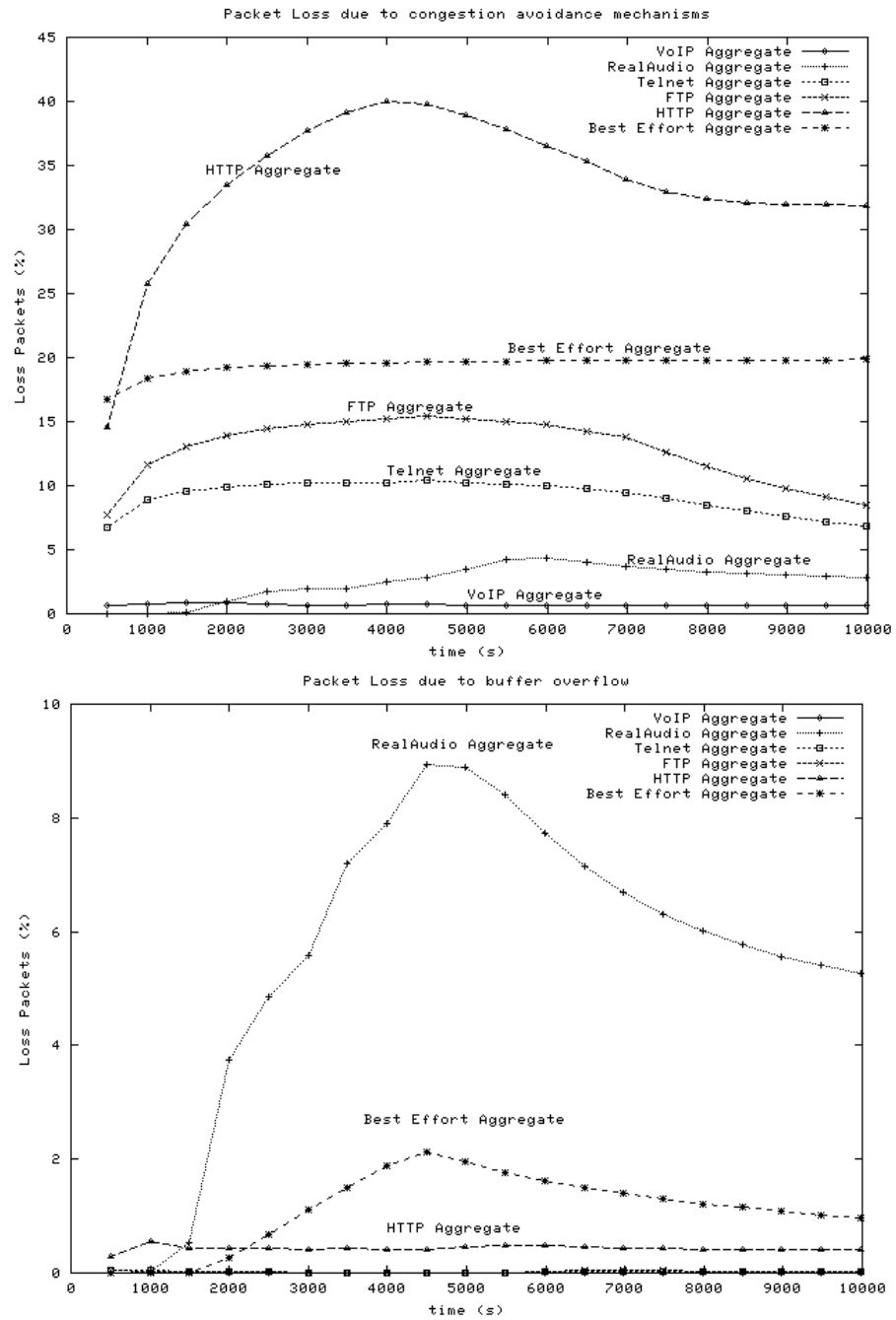


Figure C.6 caPL and boPL per traffic category